DP

Dynamic Programming Introduction

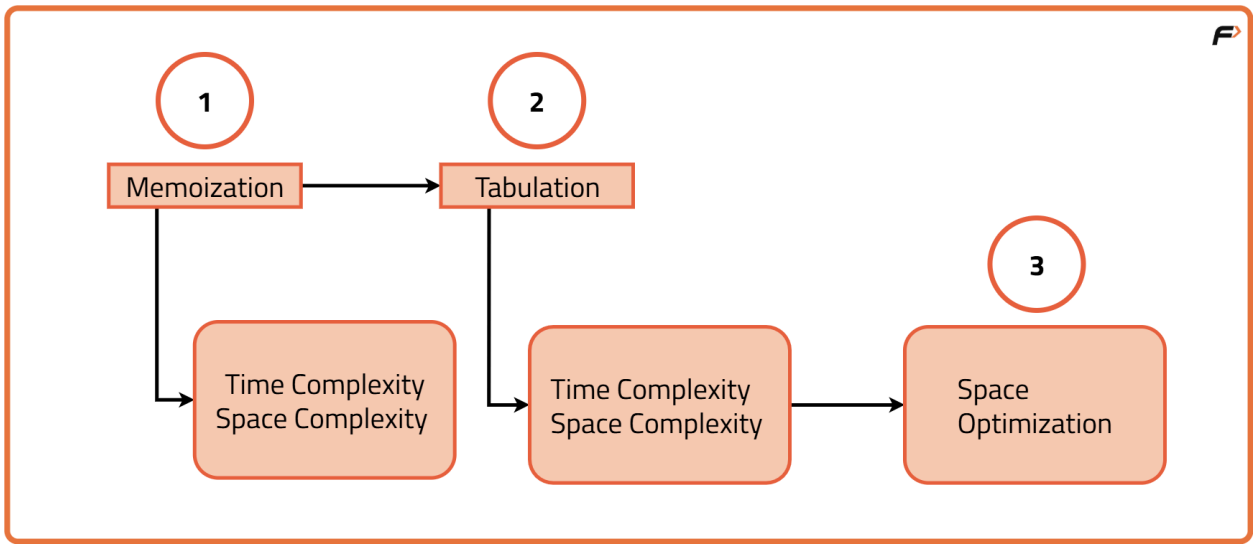
Problem Statement: Introduction To Dynamic Programming

Dynamic Programming can be described as storing answers to various sub-problems to be used later whenever required to solve the main problem.

The two common dynamic programming approaches are:

- Memoization: Known as the “top-down” dynamic programming, usually the problem is solved in the direction of the main problem to the base cases.
- Tabulation: Known as the “bottom-up " dynamic programming, usually the problem is solved in the direction of solving the base cases to the main problem

Note: The base case does not always mean smaller input. It depends upon the implementation of the algorithm.



We will be using the example of Fibonacci numbers here. The following series is called the Fibonacci series:

0,1,1,2,3,5,8,13,21,...

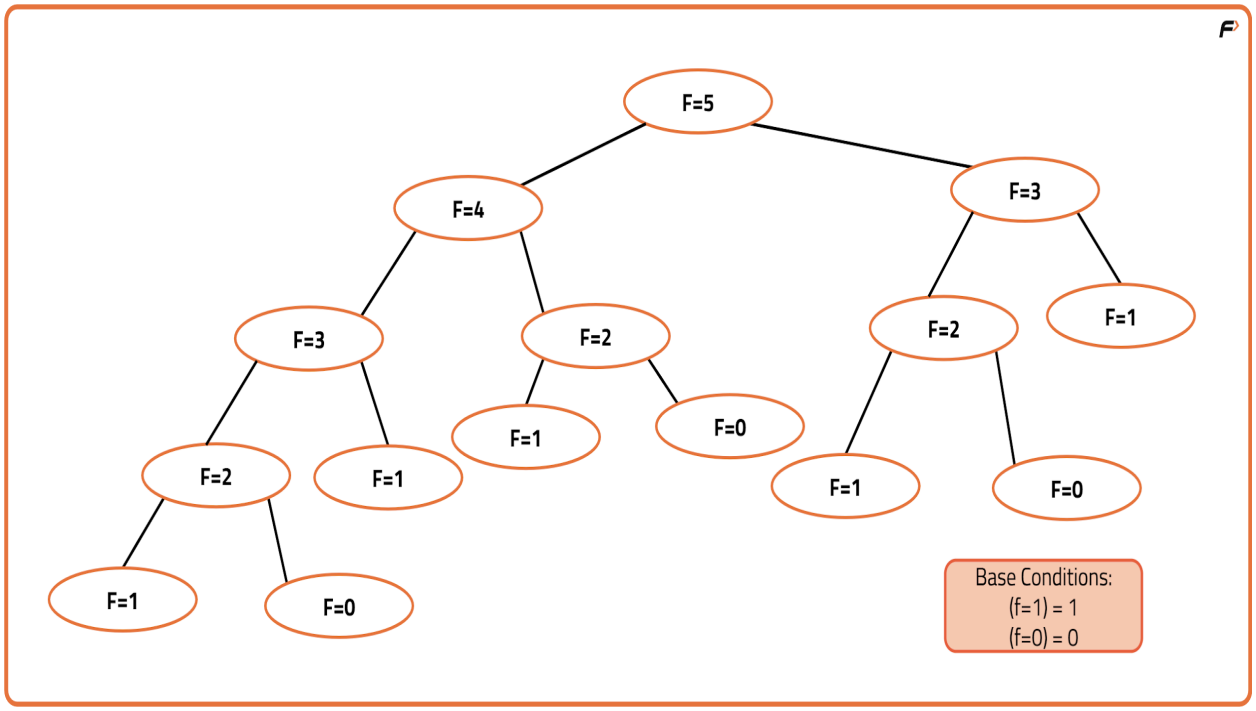
We need to find the nth Fibonacci number, where n is based on a 0-based index. Every ith number of the series is equal to the sum of (i-1)th and (i-2)th number where the first and second number is given as 0 and 1 respectively.

Part - 1: Memoizaton

As every number is equal to the sum of the previous two terms, the recurrence relation can be written as:

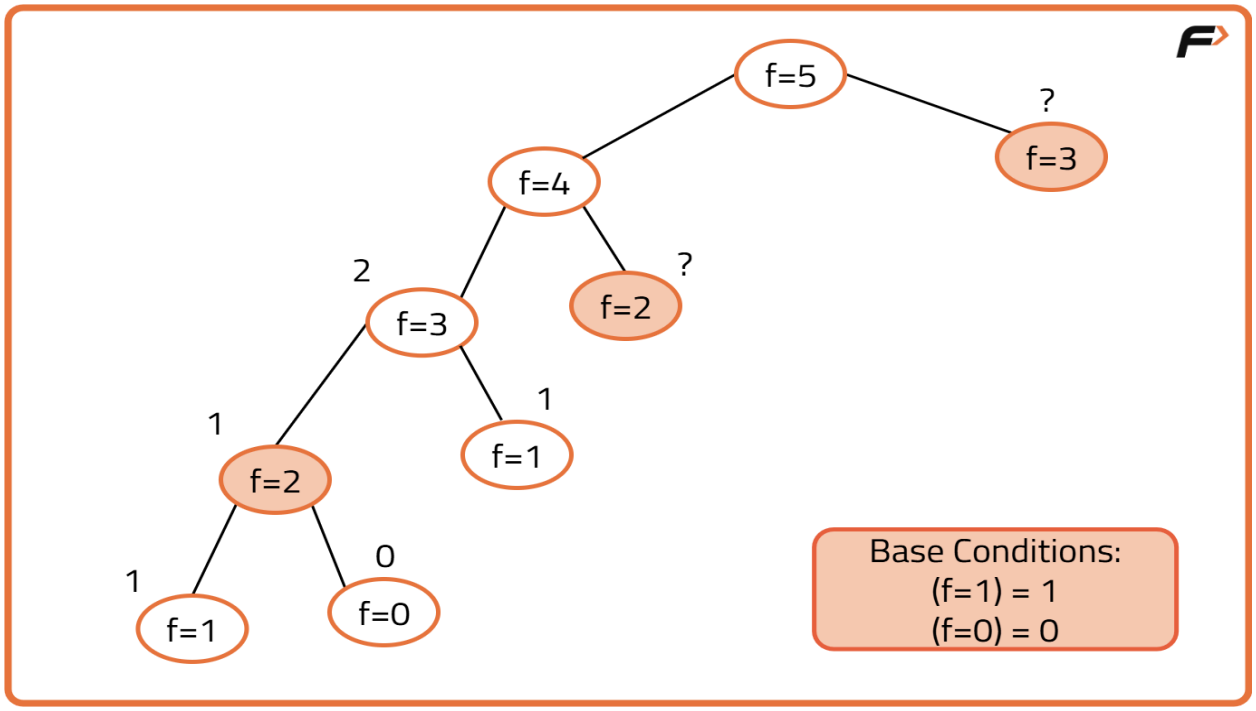
$$f(n) = f(n-1) + f(n-2)$$

If we draw the recursive tree for n=5, it will be:



If there are two recursive calls inside a function, the program will run the first call, finish its execution and then run the second call. Due to this reason, each and every call in the recursive tree will be executed. This gives the recursive code its exponential time complexity. If we can store the values of sub-problems in the first time, then we can simply find its value in constant time whenever we need

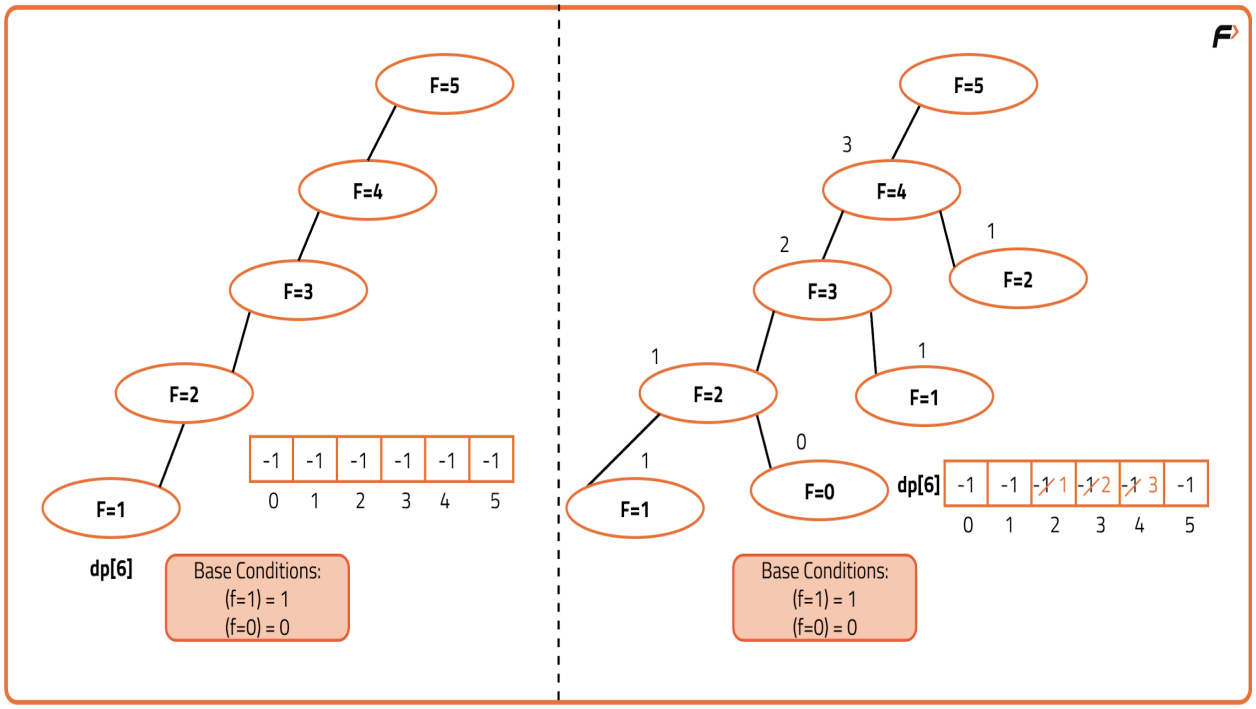
it. This technique is called **Memoization**. The cases which are solved again and again in recursion are called overlapping sub-problems.

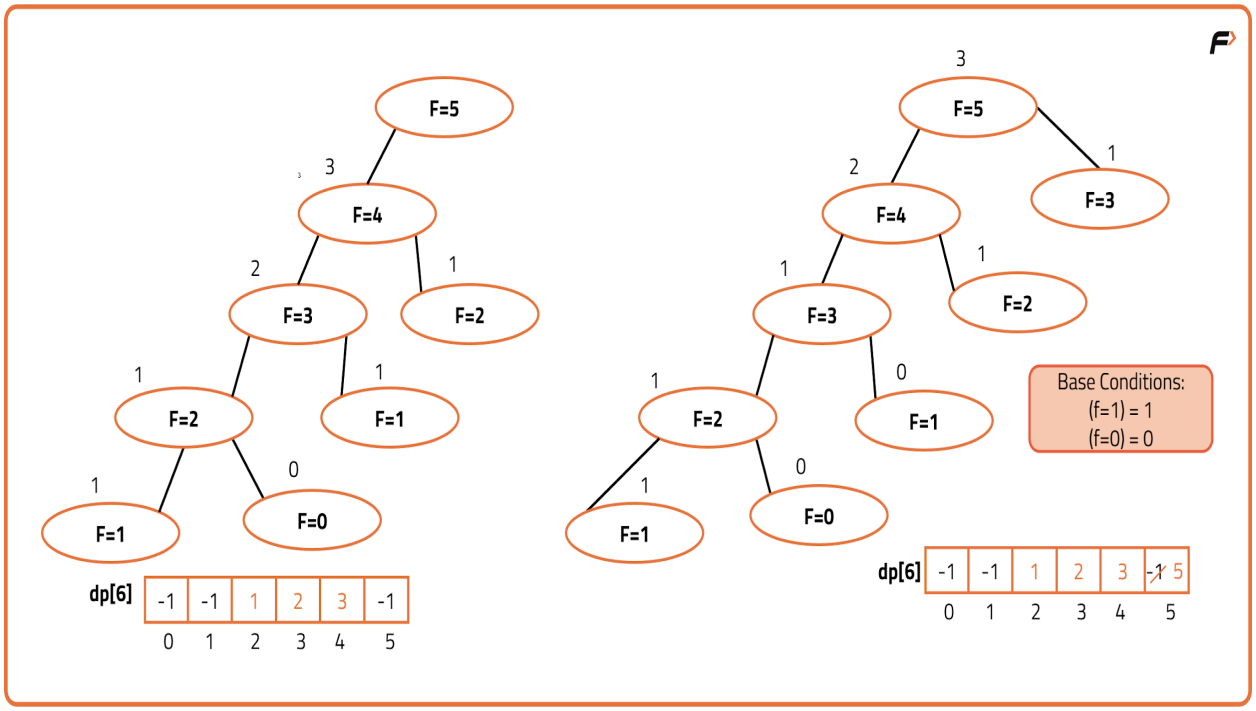


Steps to memoize a recursive solution:

Any recursive solution to a problem can be memoized using these three steps:

- Create a dp[n+1] array initialized to -1.
- Whenever we want to find the answer of a particular value (say n), we first check whether the answer is already calculated using the dp array. If yes, simply return the value from the dp array.
- If not, then we are finding the answer for the given value for the first time, we will use the recursive relation as usual but before returning from the function, we will store the solution in our dp array.






```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    // Function to calculate Fibonacci using memoization
    int fib(int n, vector<int>& dp) {
        // If base case return n
        if (n <= 1) return n;

        // If already computed, return stored value
        if (dp[n] != -1) return dp[n];

        // Otherwise compute and store
        dp[n] = fib(n - 1, dp) + fib(n - 2, dp);
        return dp[n];
    }
};

int main() {
    int n = 10;
    vector<int> dp(n + 1, -1);
    Solution sol;
    cout << sol.fib(n, dp);
    return 0;
}
```

Approach 2: Tabulation

Tabulation is a bottom-up approach where we **start from base cases** and build the solution iteratively. This avoids recursion and extra stack space.

Algorithm

1. Create a `dp` array of size `n+1`.
2. Initialize `dp[0] = 0` and `dp[1] = 1`.
3. Use a loop from 2 to `n`.
4. Set `dp[i] = dp[i-1] + dp[i-2]`.
5. Return `dp[n]`.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int fib(int n) {
        if (n <= 1) return n;
        vector<int> dp(n + 1, 0);
        dp[0] = 0;
        dp[1] = 1;
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
};

int main() {
    int n = 10;
    Solution sol;
    cout << sol.fib(n);
    return 0;
}
```

Complexity Analysis

- **Time Complexity**: O(n)
  Reason: Single loop from 2 to n.

- **Space Complexity**: O(n)
  Reason: dp array of size n+1.

Approach 3: Space Optimization

From the relation:
`dp[i] = dp[i-1] + dp[i-2]`

We observe that we only need the **previous two values**, not the entire dp array.

Algorithm

1. Handle base cases `n = 0` and `n = 1`.
2. Use two variables:
    - `prev2` for `fib(i-2)`
    - `prev` for `fib(i-1)`
3. For each iteration, compute `curr = prev + prev2`.
4. Update `prev2 = prev` and `prev = curr`.
5. After the loop, return `prev`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int fib(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;

        int prev2 = 0;
        int prev = 1;
        int curr;

        for (int i = 2; i <= n; i++) {
            curr = prev + prev2;
            prev2 = prev;
            prev = curr;
        }
        return prev;
    }
```

```
};

int main() {
    Solution s;
    int n = 10;
    cout << s.fib(n);
    return 0;
}
```

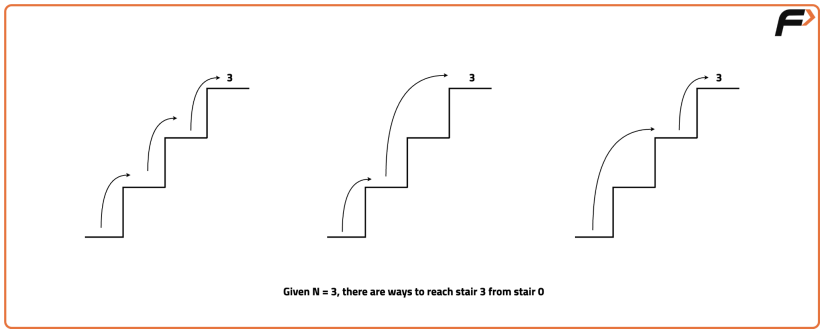
Complexity Analysis

- **Time Complexity**: O(n)
  Reason: Single loop from 2 to n.

- **Space Complexity**: O(1)
  Reason: Only constant extra variables are used.

Dynamic Programming: Climbing Stairs

You start at stair **0** and want to reach stair **n**.
At each move, you can climb **1 step** or **2 steps**.
Return the **total number of distinct ways** to reach stair n.



Key Insight

This problem is identical to a **Fibonacci-style recurrence**.

To reach stair n:

- You must come from n-1 (1 step), or
- From n-2 (2 steps)

So,

`ways(n) = ways(n-1) + ways(n-2)`

Base Cases

- `ways(0) = 1` → one way (do nothing)
- `ways(1) = 1` → one way (1 step)

Approach 1: Tabulation (Bottom-Up DP)

Algorithm

1. Create a dp array of size n+1
2. Initialize:
    - `dp[0] = 1`
    - `dp[1] = 1`

For `i = 2 → n`:

```
dp[i] = dp[i-1] + dp[i-2]
```

3.
4. Return `dp[n]`

Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 3;

    vector<int> dp(n + 1, -1);

    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    cout << dp[n];
    return 0;
}
```

Complexity

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(n)`

Approach 2: Space Optimized DP (Recommended)

We only need the **previous two values**, not the full array.

Algorithm

1. Maintain two variables:
    - `prev2 = ways(i-2)`
    - `prev1 = ways(i-1)`

For each step:

```
curr = prev1 + prev2
```

2.
3. Shift values forward
4. Return final value

Code

```
#include <bits/stdc++.h>
using namespace std;

int climbStairs(int n) {
    if (n <= 1) return 1;

    int prev2 = 1; // ways(0)
    int prev1 = 1; // ways(1)

    for (int i = 2; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}

int main() {
    int n = 3;
```

```
    cout << climbStairs(n);
    return 0;
}
```

Complexity

- **Time Complexity:** `O(n)`
- **Space Complexity:** `O(1)`

Dynamic Programming: Frog Jump (DP-3)

Problem Summary

A frog starts at stair `0` and wants to reach stair `n-1`.
From stair `i`, it can jump to:

- `i + 1`
- `i + 2`

Energy cost of a jump from `i → j`:

```
abs(height[i] - height[j])
```

Return the **minimum total energy** required to reach stair `n-1`.

Let
`dp[i]` = minimum energy required to reach stair `i`.

From stair `i`, the frog could have come from:

- `i-1`
- `i-2`

So the recurrence is:

```
dp[i] = min(
    dp[i-1] + abs(height[i] - height[i-1]),
    dp[i-2] + abs(height[i] - height[i-2])
)
```

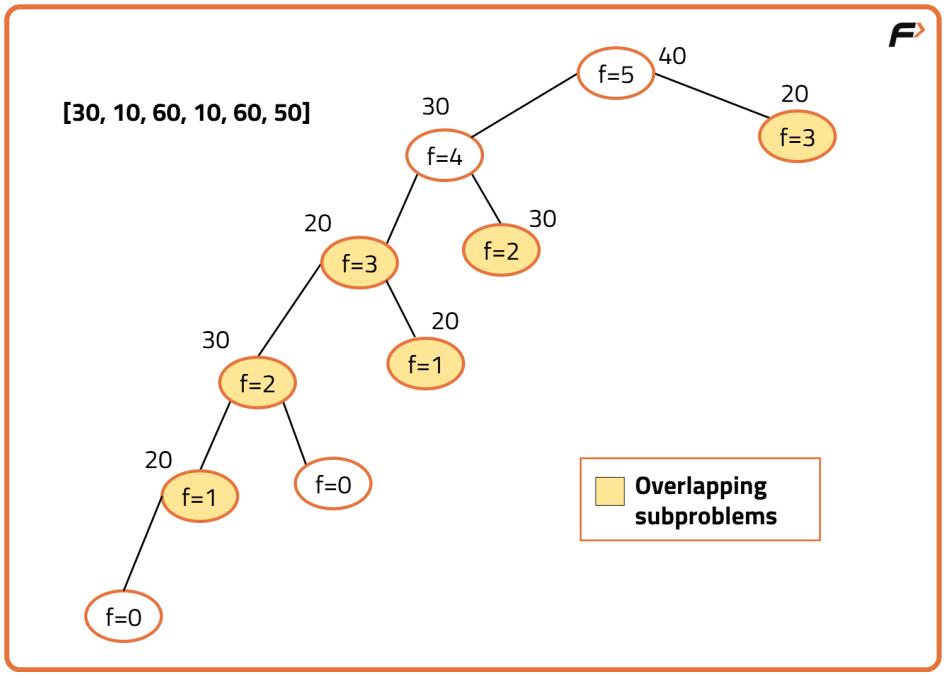
Base Case

`dp[0] = 0`

1️⃣ Memoization (Top-Down)

Algorithm

- Define a recursive function `solve(i)`
- Store already computed results in `dp[]`
- Avoid recomputation of overlapping subproblems



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i, const vector<int>& height, vector<int>& dp) {
        if (i == 0) return 0;
        if (dp[i] != -1) return dp[i];

        int jumpOne = solve(i - 1, height, dp)
                    + abs(height[i] - height[i - 1]);

        int jumpTwo = INT_MAX;
        if (i > 1) {
            jumpTwo = solve(i - 2, height, dp)
                    + abs(height[i] - height[i - 2]);
        }

        return dp[i] = min(jumpOne, jumpTwo);
    }
```

```
    int frogJump(vector<int>& height) {
        int n = height.size();
        vector<int> dp(n, -1);
        return solve(n - 1, height, dp);
    }
};
```

Complexity

- **Time:** $O(n)$
- **Space:** $O(n)$ (DP + recursion stack)

---

2 Tabulation (Bottom-Up)

Algorithm

- Build the solution iteratively
- Each state depends only on previous two states

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int frogJump(const vector<int>& height) {
        int n = height.size();
        vector<int> dp(n, INT_MAX);

        dp[0] = 0;

        for (int i = 1; i < n; i++) {
```

```
            int jumpOne = dp[i - 1] + abs(height[i] - height[i - 1]);
            int jumpTwo = INT_MAX;

            if (i > 1) {
                jumpTwo = dp[i - 2] + abs(height[i] - height[i - 2]);
            }

            dp[i] = min(jumpOne, jumpTwo);
        }
        return dp[n - 1];
    }
};
```

Complexity

- **Time:** `O(n)`
- **Space:** `O(n)`

3️⃣ Space Optimized DP Key Observation

To compute `dp[i]`, we only need:

- `dp[i-1]`
- `dp[i-2]`

So we can remove the DP array.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
```

```
    int frogJump(const vector<int>& height) {
        int n = height.size();
        if (n == 1) return 0;

        int prev = 0;   // dp[i-1]
        int prev2 = 0;  // dp[i-2]

        for (int i = 1; i < n; i++) {
            int jumpOne = prev + abs(height[i] - height[i - 1]);
            int jumpTwo = INT_MAX;

            if (i > 1) {
                jumpTwo = prev2 + abs(height[i] - height[i - 2]);
            }

            int cur = min(jumpOne, jumpTwo);
            prev2 = prev;
            prev = cur;
        }
        return prev;
    }
};
```

Complexity

- **Time:** $O(n)$
- **Space:** $O(1)$

Dynamic Programming: Frog Jump with k Distances (DP 4)

A frog wants to reach the last step of a staircase. There are `n` steps, and each step has a height given in an array `heights`. The frog starts at index `0` and wants to reach index `n-1`.

From any step `i`, the frog can jump to any step between `i+1` and `i+k` (as long as that step exists).
The energy required for a jump from step `i` to step `j` is `abs(heights[i] - heights[j])`.

The task is to find the **minimum total energy** needed to reach the last step.

Example explanation:
For `heights = [10, 5, 20, 0, 15]` and `k = 2`
The frog jumps:

- Step 0 → Step 2, cost = |10 − 20| = 10
- Step 2 → Step 4, cost = |20 − 15| = 5
  Total energy = 15

Approach 1: Memoization Approach

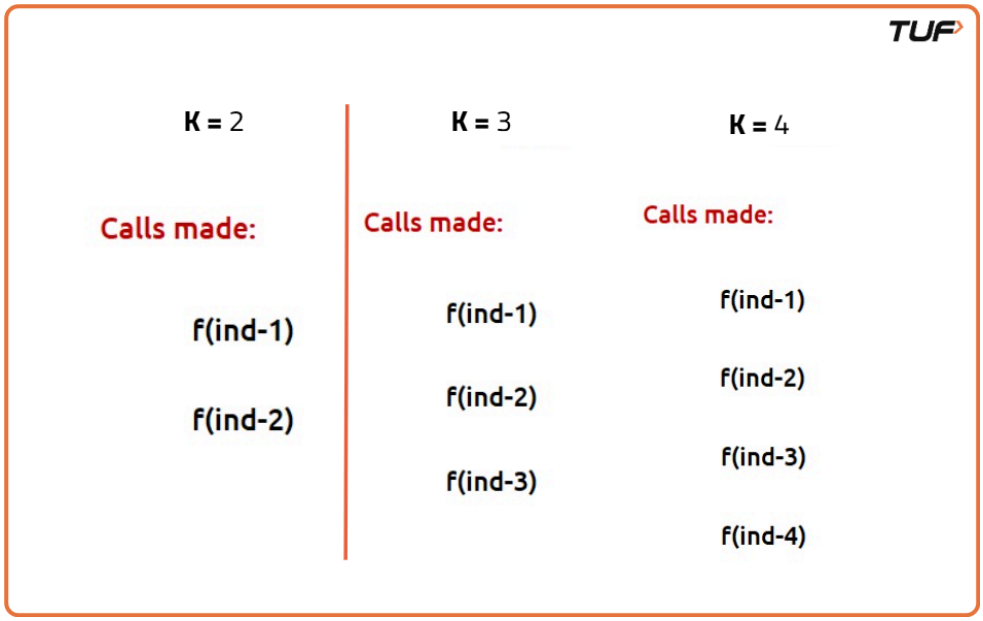
Algorithm

We use recursion with memoization to avoid recalculating the same states again and again.

- Define a function `solveUtil(ind)` that returns the minimum energy required to reach step `ind`.
- Base case:
  If `ind == 0`, the cost is `0` because the frog starts there.
- If the answer for `ind` is already stored in the `dp` array, return it.
- Otherwise, try all jumps from 1 to `k`:

- If `ind - j >= 0`, calculate the cost of jumping from `ind-j` to `ind`.
- Take the minimum of all possible jump costs.

- Store the result in `dp[ind]` and return it.
- The final answer is `solveUtil(n-1)`.



This ensures we explore all valid paths but compute each state only once.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to find the minimum cost to reach index 'ind' using at
most 'k' jumps
int solveUtil(int ind, vector<int>& height, vector<int>& dp, int k) {
    if (ind == 0) return 0;
    if (dp[ind] != -1) return dp[ind];
    int mmSteps = INT_MAX;
    for (int j = 1; j <= k; j++) {
        if (ind - j >= 0) {
            int jump = solveUtil(ind - j, height, dp, k)
                       + abs(height[ind] - height[ind - j]);
```

```
            mmSteps = min(mmSteps, jump);
        }
    }
    return dp[ind] = mmSteps;
}

int solve(int n, vector<int>& height, int k) {
    vector<int> dp(n, -1);
    return solveUtil(n - 1, height, dp, k);
}

int main() {
    vector<int> height{30, 10, 60, 10, 60, 50};
    int n = height.size();
    int k = 2;
    cout << solve(n, height, k) << endl;
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n * k)`
  For each index, we try up to k jumps, and each state is computed only once due to memoization.

- Space Complexity: `O(n)`
  The `dp` array stores results for all n indices. The recursion stack also goes up to n in the worst case.

Approach 2: Tabulation Approach

Algorithm

This approach solves the problem using bottom-up dynamic programming.

- Create a `dp` array where `dp[i]` represents the minimum energy required to reach step `i`.

- Initialize `dp[0] = 0` because no energy is needed at the start.
- For each step `i` from 1 to `n-1`:
    - Try all jumps from 1 to `k`.
    - If `i - j >= 0`, compute:
      `dp[i - j] + abs(height[i] - height[i - j])`
    - Store the minimum of all such values in `dp[i]`.
- The answer is stored in `dp[n-1]`.

This builds the solution step by step without recursion.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to compute the minimum cost to reach the end using at most
'k' jumps
int solveUtil(int n, vector<int>& height, vector<int>& dp, int k) {
    dp[0] = 0;
    for (int i = 1; i < n; i++) {
        int mmSteps = INT_MAX;
        for (int j = 1; j <= k; j++) {
            if (i - j >= 0) {
                int jump = dp[i - j] + abs(height[i] - height[i - j]);
                mmSteps = min(mmSteps, jump);
            }
        }
        dp[i] = mmSteps;
    }
    return dp[n - 1];
}

int solve(int n, vector<int>& height, int k) {
    vector<int> dp(n, -1);
    return solveUtil(n, height, dp, k);
```

```
}

int main() {
    vector<int> height{30, 10, 60, 10, 60, 50};
    int n = height.size();
    int k = 2;
    cout << solve(n, height, k) << endl;
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n * k)`
  For each of the n steps, we check up to k possible jumps.

- Space Complexity: `O(n)`
  The dp array stores the minimum cost for each step.

Maximum sum of non-adjacent elements (DP 5)

Given an array of N positive integers, we need to find the maximum sum of a subsequence such that no two chosen elements are adjacent in the original array.
A subsequence means we can delete some elements, but the remaining elements must stay in the same order.

Example explanation:
For `nums = [2, 1, 4, 9]`
If we choose 2 and 9, they are not adjacent and their sum is 11, which is the maximum possible.

Approach 1: Memoization

Algorithm

We use recursion with the pick / not-pick idea.

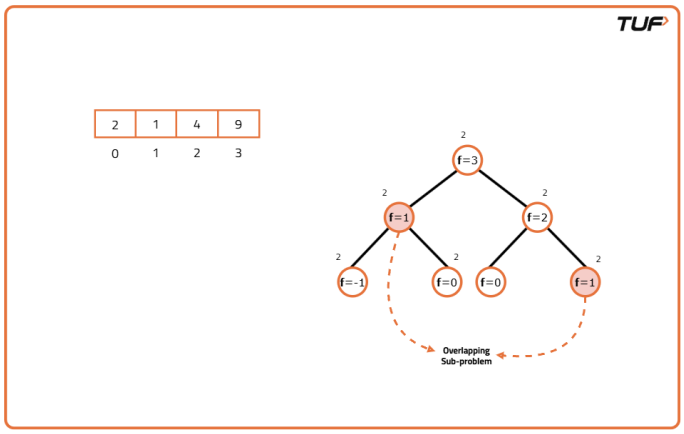
At every index `i`, we have two choices:

- Pick the current element `arr[i]`. If we pick it, we cannot pick the adjacent element, so we move to index `i-2`.
- Do not pick the current element. Then we move to index `i-1`.

We define a recursive function that returns the maximum sum we can get up to index `i`.

- If `i < 0`, return `0` because no elements are left.
- If `i == 0`, return `arr[0]` because only one element is available.
- If the answer for index `i` is already computed, return it from the dp array.
- Otherwise, compute both choices (pick and not pick), take the maximum, store it in `dp[i]`, and return it.

The final answer is the value computed for the last index.



Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int solve(vector<int>& arr, int i, vector<int>& dp) {
        if (i < 0) return 0;
        if (i == 0) return arr[0];
        if (dp[i] != -1) return dp[i];
        int pick = arr[i] + solve(arr, i - 2, dp);
        int notPick = solve(arr, i - 1, dp);
        return dp[i] = max(pick, notPick);
    }

    int maximumNonAdjacentSum(vector<int>& arr) {
        int n = arr.size();
        vector<int> dp(n, -1);
        return solve(arr, n - 1, dp);
    }
};

int main() {
    vector<int> arr = {2, 1, 4, 9};
    Solution obj;
    cout << obj.maximumNonAdjacentSum(arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N)`
  Each index is solved once and stored in the dp array.

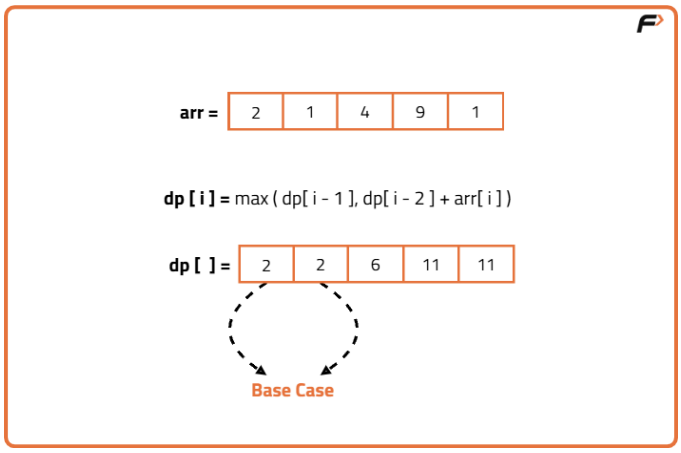
- Space Complexity: `O(N + N)`
  One `O(N)` for the dp array and one `O(N)` for the recursion stack.

Approach 2: Tabulation

Algorithm

This is a bottom-up dynamic programming approach.

- Create a dp array where `dp[i]` stores the maximum sum possible up to index `i`.
- `dp[0] = arr[0]` because only one element is available.
- `dp[1] = max(arr[0], arr[1])` because we can pick only one of them.
- For every index `i` from 2 to `n-1`:
    - If we pick `arr[i]`, the sum becomes `arr[i] + dp[i-2]`.
    - If we do not pick it, the sum is `dp[i-1]`.
    - Store the maximum of these two in `dp[i]`.
- The answer is `dp[n-1]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maximumNonAdjacentSum(vector<int>& arr) {
        int n = arr.size();
        if (n == 1) return arr[0];
        vector<int> dp(n);
```

```
        dp[0] = arr[0];
        dp[1] = max(arr[0], arr[1]);
        for (int i = 2; i < n; i++) {
            dp[i] = max(arr[i] + dp[i - 2], dp[i - 1]);
        }
        return dp[n - 1];
    }
};

int main() {
    vector<int> arr = {2, 1, 4, 9};
    Solution obj;
    cout << obj.maximumNonAdjacentSum(arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N)`
  Each element is processed once.

- Space Complexity: `O(N)`
  Extra space is used for the dp array.

Approach 3: Space Optimization

Algorithm

In the tabulation approach, we only need the results of the previous two indices to compute the current answer. So instead of using a full dp array, we use two variables.

- `prev` stores the answer up to the previous index.

- `prev2` stores the answer up to two indices back.

- For each element:

    - Include current element: `nums[i] + prev2`
    - Exclude current element: `prev`
    - Take the maximum of both and store it in a temporary variable.
    - Update `prev2` and `prev`.
- The final answer is stored in `prev`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxSum(vector<int>& nums) {
        if (nums.empty()) return 0;
        int prev2 = 0;
        int prev = nums[0];
        for (int i = 1; i < nums.size(); i++) {
            int include = nums[i] + prev2;
            int exclude = prev;
            int curr = max(include, exclude);
            prev2 = prev;
            prev = curr;
        }
        return prev;
    }
};

int main() {
    vector<int> arr = {3, 2, 5, 10, 7};
    Solution obj;
    cout << obj.maxSum(arr) << endl;
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N)`
  Each element is processed once.

- Space Complexity: `O(1)`
  Only constant extra space is used.

Dynamic Programming: House Robber (DP 6)

A thief wants to rob houses arranged in a circular street. Each house has some money stored in it, given in an array `Arr`.
The security system is active such that if two adjacent houses are robbed, the police will be notified.

Because the houses are in a **circle**, the first and the last house are also adjacent.
The task is to find the **maximum amount of money** the thief can rob without robbing any two adjacent houses.

Example explanation:
For `money = [2, 1, 4, 9]`
The thief cannot rob both house `0` and house 3 together because they are adjacent in a circular arrangement.
The maximum loot is obtained by robbing houses with money 1 and 9, giving total `10`.

Approach

Algorithm

This problem is a variation of the linear House Robber problem.

Because the houses are circular:

- We **cannot pick both the first and the last house together**.

So we split the problem into **two linear subproblems**:

1. Exclude the first house and consider houses from index `1` to `n-1`
2. Exclude the last house and consider houses from index `0` to `n-2`

For both cases, we solve the **linear House Robber problem** and take the maximum of the two results.

For the linear solution:

- `prev` stores the maximum money robbed till the previous house
- `prev2` stores the maximum money robbed till the house before the previous one
- At each house, we decide:
    - Pick the current house and add `prev2`
    - Or skip the current house and take `prev`
- We update `prev` and `prev2` accordingly

This ensures no two adjacent houses are robbed.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Function to solve the linear house robber problem
long long int solve(vector<int>& arr) {
    int n = arr.size();
    if (n == 1) return arr[0];
    long long int prev = arr[0];
    long long int prev2 = 0;
```

```
    for (int i = 1; i < n; i++) {
        long long int pick = arr[i];
        if (i > 1) pick += prev2;
        long long int nonPick = prev;
        long long int cur_i = max(pick, nonPick);
        prev2 = prev;
        prev = cur_i;
    }
    return prev;
}

// Function to solve the circular house robber problem
long long int robStreet(int n, vector<int> &arr) {
    if (n == 0) return 0;
    if (n == 1) return arr[0];
    vector<int> arr1, arr2;
    for (int i = 0; i < n; i++) {
        if (i != 0) arr1.push_back(arr[i]);
        if (i != n - 1) arr2.push_back(arr[i]);
    }
    long long int ans1 = solve(arr1);
    long long int ans2 = solve(arr2);
    return max(ans1, ans2);
}

int main() {
    vector<int> arr{1, 5, 1, 2, 6};
    int n = arr.size();
    cout << robStreet(n, arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(n)`
  We traverse the houses twice (excluding first and excluding last), each in linear time.

- Space Complexity: `O(1)`
  Only constant extra space is used for variables like `prev` and `prev2`.

Dynamic Programming: Ninja's Training (DP 7)

A ninja is training for n days. Each day, he can perform **one of three activities**:

- Running
- Fighting practice
- Stealth training

The points gained for each activity on each day are given in a 2D matrix `points`, where `points[day][activity]` represents the points earned.

The rule is that **the ninja cannot perform the same activity on two consecutive days**. We need to find the **maximum total points** the ninja can earn over all days.

Example explanation:
For
`[[10, 40, 70], [20, 50, 80], [30, 60, 90]]`

One optimal choice is:

- Day 1: Fighting practice = 70
- Day 2: Stealth training = 50
- Day 3: Fighting practice = 90

Total = 210, which is the maximum possible.

Approach 1: Memoization Approach

Algorithm

We use recursion with memoization.

- Define a function `f(day, last)` that returns the maximum points from day `0` to `day`, where `last` is the activity done on the previous day.
- `last` can be `0, 1, 2` (activities) or 3 (no activity restriction).
- Base case:
    - If `day == 0`, choose the maximum activity on day `0` that is not equal to `last`.
- For other days:
    - Try all activities except `last`.
    - Add current day's points to the result of `f(day-1, current_activity)`.
    - Take the maximum.
- Store results in a `dp[day][last]` table to avoid recomputation.
- Final answer is `f(n-1, 3)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

int f(int day, int last, vector<vector<int>> &points,
vector<vector<int>> &dp) {
    if (dp[day][last] != -1) return dp[day][last];
    if (day == 0) {
        int maxi = 0;
        for (int i = 0; i <= 2; i++) {
            if (i != last)
                maxi = max(maxi, points[0][i]);
        }
        return dp[day][last] = maxi;
    }
    int maxi = 0;
    for (int i = 0; i <= 2; i++) {
        if (i != last) {
```

```
            int activity = points[day][i] + f(day - 1, i, points, dp);
            maxi = max(maxi, activity);
        }
    }
    return dp[day][last] = maxi;
}

int ninjaTraining(int n, vector<vector<int>> &points) {
    vector<vector<int>> dp(n, vector<int>(4, -1));
    return f(n - 1, 3, points, dp);
}

int main() {
    vector<vector<int>> points = {{10, 40, 70},
                                  {20, 50, 80},
                                  {30, 60, 90}};
    int n = points.size();
    cout << ninjaTraining(n, points);
}
```

Complexity Analysis

- Time Complexity: $O(n \times 4 \times 3) = O(n)$
  For each day, we try 4 possible `last` values and 3 activities.

- Space Complexity: $O(n \times 4) + O(n)$
  $O(n \times 4)$ for the DP table and $O(n)$ for recursion stack.

Approach 2: Tabulation Approach

Algorithm

This is a bottom-up DP solution.

- Create a DP table `dp[day][last]`.

- On day `0`, initialize `dp[0][last]` with the best activity excluding `last`.
- For each next day:
    - For every possible `last`, try all activities not equal to `last`.
    - Update `dp[day][last]` with the maximum achievable points.
- The final answer is stored in `dp[n-1][3]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

int ninjaTraining(int n, vector<vector<int>>& points) {
  vector<vector<int>> dp(n, vector<int>(4, 0));

  dp[0][0] = max(points[0][1], points[0][2]);
  dp[0][1] = max(points[0][0], points[0][2]);
  dp[0][2] = max(points[0][0], points[0][1]);
  dp[0][3] = max(points[0][0], max(points[0][1], points[0][2]));

  for (int day = 1; day < n; day++) {
    for (int last = 0; last < 4; last++) {
      dp[day][last] = 0;
      for (int task = 0; task <= 2; task++) {
        if (task != last) {
          int activity = points[day][task] + dp[day - 1][task];
          dp[day][last] = max(dp[day][last], activity);
        }
      }
    }
  }
  return dp[n - 1][3];
}

int main() {
  vector<vector<int>> points = {{10, 40, 70},
                                {20, 50, 80},
```

```
                                {30, 60, 90}};
  int n = points.size();
  cout << ninjaTraining(n, points);
}
```

Complexity Analysis

- Time Complexity: $O(n \times 4 \times 3) = O(n)$
- Space Complexity: $O(n \times 4)$
  DP table stores results for all days and last activities.

Approach 3: Space Optimization Approach

Algorithm

We observe that to compute the current day, we only need results from the previous day.

- Use two arrays of size 4:
  - `prev` for previous day
  - `temp` for current day
- Initialize `prev` for day `0`.
- For each day:
  - Compute `temp[last]` by checking all `task != last`.
  - Assign `prev = temp`.
- Final answer is `prev[3]`.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
int ninjaTraining(int n, vector<vector<int>>& points) {
  vector<int> prev(4, 0);

  prev[0] = max(points[0][1], points[0][2]);
  prev[1] = max(points[0][0], points[0][2]);
  prev[2] = max(points[0][0], points[0][1]);
  prev[3] = max(points[0][0], max(points[0][1], points[0][2]));

  for (int day = 1; day < n; day++) {
    vector<int> temp(4, 0);
    for (int last = 0; last < 4; last++) {
      for (int task = 0; task <= 2; task++) {
        if (task != last) {
          temp[last] = max(temp[last], points[day][task] +
prev[task]);
        }
      }
    }
    prev = temp;
  }
  return prev[3];
}

int main() {
  vector<vector<int>> points = {{10, 40, 70},
                                {20, 50, 80},
                                {30, 60, 90}};
  int n = points.size();
  cout << ninjaTraining(n, points);
}
```

Complexity Analysis

- Time Complexity: $O(n \times 4 \times 3) = O(n)$
- Space Complexity: $O(1)$
  Only constant-sized arrays are used.

Grid Unique Paths : DP on Grids (DP 8)

You are given a grid with `m` rows and `n` columns. You start from the top-left cell `(0,0)` and want to reach the bottom-right cell `(m-1,n-1)`.

From any cell, you are allowed to move **only in two directions**:

- Right
- Down

You need to return the **total number of unique paths** from the start cell to the destination cell.

Example explanation:
For `m = 3, n = 2`
You need to make exactly 1 right move and 2 down moves.
All different orders of these moves give 3 unique paths.

Approach 1: Memoization Approach

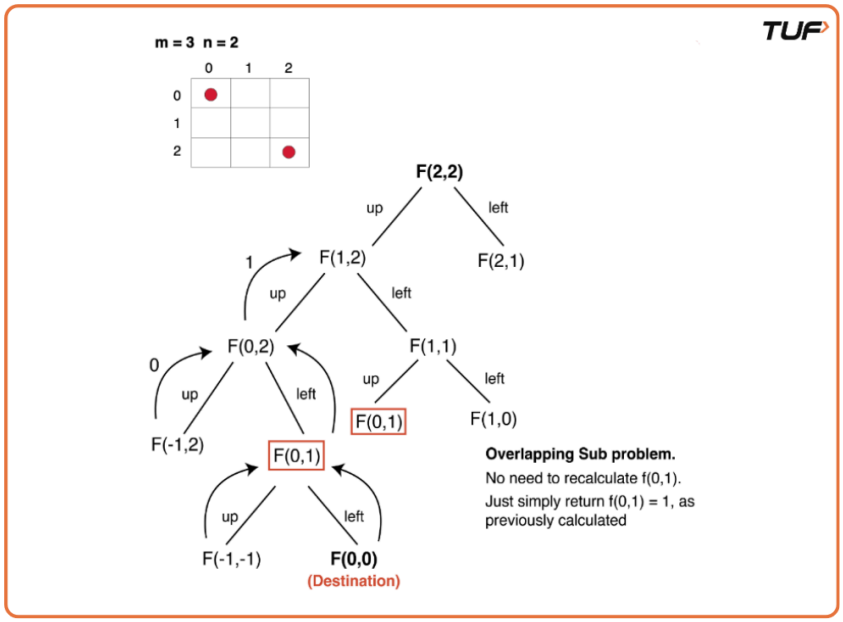
Algorithm

We use recursion with memoization.

- Define a recursive function `func(i, j)` that returns the number of ways to reach cell `(i, j)` from `(0,0)`.
- Base cases:
    - If `i == 0` and `j == 0`, there is exactly 1 way (starting point).
    - If `i < 0` or `j < 0`, return 0 because it is out of bounds.
- If the value for `(i, j)` is already stored in `dp`, return it.
- Otherwise:

    - Ways from top = `func(i-1, j)`
    - Ways from left = `func(i, j-1)`
    - Total ways = top + left
- Store the result in `dp[i][j]`.

The final answer is `func(m-1, n-1)`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int i, int j, vector<vector<int>>& dp){
        if (i == 0 && j == 0) return 1;
        if (i < 0 || j < 0) return 0;
        if (dp[i][j] != -1) return dp[i][j];
        int up = func(i - 1, j, dp);
        int left = func(i, j - 1, dp);
        return dp[i][j] = up + left;
    }
```

```
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, -1));
        return func(m - 1, n - 1, dp);
    }
};

int main() {
    int m = 3, n = 2;
    Solution sol;
    cout << sol.uniquePaths(m, n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(m * n)`
  Each cell `(i, j)` is computed only once due to memoization.

- Space Complexity: `O(m * n)`
  DP table stores results for all grid cells.

Approach 2: Tabulation Approach

Algorithm

This is a bottom-up dynamic programming solution.

- Create a dp array of size `m x n`.
- `dp[i][j]` represents the number of ways to reach cell `(i, j)`.
- Base case:
  - `dp[0][0] = 1`
- For each cell `(i, j)`:

- Ways from top = `dp[i-1][j]` if `i > 0`
- Ways from left = `dp[i][j-1]` if `j > 0`
- `dp[i][j] = up + left`

- The final answer is stored in `dp[m-1][n-1]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int m, int n, vector<vector<int>>& dp){
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (i == 0 && j == 0) {
                    dp[i][j] = 1;
                    continue;
                }
                int up = 0, left = 0;
                if (i > 0) up = dp[i - 1][j];
                if (j > 0) left = dp[i][j - 1];
                dp[i][j] = up + left;
            }
        }
        return dp[m - 1][n - 1];
    }
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m, vector<int>(n, -1));
        return func(m, n, dp);
    }
};

int main() {
    int m = 3, n = 2;
```

```
    Solution sol;
    cout << sol.uniquePaths(m, n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(m * n)`
  Each grid cell is processed once.
- Space Complexity: `O(m * n)`
  A 2D DP table is used.

Approach 3: Space Optimized Approach

Algorithm

From the relation `dp[i][j] = dp[i-1][j] + dp[i][j-1]`, we observe:

- To compute the current row, we only need the previous row and the current row.

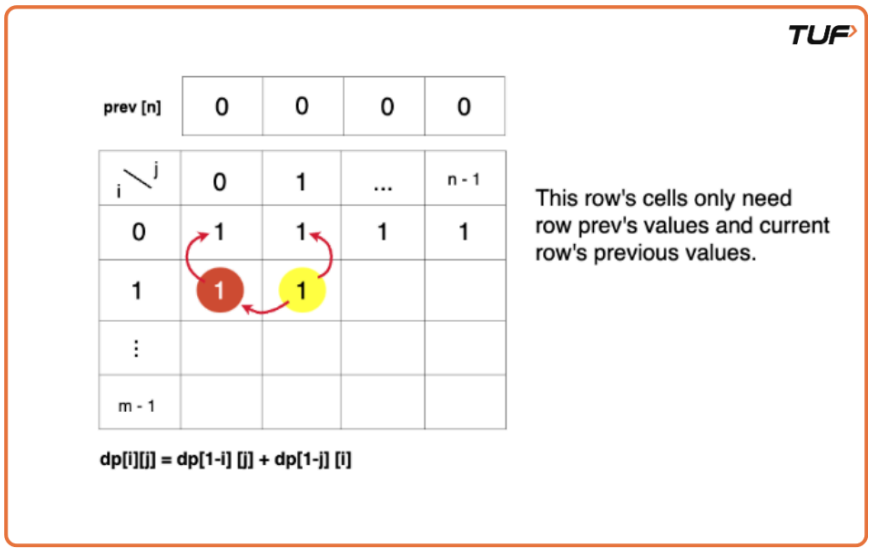
So we use:

- `prev` array → previous row
- `temp` array → current row

Steps:

- Initialize `prev` with zeros.
- For each row:
  - Build `temp` using values from `prev` (up) and `temp` itself (left).
  - Assign `prev = temp`.

- Final answer is `prev[n-1]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int m, int n){
        vector<int> prev(n, 0);
        for (int i = 0; i < m; i++) {
            vector<int> temp(n, 0);
            for (int j = 0; j < n; j++) {
                if (i == 0 && j == 0) {
                    temp[j] = 1;
                    continue;
                }
                int up = 0, left = 0;
                if (i > 0) up = prev[j];
                if (j > 0) left = temp[j - 1];
                temp[j] = up + left;
            }
            prev = temp;
        }
        return prev[n - 1];
```

```
    }
public:
    int uniquePaths(int m, int n) {
        return func(m, n);
    }
};

int main() {
    int m = 3, n = 2;
    Solution sol;
    cout << sol.uniquePaths(m, n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(m * n)`
  Each cell is still processed once.

- Space Complexity: `O(n)`
  Only two 1D arrays of size n are used.

Grid Unique Paths 2 (DP 9)

You are given an `m x n` grid where each cell contains either `0` or `1`.
`0` means the cell is free, and `1` means the cell is blocked.

You start from the top-left cell `(0,0)` and want to reach the bottom-right cell `(m-1,n-1)`.
You can move **only right or down**.
If a cell is blocked, you cannot pass through it.

Return the **total number of unique paths** from start to destination.

Example explanation:
For
`[[0,0,0],[0,1,0],[0,0,0]]`
The middle cell is blocked, so only two valid paths exist that go around it.

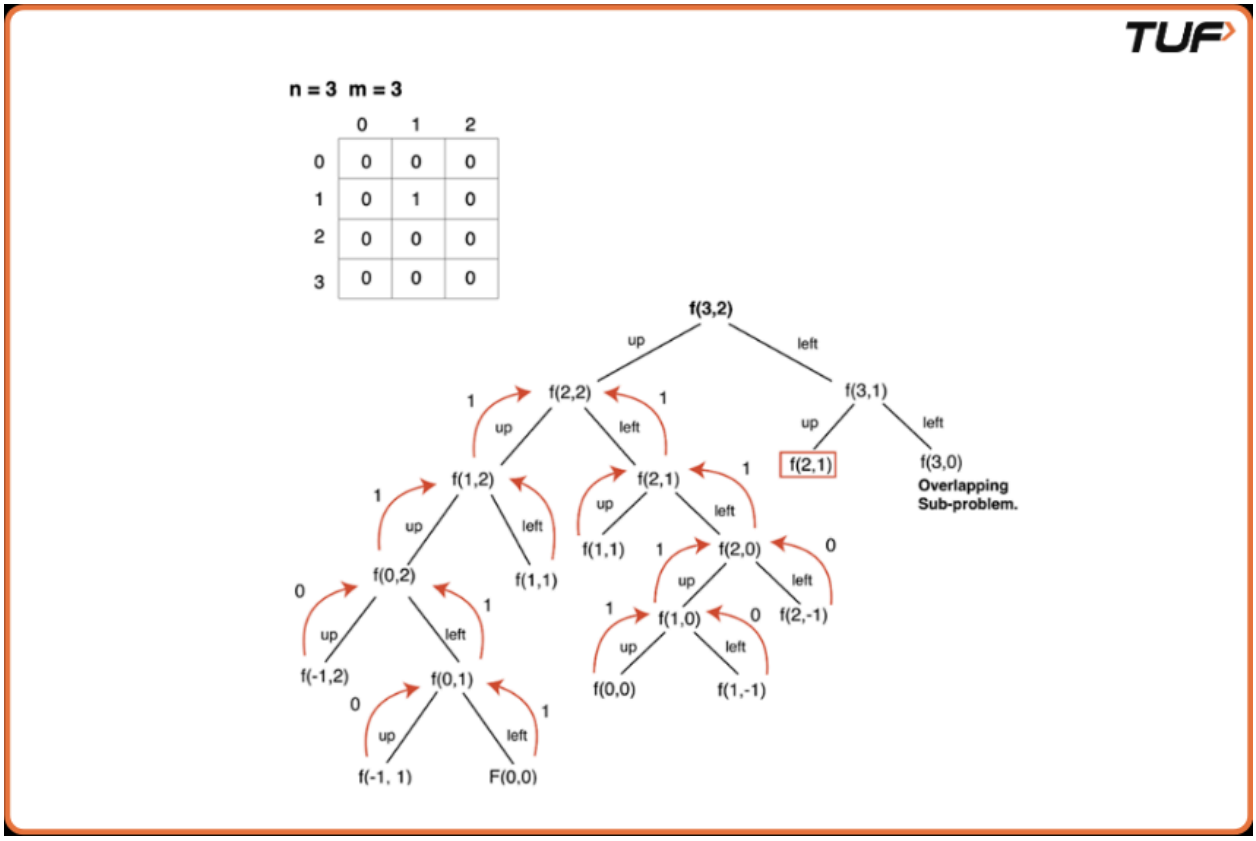
Approach 1: Memoization Approach

Algorithm

We use recursion with memoization.

- Define a function `func(i, j)` that returns the number of ways to reach cell `(i, j)` from `(0,0)`.
- Base cases:
    - If `i < 0` or `j < 0`, return `0` (out of bounds).
    - If `matrix[i][j] == 1`, return `0` (blocked cell).
    - If `i == 0` and `j == 0`, return 1 (starting point).
- If `dp[i][j]` is already computed, return it.
- Otherwise:
    - Ways from top = `func(i-1, j)`
    - Ways from left = `func(i, j-1)`
    - Store and return `dp[i][j] = up + left`.

The final answer is `func(m-1, n-1)`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int i, int j, vector<vector<int>>& matrix,
vector<vector<int>>& dp) {
        if (i < 0 || j < 0 || matrix[i][j] == 1) return 0;
        if (i == 0 && j == 0) return 1;
        if (dp[i][j] != -1) return dp[i][j];
        int up = func(i - 1, j, matrix, dp);
        int left = func(i, j - 1, matrix, dp);
        return dp[i][j] = up + left;
    }
public:
    int uniquePathsWithObstacles(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();
```

```
        vector<vector<int>> dp(m, vector<int>(n, -1));
        return func(m - 1, n - 1, matrix, dp);
    }
};

int main() {
    vector<vector<int>> maze{{0,0,0},{0,1,0},{0,0,0}};
    Solution sol;
    cout << sol.uniquePathsWithObstacles(maze);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(m * n)`
  Each cell is computed once due to memoization.

- Space Complexity: `O(m * n) + O(m + n)`
  DP table plus recursion stack depth.

Approach 2: Tabulation Approach

Algorithm

This is a bottom-up DP approach.

- Create a DP table `dp[m][n]` where `dp[i][j]` stores number of ways to reach `(i,j)`.
- If a cell is blocked (`matrix[i][j] == 1`), set `dp[i][j] = 0`.
- Base case:
    - If `(i,j) == (0,0)` and not blocked, `dp[0][0] = 1`.
- For every cell:
    - Ways from top = `dp[i-1][j]` if `i > 0`

- Ways from left = `dp[i][j-1]` if `j > 0`
- `dp[i][j] = up + left`

- Final answer is `dp[m-1][n-1]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int m, int n, vector<vector<int>>& matrix,
vector<vector<int>>& dp) {
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 1) {
                    dp[i][j] = 0;
                    continue;
                }
                if (i == 0 && j == 0) {
                    dp[i][j] = 1;
                    continue;
                }
                int up = (i > 0) ? dp[i - 1][j] : 0;
                int left = (j > 0) ? dp[i][j - 1] : 0;
                dp[i][j] = up + left;
            }
        }
        return dp[m - 1][n - 1];
    }
public:
    int uniquePathsWithObstacles(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();
        vector<vector<int>> dp(m, vector<int>(n, 0));
        return func(m, n, matrix, dp);
    }
```

```
};

int main() {
    vector<vector<int>> maze{{0,0,0},{0,1,0},{0,0,0}};
    Solution sol;
    cout << sol.uniquePathsWithObstacles(maze);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(m * n)`
  Each grid cell is processed once.

- Space Complexity: `O(m * n)`
  A 2D DP table is used.

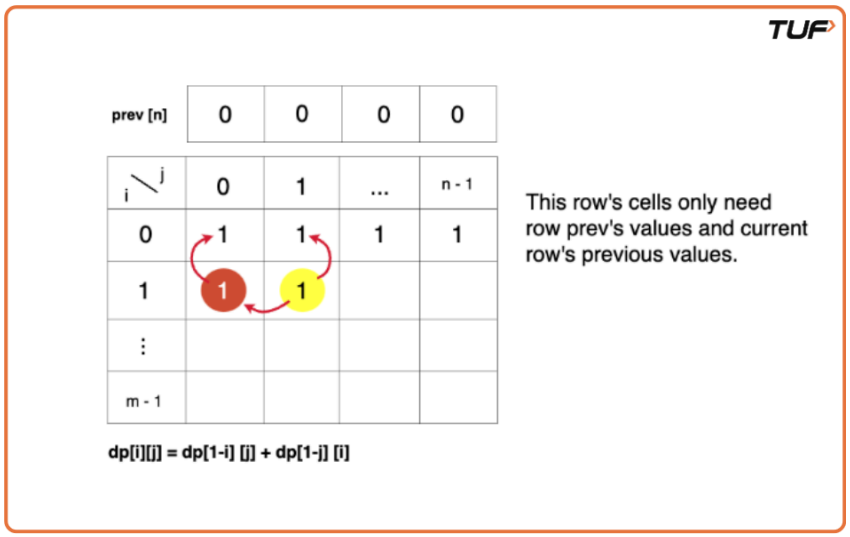
Approach 3: Space Optimized Approach

Algorithm

From the recurrence
`dp[i][j] = dp[i-1][j] + dp[i][j-1]`,
we only need the previous row and current row.

- Use two 1D arrays:
  - `prev` → previous row
  - `curr` → current row
- If a cell is blocked, set `curr[j] = 0`.
- Otherwise:
  - `up = prev[j]`

- `left = curr[j-1]`
- `curr[j] = up + left`

- After each row, assign `prev = curr`.
- Final answer is `prev[n-1]`.
- 

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int m, int n, vector<vector<int>>& matrix) {
        vector<int> prev(n, 0), curr(n, 0);
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 1) {
                    curr[j] = 0;
                    continue;
                }
                if (i == 0 && j == 0) {
                    curr[j] = 1;
                    continue;
                }
```

```
                int up = (i > 0) ? prev[j] : 0;
                int left = (j > 0) ? curr[j - 1] : 0;
                curr[j] = up + left;
            }
            prev = curr;
        }
        return prev[n - 1];
    }
public:
    int uniquePathsWithObstacles(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();
        return func(m, n, matrix);
    }
};

int main() {
    vector<vector<int>> maze{{0,0,0},{0,1,0},{0,0,0}};
    Solution sol;
    cout << sol.uniquePathsWithObstacles(maze);
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(m * n)$
  Each cell is processed once.

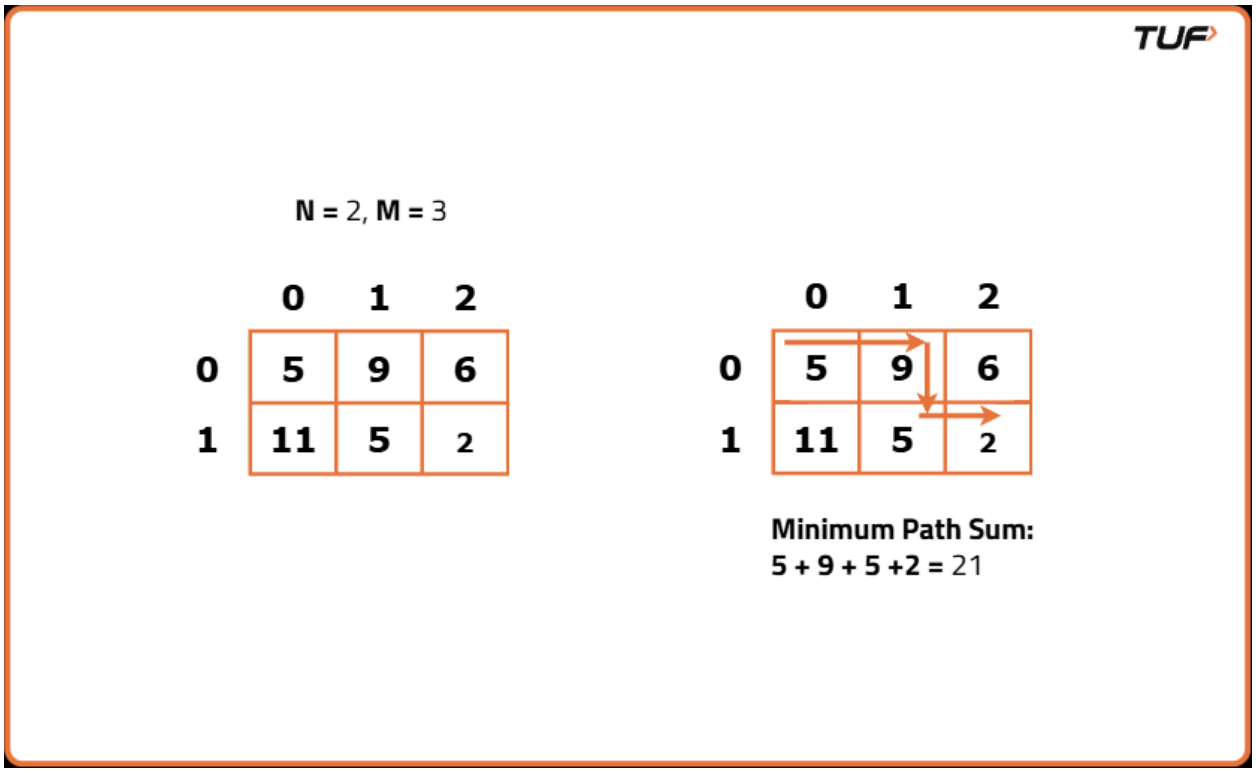
- Space Complexity: $O(n)$
  Only two 1D arrays of size n are used.

Minimum Path Sum In a Grid (DP 10)

You are given an `m x n` grid filled with **non-negative numbers**.
You start from the **top-left cell (0,0)** and want to reach the **bottom-right cell (m-1,n-1)**.

You are allowed to move **only right or down** at any point.
Your task is to find the **minimum possible sum** of numbers along any valid path.

Example explanation:
For `[[5,9,6],[11,5,2]]`
The path 5 → 9 → 5 → 2 gives sum 21, which is minimum.

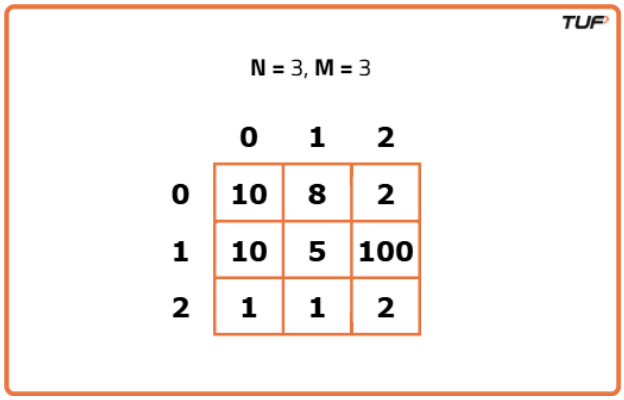


Approach 1: Memoization Approach

Algorithm

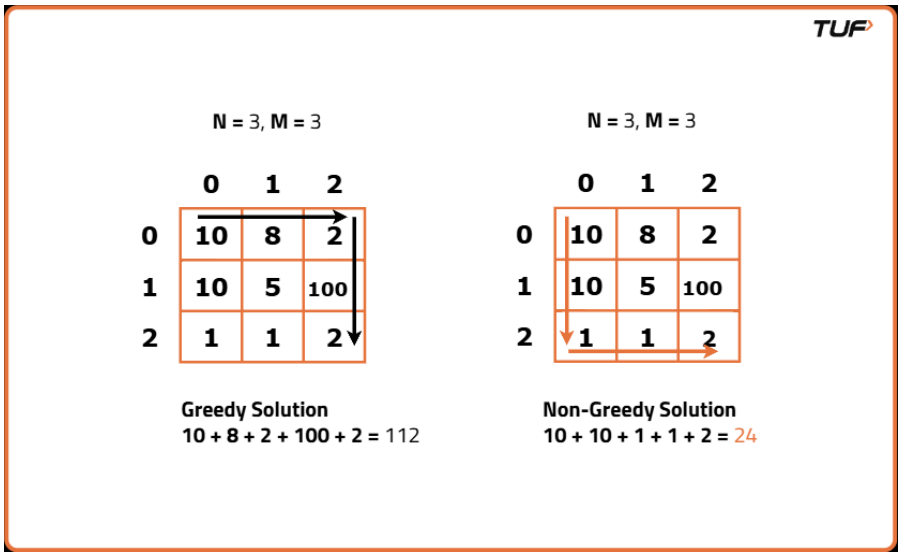
This problem is similar to Grid Unique Paths, but instead of counting paths, we **minimize cost**.

Why greedy does not work:
Choosing the locally smallest value can lead to a path with a larger cost later. Hence, we must explore all valid paths using DP.



Steps:

- Define a recursive function `minPath(i, j)` that returns the minimum path sum to reach cell `(i,j)`.
- Base cases:
  - If `(i,j) == (0,0)`, return `grid[0][0]`.
  - If `i < 0` or `j < 0`, return a very large value (`1e9`) to reject invalid paths.
- If `dp[i][j]` is already computed, return it.
- Otherwise:
  - Cost from up = `grid[i][j] + minPath(i-1, j)`
  - Cost from left = `grid[i][j] + minPath(i, j-1)`
  - Store and return the minimum of the two.
- Final answer is `minPath(n-1, m-1)`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minPath(int i, int j, vector<vector<int>> &grid,
vector<vector<int>> &dp) {
        if (i == 0 && j == 0) return grid[0][0];
        if (i < 0 || j < 0) return 1e9;
        if (dp[i][j] != -1) return dp[i][j];
        int up = grid[i][j] + minPath(i - 1, j, grid, dp);
        int left = grid[i][j] + minPath(i, j - 1, grid, dp);
        return dp[i][j] = min(up, left);
    }

    int minPathSum(vector<vector<int>> &grid) {
        int n = grid.size();
        int m = grid[0].size();
        vector<vector<int>> dp(n, vector<int>(m, -1));
        return minPath(n - 1, m - 1, grid, dp);
    }
};
```

```
int main() {
    vector<vector<int>> grid{{5,9,6},{11,5,2}};
    Solution obj;
    cout << obj.minPathSum(grid);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*M)`
  Each cell is computed once due to memoization.

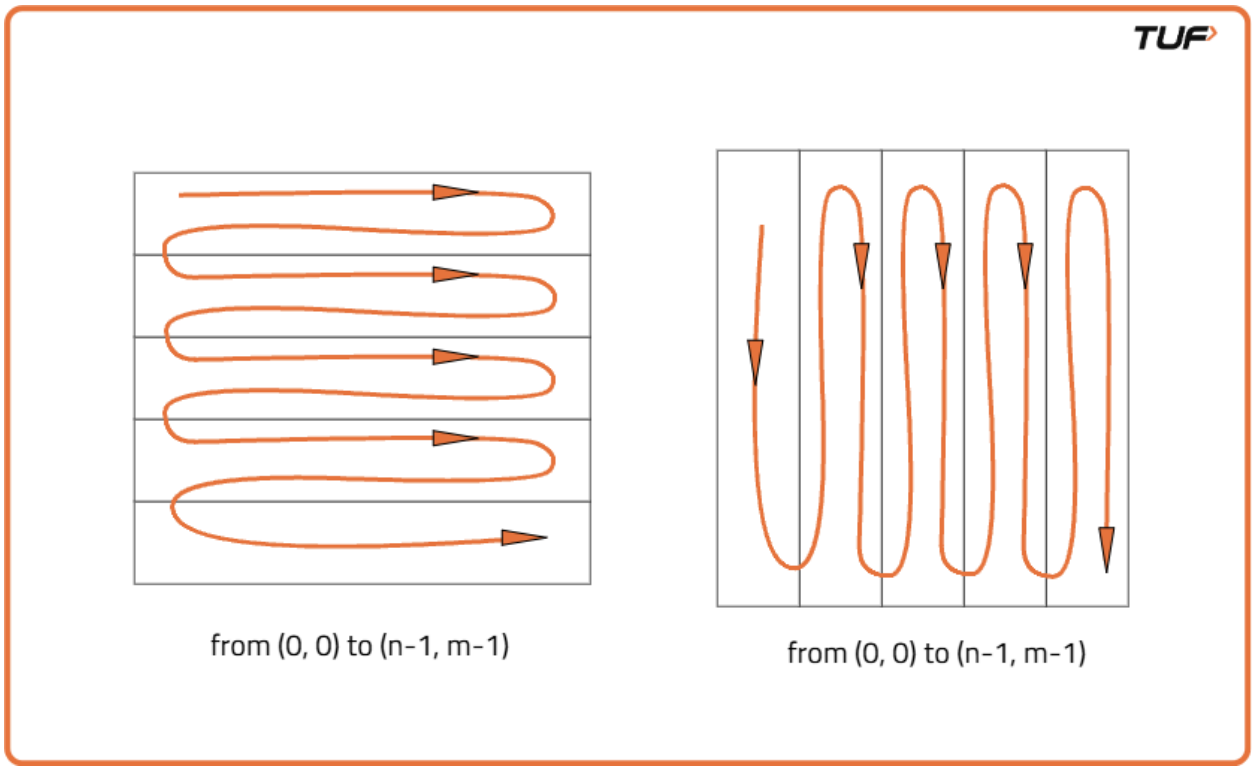
- Space Complexity: `O(N*M) + O(N+M)`
  DP table plus recursion stack.

Approach 2: Tabulation Approach

Algorithm

This is a bottom-up DP solution.

- Create a `dp[n][m]` table.
- Base case: `dp[0][0] = grid[0][0]`.
- Traverse the grid row by row:
    - For each cell `(i, j)`:
        - Cost from up = `dp[i-1][j]` if `i>0` else large value
        - Cost from left = `dp[i][j-1]` if `j>0` else large value
        - `dp[i][j] = grid[i][j] + min(up, left)`
- Final answer is `dp[n-1][m-1]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minPathSum(vector<vector<int>> &matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        vector<vector<int>> dp(n, vector<int>(m, 0));

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (i == 0 && j == 0)
                    dp[i][j] = matrix[i][j];
                else {
                    int up = matrix[i][j] + (i > 0 ? dp[i - 1][j] :
1e9);
                    int left = matrix[i][j] + (j > 0 ? dp[i][j - 1] :
1e9);
```

```
                    dp[i][j] = min(up, left);
                }
            }
        }
        return dp[n - 1][m - 1];
    }
};

int main() {
    vector<vector<int>> matrix{{5,9,6},{11,5,2}};
    Solution obj;
    cout << obj.minPathSum(matrix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*M)`
- Space Complexity: `O(N*M)`

Approach 3: Space Optimization Approach

Algorithm

From the recurrence:
`dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])`

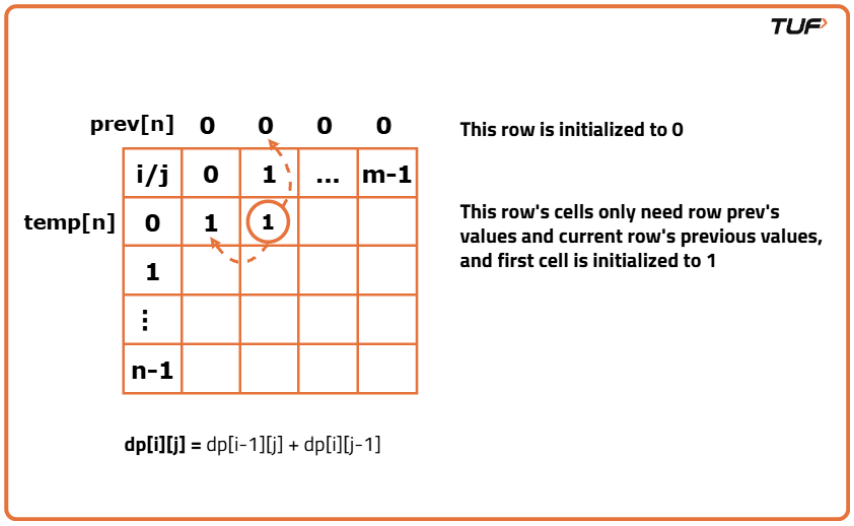
We only need:

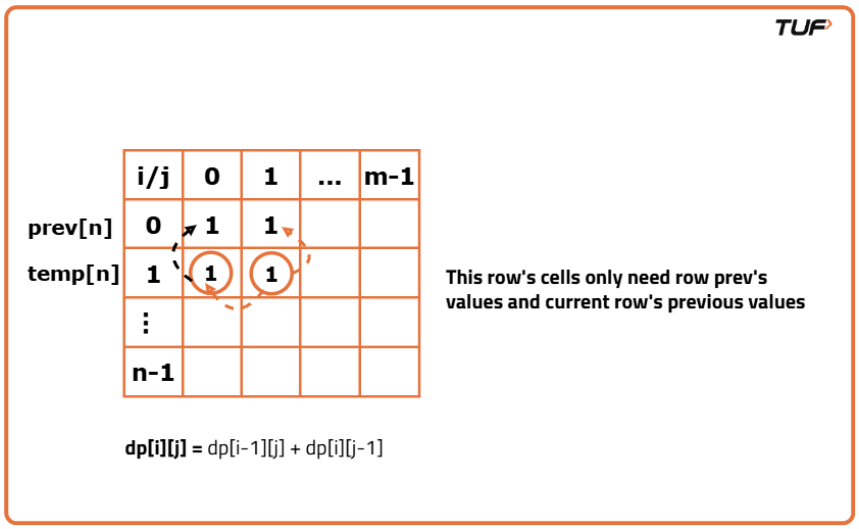
- Previous row
- Current row

Steps:

- Use a 1D array `prev` for previous row.

- For each row, build a `temp` array for current row.
- After finishing a row, assign `prev = temp`.
- Final answer is `prev[m-1]`.





Code

```
#include <bits/stdc++.h>
using namespace std;

class solution {
public:
    int minSumPath(int n, int m, vector<vector<int>> &matrix) {
```

```
        vector<int> prev(m, 0);
        for (int i = 0; i < n; i++) {
            vector<int> temp(m, 0);
            for (int j = 0; j < m; j++) {
                if (i == 0 && j == 0)
                    temp[j] = matrix[i][j];
                else {
                    int up = matrix[i][j] + (i > 0 ? prev[j] : 1e9);
                    int left = matrix[i][j] + (j > 0 ? temp[j - 1] :
1e9);
                    temp[j] = min(up, left);
                }
            }
            prev = temp;
        }
        return prev[m - 1];
    }
};

int main() {
    vector<vector<int>> matrix{{5,9,6},{11,5,2}};
    int n = matrix.size();
    int m = matrix[0].size();
    solution obj;
    cout << obj.minSumPath(n, m, matrix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*M)`

- Space Complexity: `O(M)`
  Only one row is stored at a time.

Minimum Path Sum in Triangular Grid (DP 11)

You are given a triangular grid with n rows.
The first row has 1 element, the second row has 2 elements, and so on.

You start from the **top element (0,0)** and move to the **last row**.
From a cell `(i, j)`, you are allowed to move only to:

- the **bottom cell** `(i+1, j)`
- the **bottom-right cell** `(i+1, j+1)`

You need to return the **minimum possible path sum** from the top to any cell in the last row.

Example explanation:
For `[[1],[1,2],[1,2,4]]`
One minimum path is 1 → 1 → 1, sum = 3.

Approach 1: Memoization Approach

Algorithm

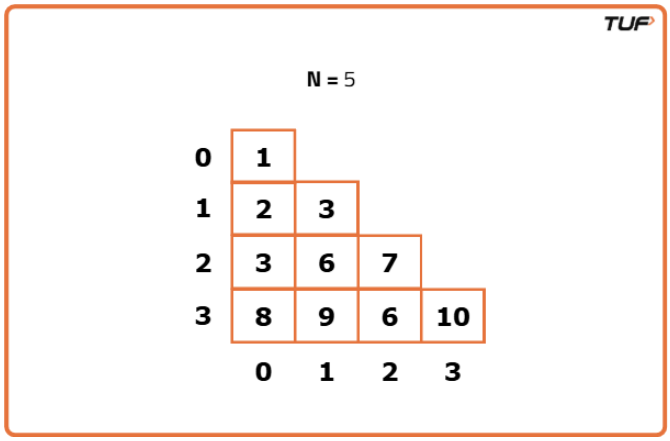
This problem is a variation of the minimum path sum in a grid, but the grid is triangular.

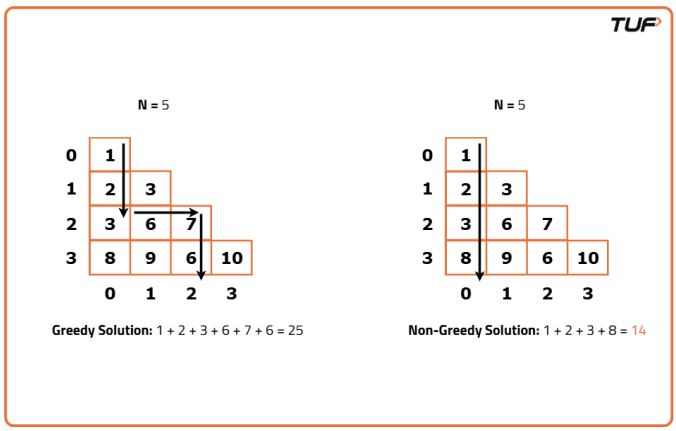
A greedy approach fails because choosing the locally smaller value may lead to a larger cost later.

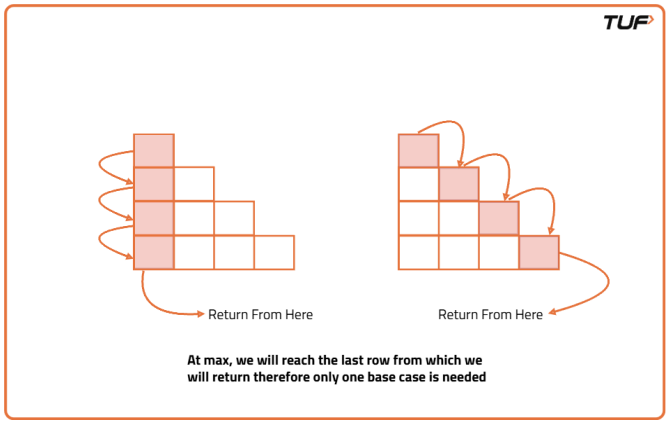
We use recursion with memoization.

- Define a recursive function `solve(i, j)` that returns the minimum path sum starting from cell `(i, j)` to the bottom.
- Base case:
    - If `i == n-1` (last row), return `triangle[i][j]`.

- If the value is already stored in `dp[i][j]`, return it.
- Otherwise:
    - Move **down**: `triangle[i][j] + solve(i+1, j)`
    - Move **diagonal**: `triangle[i][j] + solve(i+1, j+1)`
    - Take the minimum of both.
- Store the result in dp to avoid recomputation.
- Final answer is `solve(0, 0)`.







Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i, int j, vector<vector<int>> &triangle, int n,
vector<vector<int>> &dp) {
        if (dp[i][j] != -1) return dp[i][j];
        if (i == n - 1) return triangle[i][j];
        int down = triangle[i][j] + solve(i + 1, j, triangle, n, dp);
        int diag = triangle[i][j] + solve(i + 1, j + 1, triangle, n,
dp);
        return dp[i][j] = min(down, diag);
    }

    int minimumPathSum(vector<vector<int>> &triangle) {
        int n = triangle.size();
        vector<vector<int>> dp(n, vector<int>(n, -1));
        return solve(0, 0, triangle, n, dp);
    }
};

int main() {
    vector<vector<int>> triangle{{1},{2,3},{3,6,7},{8,9,6,10}};
    Solution obj;
    cout << obj.minimumPathSum(triangle);
    return 0;
```

```
}
```

Complexity Analysis

- Time Complexity: `O(N*N)`
  Every cell in the triangle is computed once.

- Space Complexity: `O(N*N) + O(N)`
  DP table plus recursion stack depth.

Approach 2: Tabulation Approach

Algorithm

This is a bottom-up DP approach.

- Create a `dp` table with the same shape as the triangle.

- Initialize the **last row** of `dp` with the values from the triangle, since there is no choice after that.

- Start from the second-last row and move upwards:

    - For each cell `(i, j)`:

        - `dp[i][j] = triangle[i][j] + min(dp[i+1][j], dp[i+1][j+1])`

- The answer will be stored at `dp[0][0]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minimumPathSum(vector<vector<int>> &triangle, int n) {
        vector<vector<int>> dp(n, vector<int>(n, 0));
```

```
        for (int j = 0; j < n; j++)
            dp[n - 1][j] = triangle[n - 1][j];

        for (int i = n - 2; i >= 0; i--) {
            for (int j = i; j >= 0; j--) {
                int down = triangle[i][j] + dp[i + 1][j];
                int diag = triangle[i][j] + dp[i + 1][j + 1];
                dp[i][j] = min(down, diag);
            }
        }
        return dp[0][0];
    }
};

int main() {
    vector<vector<int>> triangle{{1},{2,3},{3,6,7},{8,9,6,10}};
    int n = triangle.size();
    Solution obj;
    cout << obj.minimumPathSum(triangle, n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*N)`
  Entire triangular grid is processed once.

- Space Complexity: `O(N*N)`
  DP table stores results for all cells.

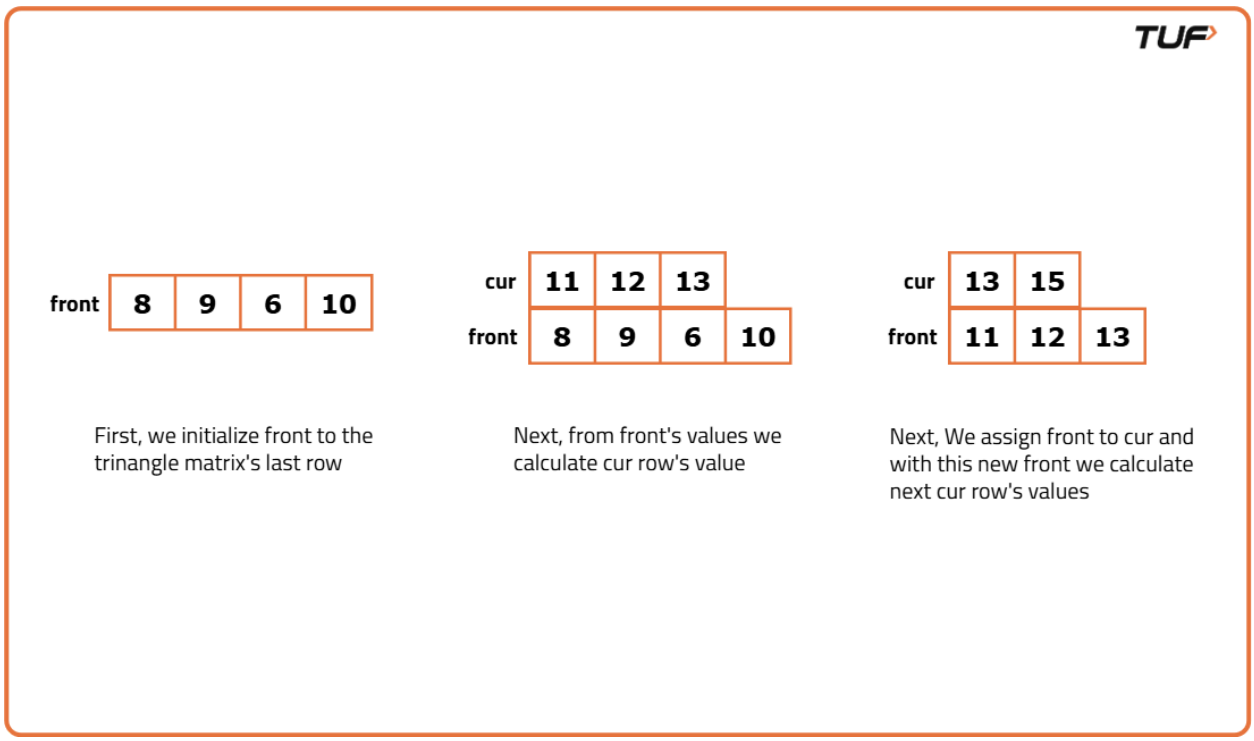
Approach 3: Space Optimization Approach

Algorithm

From the recurrence relation:
`dp[i][j] = triangle[i][j] + min(dp[i+1][j], dp[i+1][j+1])`

We only need the **next row** to compute the current row.

- Use a 1D array `front` initialized with the last row of the triangle.
- Use another array `cur` for the current row.
- Traverse from bottom to top:
    - For each cell, compute minimum using `front[j]` and `front[j+1]`.
- After each row, assign `front = cur`.
- Final answer is `front[0]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minimumPathSum(vector<vector<int>> &triangle, int n) {
        vector<int> front(n, 0), cur(n, 0);
        for (int j = 0; j < n; j++)
            front[j] = triangle[n - 1][j];
```

```
        for (int i = n - 2; i >= 0; i--) {
            for (int j = i; j >= 0; j--) {
                int down = triangle[i][j] + front[j];
                int diag = triangle[i][j] + front[j + 1];
                cur[j] = min(down, diag);
            }
            front = cur;
        }
        return front[0];
    }
};

int main() {
    vector<vector<int>> triangle{{1},{2,3},{3,6,7},{8,9,6,10}};
    int n = triangle.size();
    Solution obj;
    cout << obj.minimumPathSum(triangle, n);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*N)`
  Every element in the triangle is processed once.

- Space Complexity: `O(N)`
  Only one row is stored at any time.

Minimum / Maximum Falling Path Sum (DP 12)

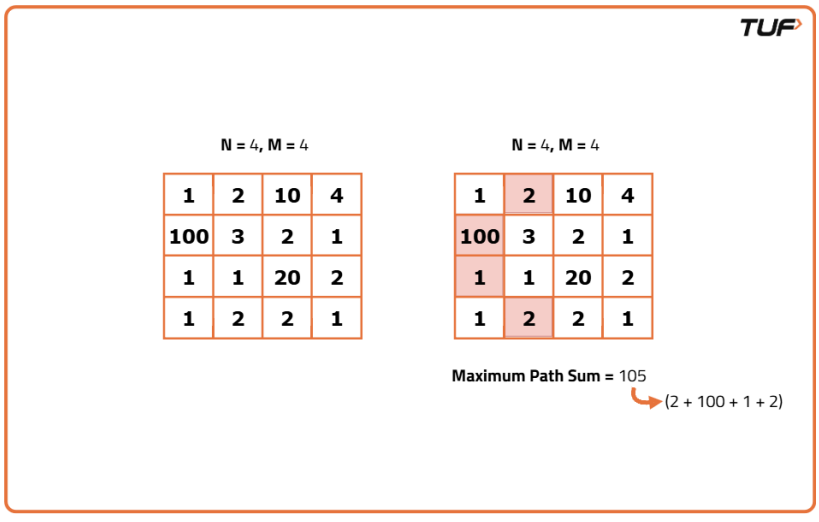
You are given a 2D grid `matrix` with integer values.
You can start from **any cell in the first row** and must reach **any cell in the last row**.

From a cell `(i, j)`, you are allowed to move to:

- bottom `(i+1, j)`
- bottom-left `(i+1, j-1)`
- bottom-right `(i+1, j+1)`

You need to return the **minimum falling path sum** (same logic applies for maximum by replacing `min` with `max`).

Example explanation:
For
`[[1,4,3,1],[2,3,-1,-1],[1,1,-1,8]]`
One valid minimum path is 4 → -1 → 8, sum = 11.



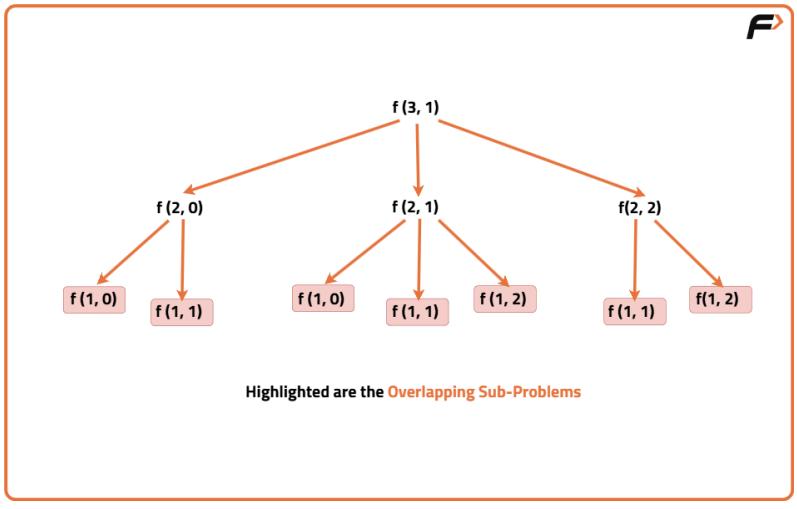
Approach 1: Memoization

Algorithm

A greedy approach does not work because choosing a locally smaller value may lead to a worse path later.

We use recursion with memoization.

- Define a recursive function `dfs(row, col)` that returns the minimum falling path sum starting from `(row, col)`.
- Base cases:
    - If `col < 0` or `col >= m`, return a very large value (`1e9`) to reject invalid paths.
    - If `row == n-1` (last row), return `matrix[row][col]`.
- If `dp[row][col]` is already computed, return it.
- Otherwise:
    - Try all three possible moves: down, down-left, down-right.
    - Add the current cell value to the minimum of these.
- Since the path can start from any column in the first row, compute the answer for all columns in row `0` and take the minimum.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int dfs(int row, int col, vector<vector<int>>& matrix,
vector<vector<int>>& dp) {
        int n = matrix.size();
        int m = matrix[0].size();
        if (col < 0 || col >= m) return 1e9;
        if (row == n - 1) return matrix[row][col];
        if (dp[row][col] != -1) return dp[row][col];
        int down = dfs(row + 1, col, matrix, dp);
        int downLeft = dfs(row + 1, col - 1, matrix, dp);
        int downRight = dfs(row + 1, col + 1, matrix, dp);
        return dp[row][col] =
            matrix[row][col] + min({down, downLeft, downRight});
    }

    int minFallingPathSum(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        vector<vector<int>> dp(n, vector<int>(m, -1));
        int ans = 1e9;
        for (int col = 0; col < m; col++) {
            ans = min(ans, dfs(0, col, matrix, dp));
        }
        return ans;
    }
};

int main() {
    vector<vector<int>> matrix{
        {1,4,3,1},
        {2,3,-1,-1},
        {1,1,-1,8}
    };
```

```
    Solution obj;
    cout << obj.minFallingPathSum(matrix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*M)`
  Each cell `(row, col)` is computed once.

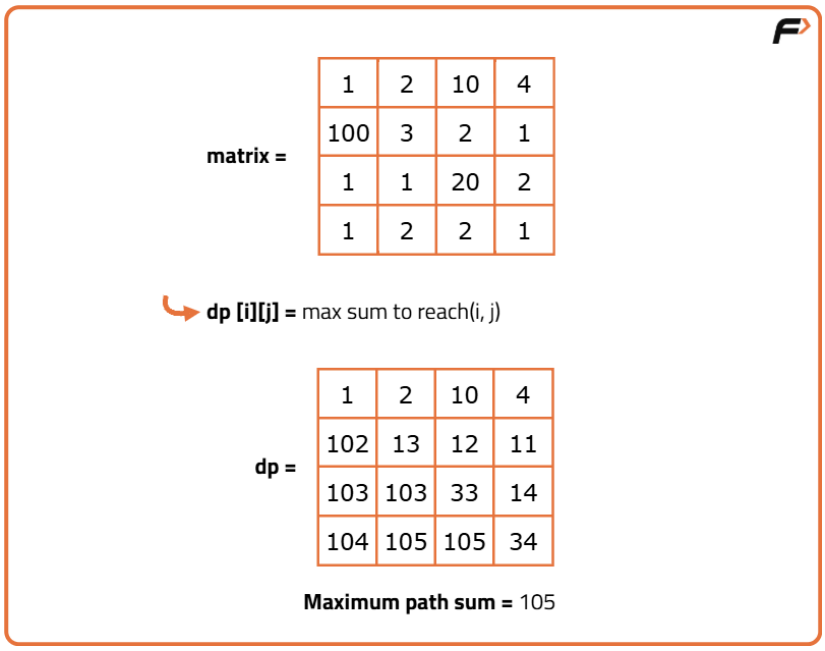
- Space Complexity: `O(N*M) + O(N)`
  DP table plus recursion stack depth.

Approach 2: Tabulation

Algorithm

This is a bottom-up DP approach.

- Create a `dp` array of size `n x m`.
- Initialize the **last row** of `dp` with the values from the matrix.
- Move from the second-last row upwards:
    - For each cell `(row, col)`:
        - Take minimum of:
            - `dp[row+1][col]`
            - `dp[row+1][col-1]` if valid
            - `dp[row+1][col+1]` if valid
        - Add current cell value.
- After filling, the answer is the minimum value in `dp[0]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minFallingPathSum(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        vector<vector<int>> dp(n, vector<int>(m, 0));

        for (int col = 0; col < m; col++)
            dp[n - 1][col] = matrix[n - 1][col];

        for (int row = n - 2; row >= 0; row--) {
            for (int col = 0; col < m; col++) {
                int down = dp[row + 1][col];
                int downLeft = (col > 0) ? dp[row + 1][col - 1] : 1e9;
                int downRight = (col < m - 1) ? dp[row + 1][col + 1] :
1e9;
                dp[row][col] =
                    matrix[row][col] + min({down, downLeft,
downRight});
```

```
            }
        }

        return *min_element(dp[0].begin(), dp[0].end());
    }
};

int main() {
    vector<vector<int>> matrix{
        {1,4,3,1},
        {2,3,-1,-1},
        {1,1,-1,8}
    };
    Solution obj;
    cout << obj.minFallingPathSum(matrix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*M)`
  Entire grid is processed once.

- Space Complexity: `O(N*M)`
  DP table stores results for all cells.

Approach 3: Space Optimization

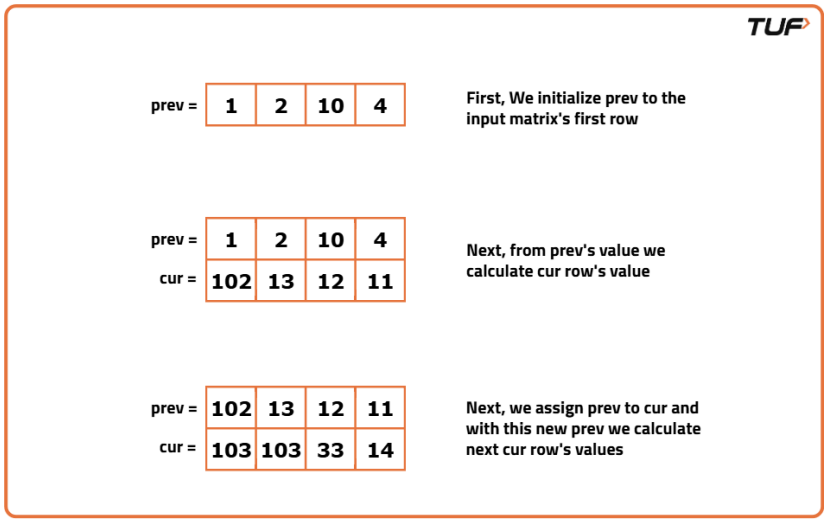
Algorithm

From the relation:
`dp[row][col] = matrix[row][col] + min(dp[row+1][col], dp[row+1][col-1], dp[row+1][col+1])`

We only need the **next row** to compute the current row.

- Use a 1D array `dp` initialized with the last row of the matrix.

- For each row from bottom to top:
    - Build a `curr` array using values from dp.
    - Replace dp with `curr`.
- Final answer is the minimum value in dp.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minFallingPathSum(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        vector<int> dp = matrix[n - 1];

        for (int row = n - 2; row >= 0; row--) {
            vector<int> curr(m, 0);
            for (int col = 0; col < m; col++) {
                int down = dp[col];
                int downLeft = (col > 0) ? dp[col - 1] : 1e9;
                int downRight = (col < m - 1) ? dp[col + 1] : 1e9;
                curr[col] =
```

```
                    matrix[row][col] + min({down, downLeft,
downRight});
            }
            dp = curr;
        }

        return *min_element(dp.begin(), dp.end());
    }
};

int main() {
    vector<vector<int>> matrix{
        {1,4,3,1},
        {2,3,-1,-1},
        {1,1,-1,8}
    };
    Solution obj;
    cout << obj.minFallingPathSum(matrix);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*M)`
  Each cell is processed once.

- Space Complexity: `O(M)`
  Only one row is stored at a time.

3-D DP : Ninja and his Friends (DP-13)

We are given a grid of size `N x M` where each cell contains some chocolates.
Alice starts at cell `(0,0)` and Bob starts at cell `(0, M-1)`.

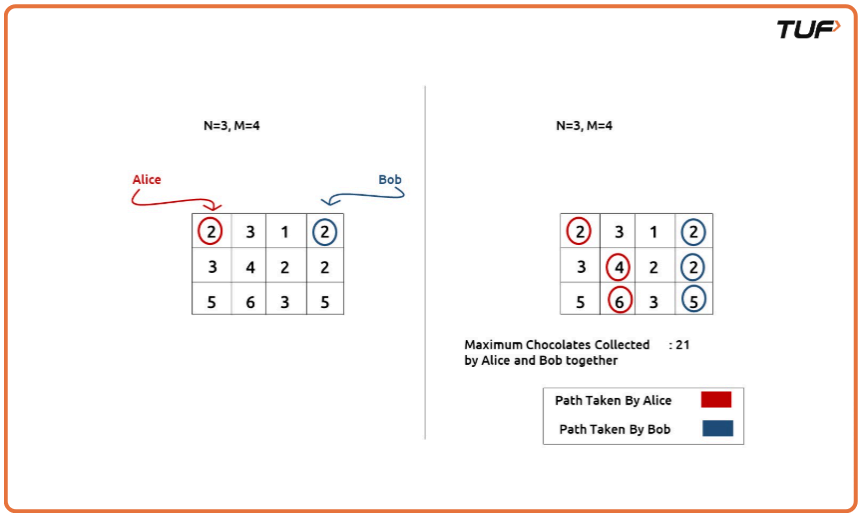
Both Alice and Bob move **row by row downward**. From a cell `(i, j)`, they can move to:

- bottom `(i+1, j)`
- bottom-left `(i+1, j-1)`
- bottom-right `(i+1, j+1)`

When they visit a cell, they collect all chocolates from that cell.
If both of them land on the **same cell**, chocolates are counted **only once**.

They cannot move outside the grid.
We need to return the **maximum total chocolates** Alice and Bob can collect together.

Example explanation:
For
`[[2,3,1,2],[3,4,2,2],[5,6,3,5]]`
One optimal path allows Alice and Bob to collect a total of 21 chocolates



.

Approach 1: Memoization Approach

Algorithm

This is a 3-dimensional DP problem because the state depends on:

- current row `i`
- Alice's column `j1`
- Bob's column `j2`

Greedy does not work because taking the maximum chocolates at the current step may block better choices later.

We use recursion with memoization.

- Define `solve(i, j1, j2)` = maximum chocolates collectible starting from row `i`, where Alice is at column `j1` and Bob at column `j2`.
- If `j1` or `j2` goes out of bounds, return a very small value (`-1e9`) to mark an invalid path.
- Base case:
    - If `i == n-1` (last row):
        - If `j1 == j2`, return `grid[i][j1]`
        - Else return `grid[i][j1] + grid[i][j2]`
- If the value is already stored in dp, return it.
- Otherwise:
    - From `(i, j1, j2)`, try all `3 x 3 = 9` combinations of moves for Alice and Bob.
    - For each combination, move to the next row and take the maximum result.
- Store and return the result.

The final answer is `solve(0, 0, M-1)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i, int j1, int j2, int n, int m,
              vector<vector<int>>& grid,
              vector<vector<vector<int>>>& dp) {
        if (j1 < 0 || j1 >= m || j2 < 0 || j2 >= m)
            return -1e9;
        if (i == n - 1) {
            if (j1 == j2) return grid[i][j1];
            return grid[i][j1] + grid[i][j2];
        }
        if (dp[i][j1][j2] != -1) return dp[i][j1][j2];
        int maxi = -1e9;
        int curr = (j1 == j2) ? grid[i][j1]
                              : grid[i][j1] + grid[i][j2];
        for (int dj1 = -1; dj1 <= 1; dj1++) {
            for (int dj2 = -1; dj2 <= 1; dj2++) {
                maxi = max(maxi,
                           curr + solve(i + 1, j1 + dj1, j2 + dj2,
                                        n, m, grid, dp));
            }
        }
        return dp[i][j1][j2] = maxi;
    }

    int maximumChocolates(int n, int m, vector<vector<int>>& grid) {
        vector<vector<vector<int>>> dp(
            n, vector<vector<int>>(m, vector<int>(m, -1)));
        return solve(0, 0, m - 1, n, m, grid, dp);
    }
};

int main() {
```

```
    vector<vector<int>> grid = {
        {2, 3, 1, 2},
        {3, 4, 2, 2},
        {5, 6, 3, 5}
    };
    Solution obj;
    cout << obj.maximumChocolates(grid.size(), grid[0].size(), grid);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N * M * M * 9)`
  There are `N * M * M` states, and for each we try 9 move combinations.

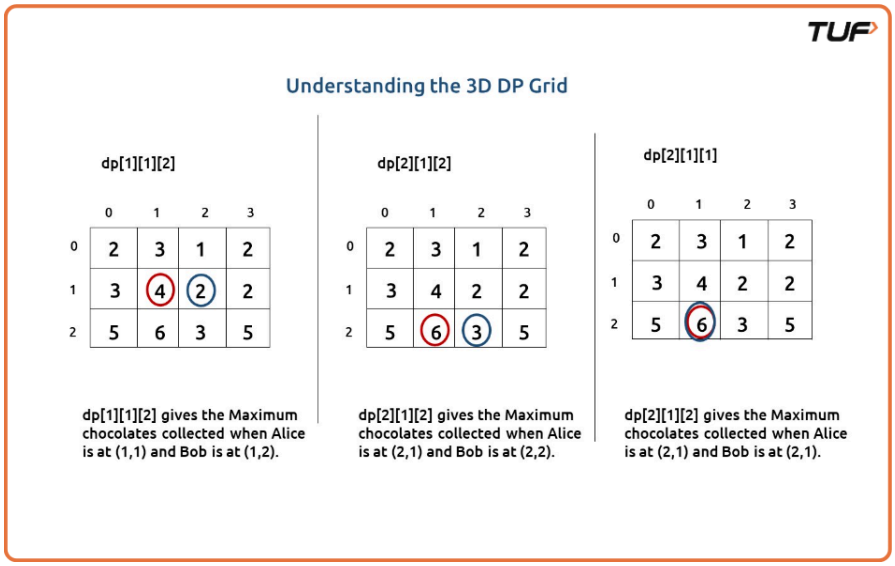
- Space Complexity: `O(N * M * M) + O(N)`
  DP table plus recursion stack.

Approach 2: Tabulation Approach

Algorithm

This is the bottom-up version of the memoized solution.

- Create a 3D DP table `dp[i][j1][j2]`.
- Base case:
  - Fill the last row (`i = n-1`) directly using grid values.
- Move from row `n-2` up to `0`:
  - For each `(j1, j2)` pair, try all 9 move combinations.
  - Take the maximum possible chocolates.
- The final answer is stored at `dp[0][0][m-1]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maximumChocolates(int n, int m, vector<vector<int>>& grid) {
        vector<vector<vector<int>>> dp(
            n, vector<vector<int>>(m, vector<int>(m, 0)));

        for (int j1 = 0; j1 < m; j1++) {
            for (int j2 = 0; j2 < m; j2++) {
                if (j1 == j2) dp[n-1][j1][j2] = grid[n-1][j1];
                else dp[n-1][j1][j2] = grid[n-1][j1] + grid[n-1][j2];
            }
        }

        for (int i = n - 2; i >= 0; i--) {
            for (int j1 = 0; j1 < m; j1++) {
                for (int j2 = 0; j2 < m; j2++) {
                    int maxi = -1e9;
                    int curr = (j1 == j2) ? grid[i][j1]
                                          : grid[i][j1] + grid[i][j2];
                    for (int dj1 = -1; dj1 <= 1; dj1++) {
                        for (int dj2 = -1; dj2 <= 1; dj2++) {
```

```
                            int nj1 = j1 + dj1;
                            int nj2 = j2 + dj2;
                            if (nj1 >= 0 && nj1 < m &&
                                nj2 >= 0 && nj2 < m) {
                                maxi = max(maxi,
                                           curr + dp[i+1][nj1][nj2]);
                            }
                        }
                    }
                    dp[i][j1][j2] = maxi;
                }
            }
        }
        return dp[0][0][m-1];
    }
};

int main() {
    vector<vector<int>> grid = {
        {2, 3, 1, 2},
        {3, 4, 2, 2},
        {5, 6, 3, 5}
    };
    Solution obj;
    cout << obj.maximumChocolates(grid.size(), grid[0].size(), grid);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N * M * M * 9)`
- Space Complexity: `O(N * M * M)`

Approach 3: Space Optimization Approach

Algorithm

In tabulation, each row depends only on the **next row**.

- Instead of a full 3D DP table, use:
    - `next[m][m]` → stores values for row `i+1`
    - `curr[m][m]` → stores values for row `i`
- Initialize `next` using the last row of the grid.
- Move upward row by row, filling `curr` from `next`.
- After each row, assign `next = curr`.
- Final answer is `next[0][m-1]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maximumChocolates(int n, int m, vector<vector<int>>& grid) {
        vector<vector<int>> next(m, vector<int>(m, 0));
        vector<vector<int>> curr(m, vector<int>(m, 0));

        for (int j1 = 0; j1 < m; j1++) {
            for (int j2 = 0; j2 < m; j2++) {
                if (j1 == j2) next[j1][j2] = grid[n-1][j1];
                else next[j1][j2] = grid[n-1][j1] + grid[n-1][j2];
            }
        }

        for (int i = n - 2; i >= 0; i--) {
            for (int j1 = 0; j1 < m; j1++) {
                for (int j2 = 0; j2 < m; j2++) {
                    int maxi = -1e9;
```

```
                    int currVal = (j1 == j2) ? grid[i][j1]
                                             : grid[i][j1] +
grid[i][j2];
                    for (int dj1 = -1; dj1 <= 1; dj1++) {
                        for (int dj2 = -1; dj2 <= 1; dj2++) {
                            int nj1 = j1 + dj1;
                            int nj2 = j2 + dj2;
                            if (nj1 >= 0 && nj1 < m &&
                                nj2 >= 0 && nj2 < m) {
                                maxi = max(maxi,
                                           currVal + next[nj1][nj2]);
                            }
                        }
                    }
                    curr[j1][j2] = maxi;
                }
            }
            next = curr;
        }
        return next[0][m-1];
    }
};

int main() {
    vector<vector<int>> grid = {
        {2, 3, 1, 2},
        {3, 4, 2, 2},
        {5, 6, 3, 5}
    };
    Solution obj;
    cout << obj.maximumChocolates(grid.size(), grid[0].size(), grid);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N * M * M * 9)`

- Space Complexity: `O(M * M)`

Subset Sum Equal to Target (DP-14)

We are given an array `ARR` of `N` positive integers and an integer `K`.
We need to check whether there exists **any subset (subsequence)** of the array whose sum is exactly equal to `K`.

A subset can be contiguous or non-contiguous, but the order of elements must remain the same as in the original array.

Example explanation:
For `ARR = [4,3,5,2]` and `K = 6`
Subset `[4,2]` gives sum `6`, so the answer is `true`.

Approach 1: Memoization Approach

Algorithm

Greedy does not work here because we are not optimizing step by step; we just need to check existence.

We use recursion with memoization using the **pick / not pick** technique.

Define a function `f(ind, target)` that returns `true` if a subset with sum `target` can be formed using elements from index `0` to `ind`.

Base cases:

- If `target == 0`, we already formed the required sum, return `true`.
- If `ind == 0`, return `true` if `arr[0] == target`, else `false`.

At each index, we have two choices:

- **Not pick** the current element → `f(ind-1, target)`
- **Pick** the current element (only if `arr[ind] <= target`) → `f(ind-1, target-arr[ind])`

If either choice returns `true`, the answer is `true`.

To avoid recomputation, store results in a DP table `dp[ind][target]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool subsetSumUtil(int ind, int target, vector<int>& arr,
vector<vector<int>>& dp) {
        if (target == 0) return true;
        if (ind == 0) return arr[0] == target;
        if (dp[ind][target] != -1) return dp[ind][target];

        bool notTaken = subsetSumUtil(ind - 1, target, arr, dp);
        bool taken = false;
        if (arr[ind] <= target)
            taken = subsetSumUtil(ind - 1, target - arr[ind], arr,
dp);

        return dp[ind][target] = notTaken || taken;
    }

    bool subsetSumToK(int n, int k, vector<int>& arr) {
        vector<vector<int>> dp(n, vector<int>(k + 1, -1));
        return subsetSumUtil(n - 1, k, arr, dp);
    }
};

int main() {
    vector<int> arr = {1, 2, 3, 4};
```

```
    int k = 4;
    Solution sol;
    cout << sol.subsetSumToK(arr.size(), k, arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*K)`
  There are `N*K` states, each solved once.

- Space Complexity: `O(N*K) + O(N)`
  DP table plus recursion stack.

Approach 2: Tabulation Approach

Algorithm

We convert the recursive solution into bottom-up DP.

Let `dp[i][t]` be `true` if a subset with sum `t` can be formed using elements from index `0` to `i`.

Initialization:

- For all `i`, `dp[i][0] = true` (sum `0` is always possible).

- If `arr[0] <= K`, set `dp[0][arr[0]] = true`.

Transition:

- `notTaken = dp[i-1][t]`

- `taken = dp[i-1][t-arr[i]]` if `arr[i] <= t`

- `dp[i][t] = notTaken || taken`

Final answer is `dp[n-1][K]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool subsetSumToK(int n, int k, vector<int>& arr) {
        vector<vector<bool>> dp(n, vector<bool>(k + 1, false));

        for (int i = 0; i < n; i++)
            dp[i][0] = true;

        if (arr[0] <= k)
            dp[0][arr[0]] = true;

        for (int i = 1; i < n; i++) {
            for (int target = 1; target <= k; target++) {
                bool notTaken = dp[i - 1][target];
                bool taken = false;
                if (arr[i] <= target)
                    taken = dp[i - 1][target - arr[i]];
                dp[i][target] = notTaken || taken;
            }
        }
        return dp[n - 1][k];
    }
};

int main() {
    vector<int> arr = {1, 2, 3, 4};
    int k = 4;
    Solution sol;
    cout << sol.subsetSumToK(arr.size(), k, arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*K)`
  Two nested loops.

- Space Complexity: `O(N*K)`
  2D DP table, no recursion stack.

Approach 3: Space Optimization Approach

Algorithm

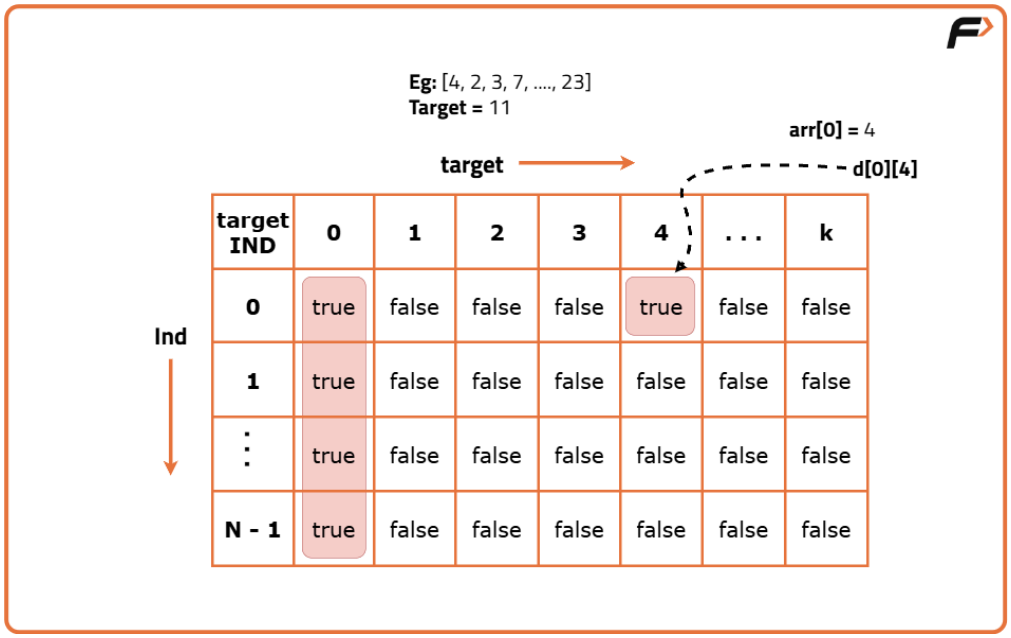
From the relation:
`dp[i][t] = dp[i-1][t] || dp[i-1][t-arr[i]]`

We only need the **previous row**, so we use a 1D array.

Steps:

- Use a boolean array `prev` of size `K+1`.
- Initialize `prev[0] = true`.
- If `arr[0] <= K`, set `prev[arr[0]] = true`.
- For each new element, create `cur` array:
  - `cur[0] = true`
  - Update `cur[target]` using values from `prev`.
- Move `cur` into `prev`.

Final answer is `prev[K]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool subsetSumToK(int n, int k, vector<int>& arr) {
        vector<bool> prev(k + 1, false);
        prev[0] = true;

        if (arr[0] <= k)
            prev[arr[0]] = true;

        for (int i = 1; i < n; i++) {
            vector<bool> cur(k + 1, false);
            cur[0] = true;
            for (int target = 1; target <= k; target++) {
                bool notTaken = prev[target];
                bool taken = false;
                if (arr[i] <= target)
                    taken = prev[target - arr[i]];
                cur[target] = notTaken || taken;
            }
```

```
            prev = cur;
        }
        return prev[k];
    }
};

int main() {
    vector<int> arr = {1, 2, 3, 4};
    int k = 4;
    Solution sol;
    cout << sol.subsetSumToK(arr.size(), k, arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*K)`
  Each element processes all target values.

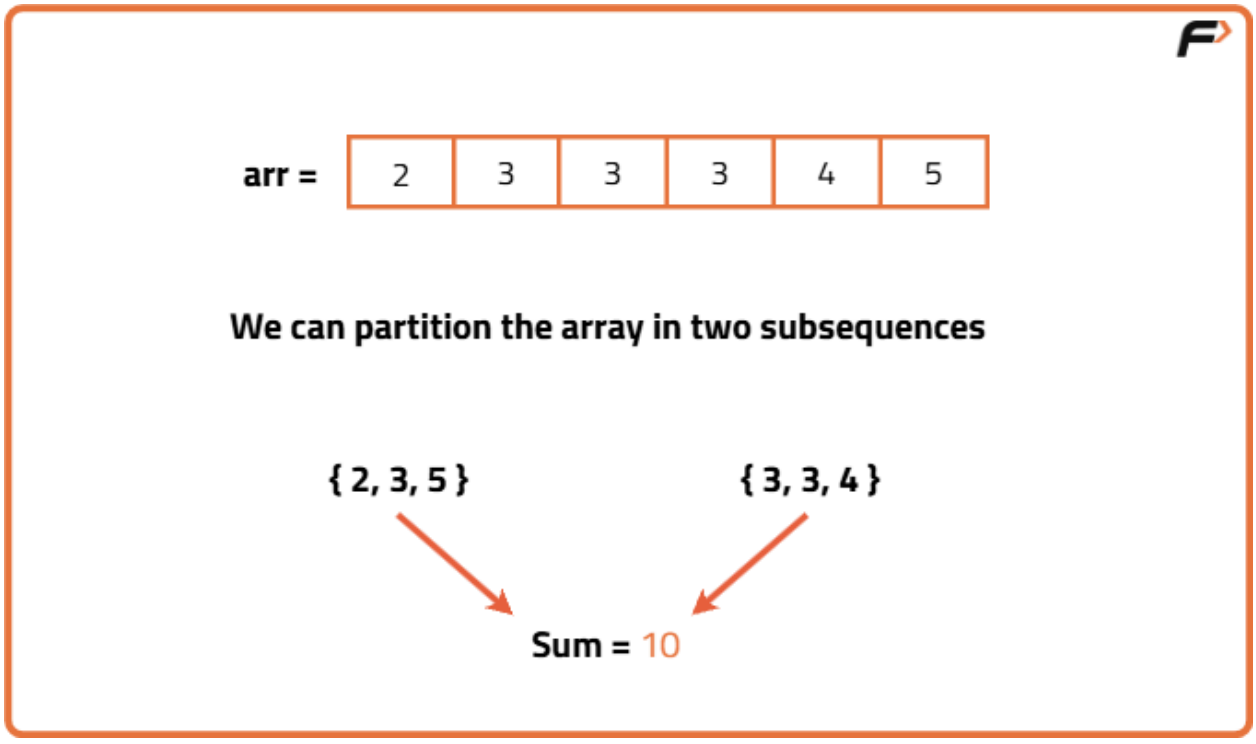
- Space Complexity: `O(K)`
  Only one DP row is stored.

Partition Equal Subset Sum (DP-15)

We are given an array `arr` of n integers.
We need to check whether the array can be divided into **two subsets** such that the **sum of elements in both subsets is equal**.

If such a partition exists, return `true`, otherwise return `false`.

Explanation idea:
If the total sum of the array is S, then each subset must have sum `S/2`.
So the problem reduces to checking whether there exists a **subset with sum = S/2**.



Memoization Approach

Algorithm

This problem is a direct modification of **Subset Sum Equal to Target**.

1. Calculate the total sum of the array.
2. If the total sum is **odd**, it is impossible to split it into two equal subsets, so return `false`.
3. Otherwise, set `target = totalSum / 2`.
4. Now check whether there exists a subset with sum equal to `target`.

Define a recursive function `f(ind, target)`:

- It returns `true` if a subset with sum `target` can be formed using elements from index `0` to `ind`.

Base cases:

- If `target == 0`, return `true` (subset found).
- If `ind == 0`, return `true` if `arr[0] == target`, else `false`.

Recursive choices:

- **Not take** the current element → `f(ind-1, target)`
- **Take** the current element (only if `arr[ind] <= target`) → `f(ind-1, target-arr[ind])`

Return `true` if any of the above choices gives `true`.

To avoid recomputation, store results in a DP table `dp[ind][target]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

bool subsetSumUtil(int ind, int target, vector<int>& arr,
vector<vector<int>>& dp) {
    if (target == 0) return true;
    if (ind == 0) return arr[0] == target;
    if (dp[ind][target] != -1) return dp[ind][target];

    bool notTaken = subsetSumUtil(ind - 1, target, arr, dp);
    bool taken = false;
    if (arr[ind] <= target)
        taken = subsetSumUtil(ind - 1, target - arr[ind], arr, dp);

    return dp[ind][target] = notTaken || taken;
}

bool canPartition(int n, vector<int>& arr) {
    int totalSum = 0;
    for (int i = 0; i < n; i++) totalSum += arr[i];
    if (totalSum % 2 != 0) return false;
```

```
    int target = totalSum / 2;
    vector<vector<int>> dp(n, vector<int>(target + 1, -1));
    return subsetSumUtil(n - 1, target, arr, dp);
}

int main() {
    vector<int> arr = {2, 3, 3, 3, 4, 5};
    int n = arr.size();

    if (canPartition(n, arr))
        cout << "The Array can be partitioned into two equal subsets";
    else
        cout << "The Array cannot be partitioned into two equal
subsets";

    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*K)`
  There are `N*K` states (`N` = number of elements, `K` = target sum).

- Space Complexity: `O(N*K) + O(N)`
  DP table plus recursion stack.

Tabulation Approach

Algorithm

This is the bottom-up version of the memoization solution.

1. Compute total sum. If it is odd, return `false`.

2. Set `target = totalSum / 2`.

3. Create a DP table `dp[n][target+1]` where
   `dp[i][t]` is `true` if a subset with sum `t` can be formed using elements from `0` to `i`.

Initialization:

- For all `i`, set `dp[i][0] = true` (sum `0` is always possible).
- If `arr[0] <= target`, set `dp[0][arr[0]] = true`.

Transition:

- `notTaken = dp[i-1][t]`
- `taken = dp[i-1][t-arr[i]]` if `arr[i] <= t`
- `dp[i][t] = notTaken || taken`

Final answer is `dp[n-1][target]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool canPartition(int n, vector<int>& arr) {
        int totalSum = 0;
        for (int x : arr) totalSum += x;
        if (totalSum % 2 != 0) return false;

        int target = totalSum / 2;
        vector<vector<bool>> dp(n, vector<bool>(target + 1, false));

        for (int i = 0; i < n; i++)
            dp[i][0] = true;

        if (arr[0] <= target)
            dp[0][arr[0]] = true;
```

```
        for (int i = 1; i < n; i++) {
            for (int t = 1; t <= target; t++) {
                bool notTaken = dp[i - 1][t];
                bool taken = false;
                if (arr[i] <= t)
                    taken = dp[i - 1][t - arr[i]];
                dp[i][t] = notTaken || taken;
            }
        }
        return dp[n - 1][target];
    }
};

int main() {
    vector<int> arr = {2, 3, 3, 3, 4, 5};
    Solution obj;
    cout << obj.canPartition(arr.size(), arr);
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*K)`
- Space Complexity: `O(N*K)`

Space Optimization Approach

Algorithm

From the relation
`dp[i][t] = dp[i-1][t] || dp[i-1][t-arr[i]]`

We only need the **previous row**, so we can optimize space to 1D.

Steps:

1. Compute total sum, return `false` if odd.
2. Set `target = totalSum / 2`.
3. Use a 1D array `prev[target+1]`.
4. Initialize:
    - `prev[0] = true`
    - If `arr[0] <= target`, set `prev[arr[0]] = true`
5. For each new element, build a new `cur` array using `prev`.
6. Move `cur` into `prev`.

Final answer is `prev[target]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

bool canPartition(int n, vector<int>& arr) {
    int totalSum = 0;
    for (int x : arr) totalSum += x;
    if (totalSum % 2 != 0) return false;

    int target = totalSum / 2;
    vector<bool> prev(target + 1, false);
    prev[0] = true;

    if (arr[0] <= target)
        prev[arr[0]] = true;

    for (int i = 1; i < n; i++) {
        vector<bool> cur(target + 1, false);
        cur[0] = true;
        for (int t = 1; t <= target; t++) {
            bool notTaken = prev[t];
```

```
            bool taken = false;
            if (arr[i] <= t)
                taken = prev[t - arr[i]];
            cur[t] = notTaken || taken;
        }
        prev = cur;
    }
    return prev[target];
}

int main() {
    vector<int> arr = {2, 3, 3, 3, 4, 5};
    int n = arr.size();

    if (canPartition(n, arr))
        cout << "The Array can be partitioned into two equal subsets";
    else
        cout << "The Array cannot be partitioned into two equal
subsets";

    return 0;
}
```

Complexity Analysis

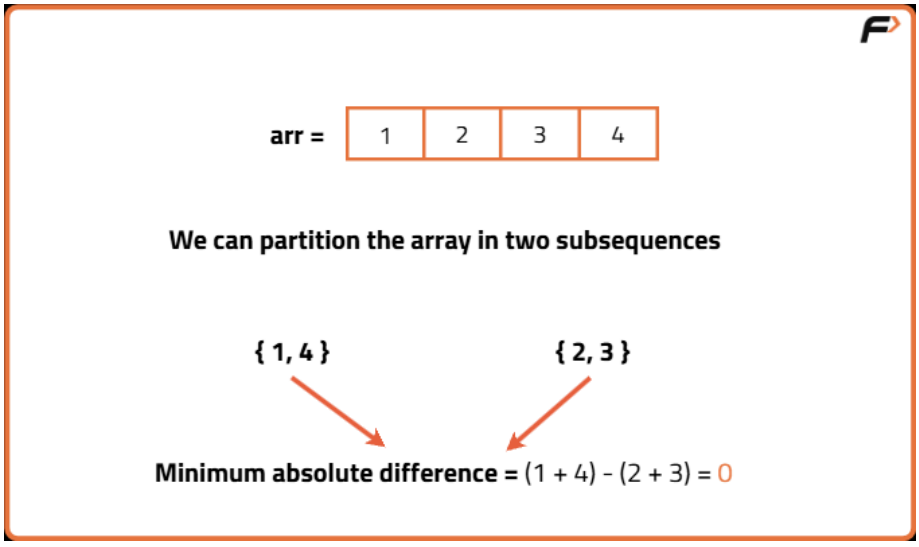
- Time Complexity: `O(N*K)`
- Space Complexity: `O(K)`

Partition Set Into 2 Subsets With Minimum Absolute Sum Difference (DP-16)

We are given an array `arr` of n integers.
We need to partition the array into two subsets such that the **absolute difference between their sums is minimum**.

Key idea:
If the total sum of the array is S and one subset has sum `s1`, then the other subset has sum `S - s1`.
The absolute difference will be `| (S - s1) - s1 | = |S - 2*s1|`.
So, we only need to find all possible subset sums `s1` and choose the one that minimizes this difference.

Example explanation:
For `arr = [1,2,3,4]`, total sum = `10`.
Possible subset sum `s1 = 5` gives difference `|10 - 2*5| = 0`, which is minimum.



Approach 1: Memoization

This problem is a variation of **Subset Sum Equal to Target**.

- First calculate the total sum `totSum`.
- A subset sum can range from `0` to `totSum`.
- Use a recursive function `f(ind, target)` that returns `true` if a subset with sum `target` can be formed using elements from index `0` to `ind`.

Base cases:

- If `target == 0`, return `true`.
- If `ind == 0`, return `true` if `arr[0] == target`, else `false`.

Choices:

- Do not take the current element → `f(ind-1, target)`
- Take the current element (only if `arr[ind] <= target`) → `f(ind-1, target-arr[ind])`

Store results in a DP table to avoid recomputation.

After filling the DP table, iterate over all possible subset sums `s1` that are achievable and compute
`abs(totSum - 2*s1)`.
Return the minimum value.

Code

```
#include <bits/stdc++.h>
using namespace std;

bool subsetSumUtil(int ind, int target, vector<int>& arr,
vector<vector<int>>& dp) {
    if (target == 0) return true;
    if (ind == 0) return arr[0] == target;
    if (dp[ind][target] != -1) return dp[ind][target];

    bool notTaken = subsetSumUtil(ind - 1, target, arr, dp);
    bool taken = false;
```

```
    if (arr[ind] <= target)
        taken = subsetSumUtil(ind - 1, target - arr[ind], arr, dp);

    return dp[ind][target] = notTaken || taken;
}

int minSubsetSumDifference(vector<int>& arr, int n) {
    int totSum = 0;
    for (int x : arr) totSum += x;

    vector<vector<int>> dp(n, vector<int>(totSum + 1, -1));

    for (int s = 0; s <= totSum; s++) {
        subsetSumUtil(n - 1, s, arr, dp);
    }

    int mini = INT_MAX;
    for (int s = 0; s <= totSum; s++) {
        if (dp[n - 1][s]) {
            int diff = abs(totSum - 2 * s);
            mini = min(mini, diff);
        }
    }
    return mini;
}

int main() {
    vector<int> arr = {1, 2, 3, 4};
    cout << minSubsetSumDifference(arr, arr.size());
    return 0;
}
```

Complexity Analysis

- Time Complexity: `O(N*K)`
  `N` is number of elements, `K` is total sum.

- Space Complexity: `O(N*K) + O(N)`
  DP table plus recursion stack.

Approach 2: Tabulation

Algorithm

Convert the memoization solution into bottom-up DP.

- Create a DP table `dp[n][totSum+1]` where
  `dp[i][s] = true` if a subset with sum `s` can be formed using elements `0..i`.
- Initialization:
  - For all `i`, `dp[i][0] = true`.
  - If `arr[0] <= totSum`, set `dp[0][arr[0]] = true`.
- Fill the table using:
  - `notTaken = dp[i-1][s]`
  - `taken = dp[i-1][s-arr[i]]` if `arr[i] <= s`
- After filling, check all `s` where `dp[n-1][s]` is true and compute the minimum difference.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minSubsetSumDifference(vector<int>& arr, int n) {
        int totSum = 0;
        for (int x : arr) totSum += x;

        vector<vector<bool>> dp(n, vector<bool>(totSum + 1, false));
```

```
        for (int i = 0; i < n; i++)
            dp[i][0] = true;

        if (arr[0] <= totSum)
            dp[0][arr[0]] = true;

        for (int i = 1; i < n; i++) {
            for (int s = 1; s <= totSum; s++) {
                bool notTaken = dp[i - 1][s];
                bool taken = false;
                if (arr[i] <= s)
                    taken = dp[i - 1][s - arr[i]];
                dp[i][s] = notTaken || taken;
            }
        }

        int mini = INT_MAX;
        for (int s = 0; s <= totSum; s++) {
            if (dp[n - 1][s]) {
                int diff = abs(totSum - 2 * s);
                mini = min(mini, diff);
            }
        }
        return mini;
    }
};

int main() {
    vector<int> arr = {1, 2, 3, 4};
    Solution sol;
    cout << sol.minSubsetSumDifference(arr, arr.size());
    return 0;
}
```

Complexity Analysis

- Time Complexity: O(N*K)

- Space Complexity: `O(N*K)`

Approach 3: Space Optimization

Algorithm

From the relation
`dp[i][s] = dp[i-1][s] || dp[i-1][s-arr[i]]`,
we only need the previous row.

- Use a 1D boolean array `prev`.
- Initialize `prev[0] = true`.
- If `arr[0] <= totSum`, set `prev[arr[0]] = true`.
- For each element, build a new `cur` array using `prev`.
- After processing all elements, check all achievable sums and compute the minimum difference.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minSubsetSumDifference(vector<int>& arr, int n) {
        int totSum = 0;
        for (int x : arr) totSum += x;

        vector<bool> prev(totSum + 1, false);
        prev[0] = true;
        if (arr[0] <= totSum)
            prev[arr[0]] = true;

        for (int i = 1; i < n; i++) {
```

```
            vector<bool> cur(totSum + 1, false);
            cur[0] = true;
            for (int s = 1; s <= totSum; s++) {
                bool notTaken = prev[s];
                bool taken = false;
                if (arr[i] <= s)
                    taken = prev[s - arr[i]];
                cur[s] = notTaken || taken;
            }
            prev = cur;
        }

        int mini = INT_MAX;
        for (int s = 0; s <= totSum; s++) {
            if (prev[s]) {
                int diff = abs(totSum - 2 * s);
                mini = min(mini, diff);
            }
        }
        return mini;
    }
};

int main() {
    vector<int> arr = {1, 2, 3, 4};
    Solution sol;
    cout << sol.minSubsetSumDifference(arr, arr.size());
    return 0;
}
```

Complexity Analysis

- Time Complexity: $O(N*K)$
- Space Complexity: $O(K)$

Count Subsets with Sum K (DP-17)

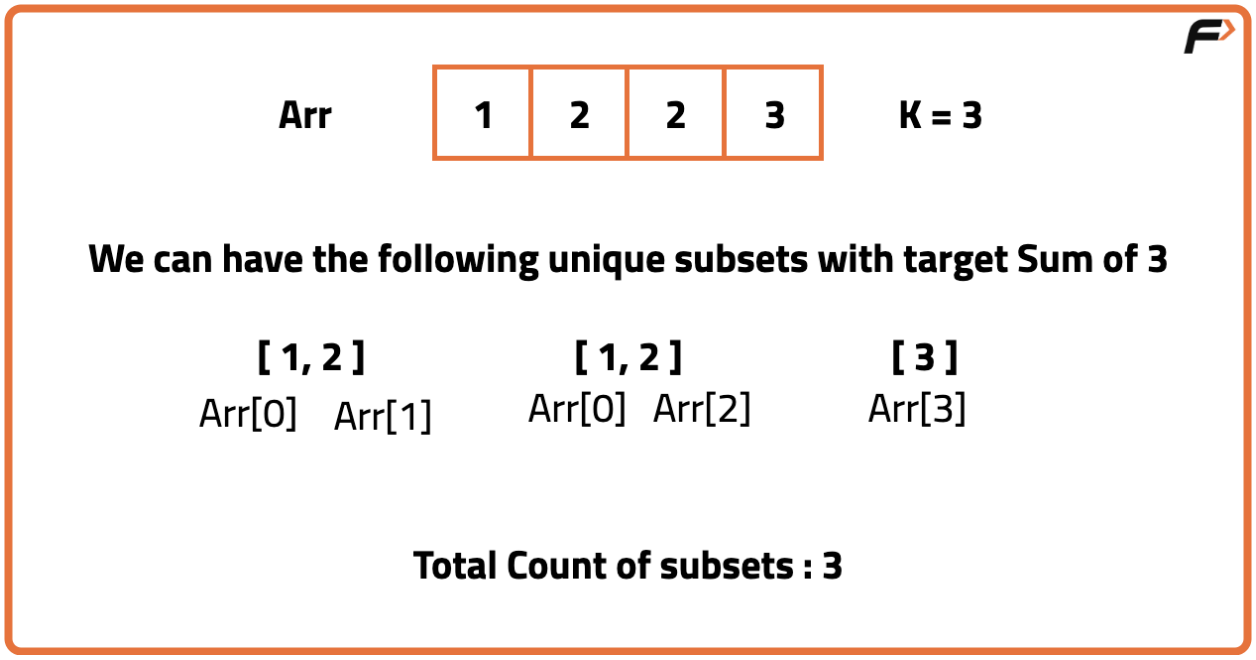
We are given an array `arr` of n integers and an integer K.
We need to **count the number of subsets** whose sum is exactly K.

A subset can be **non-contiguous**, but elements must follow the original order.

For every element, we have **two choices**:

1. **Include** the element → reduce target by `arr[i]`
2. **Exclude** the element → target remains the same

This naturally leads to a **pick / not-pick DP pattern**.



Memoization Approach

- Define `dp[ind][target]` = number of subsets using elements `0..ind` with sum = `target`

Recursive relation:

```
dp[ind][target] =
    dp[ind-1][target]                  // not pick
  + dp[ind-1][target - arr[ind]]       // pick (if arr[ind] <= target)
```

-

Base Cases

- If `target == 0` → return 1 (empty subset)
- If `ind == 0`:
    - return 1 if `arr[0] == target`
    - else `0`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int ind, int target, vector<int>& arr,
vector<vector<int>>& dp) {
        if (target == 0) return 1;
        if (ind == 0) return (arr[0] == target);

        if (dp[ind][target] != -1)
            return dp[ind][target];

        int notTake = solve(ind - 1, target, arr, dp);
```

```
        int take = 0;
        if (arr[ind] <= target)
            take = solve(ind - 1, target - arr[ind], arr, dp);

        return dp[ind][target] = take + notTake;
    }

    int countSubsets(vector<int>& arr, int K) {
        int n = arr.size();
        vector<vector<int>> dp(n, vector<int>(K + 1, -1));
        return solve(n - 1, K, arr, dp);
    }
};

int main() {
    vector<int> arr = {1, 2, 2, 3};
    int K = 3;
    Solution obj;
    cout << obj.countSubsets(arr, K);
    return 0;
}
```

Complexity

- **Time:** `O(N * K)`
- **Space:** `O(N * K)` + recursion stack

Tabulation Approach

Algorithm

- `dp[i][t]` = number of subsets using elements `0..i` with sum `t`
- Initialize:

- `dp[i][0] = 1` for all `i`
- `dp[0][arr[0]] = 1` (if `arr[0] <= K`)

- Fill table using the same include / exclude logic

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countSubsets(vector<int>& arr, int K) {
        int n = arr.size();
        vector<vector<int>> dp(n, vector<int>(K + 1, 0));

        for (int i = 0; i < n; i++)
            dp[i][0] = 1;

        if (arr[0] <= K)
            dp[0][arr[0]] = 1;

        for (int i = 1; i < n; i++) {
            for (int t = 0; t <= K; t++) {
                int notTake = dp[i - 1][t];
                int take = 0;
                if (arr[i] <= t)
                    take = dp[i - 1][t - arr[i]];
                dp[i][t] = take + notTake;
            }
        }
        return dp[n - 1][K];
    }
};

int main() {
    vector<int> arr = {1, 2, 2, 3};
    int K = 3;
```

```
    Solution obj;
    cout << obj.countSubsets(arr, K);
    return 0;
}
```

Complexity

- **Time:** `O(N * K)`
- **Space:** `O(N * K)`

Space Optimized Approach

Algorithm

We observe:

`dp[i][t] depends only on dp[i-1][t] and dp[i-1][t - arr[i]]`

So we only need **one 1D array**.

- `dp[t]` = number of subsets with sum `t`
- Traverse `t` **backwards** to avoid overwriting needed values

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countSubsets(vector<int>& arr, int K) {
        vector<int> dp(K + 1, 0);
        dp[0] = 1;
```

```
        if (arr[0] <= K)
            dp[arr[0]]++;

        for (int i = 1; i < arr.size(); i++) {
            for (int t = K; t >= arr[i]; t--) {
                dp[t] += dp[t - arr[i]];
            }
        }
        return dp[K];
    }
};

int main() {
    vector<int> arr = {1, 2, 2, 3};
    int K = 3;
    Solution obj;
    cout << obj.countSubsets(arr, K);
    return 0;
}
```

Complexity

- **Time:** `O(N * K)`
- **Space:** `O(K)`

Count Partitions with Given Difference (DP-18)

Given an array of **N positive integers** and an integer **D**, we need to count the number of ways to partition the array into two subsets **S1** and **S2** such that
S1 − S2 = D and **S1 ≥ S2**.

The order of elements inside subsets does not matter, but each element must belong to exactly one subset.

Example Explanation

Example 1
Input: `arr = [1, 1, 2, 3]`, `D = 1`

Total sum = 1 + 1 + 2 + 3 = 7

Let sum of S1 = x and sum of S2 = y

We know:

- x − y = 1
- x + y = 7

Adding both equations:
2x = 8 → x = 4

So the problem becomes:
Count subsets with sum = 4

Valid subsets with sum 4:

- [1, 1, 2]
- [1, 3] (using first 1)
- [1, 3] (using second 1)

So, answer = **3**

Approach

1. Let totalSum be the sum of all elements in the array.

2. Let subset sums be S1 and S2.
3. From the problem:
    - S1 − S2 = D
    - S1 + S2 = totalSum
4. Adding both equations:
    - 2 × S1 = totalSum + D
    - S1 = (totalSum + D) / 2
5. If `(totalSum + D)` is odd, partition is impossible → return 0.
6. If `D > totalSum`, partition is impossible → return 0.
7. Now the problem reduces to:
    - **Count the number of subsets with sum = target**
    - where `target = (totalSum + D) / 2`
8. Use Dynamic Programming to count subsets with given sum.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int countPartitions(vector<int>& arr, int d) {
        int totalSum = accumulate(arr.begin(), arr.end(), 0);

        // Invalid cases
        if (d > totalSum || (totalSum + d) % 2 != 0)
            return 0;
```

```
        int target = (totalSum + d) / 2;
        int n = arr.size();

        vector<int> dp(target + 1, 0);

        // Base case: one way to make sum 0 (empty subset)
        dp[0] = 1;

        // First element handling
        if (arr[0] <= target)
            dp[arr[0]] += 1;

        // Fill DP array
        for (int i = 1; i < n; i++) {
            vector<int> curr(target + 1, 0);
            curr[0] = 1;

            for (int sum = 0; sum <= target; sum++) {
                int notTake = dp[sum];
                int take = 0;
                if (arr[i] <= sum)
                    take = dp[sum - arr[i]];
                curr[sum] = take + notTake;
            }
            dp = curr;
        }

        return dp[target];
    }
};

int main() {
    Solution sol;
    vector<int> arr = {1, 1, 2, 3};
    int d = 1;
    cout << sol.countPartitions(arr, d) << endl;
    return 0;
}
```

Complexity Analysis

Time Complexity:
O(N × K)
Where N is the number of elements and K = (totalSum + D) / 2.
Each DP state is computed once.

Space Complexity:
O(K)
We use a single 1D DP array of size `target + 1`.

Assign Cookies

A teacher wants to distribute cookies to students.
Each student can get **at most one cookie**.

- `student[i]` = minimum cookie size required by the ith student
- `cookie[j]` = size of the jth cookie
- A cookie can be assigned if `cookie[j] >= student[i]`

The task is to **maximize the number of students who get a cookie**.

Example Explanation

Example 1
Student = [1, 2, 3]
Cookie = [1, 1]

Sort both arrays:
Student → [1, 2, 3]
Cookie → [1, 1]

- Cookie 1 → satisfies student 1
- Next cookie 1 → cannot satisfy student 2

So only **1 student** is satisfied.

Example 2
Student = [1, 2]
Cookie = [1, 2, 3]

Sorted arrays remain the same.

- Cookie 1 → student 1
- Cookie 2 → student 2

So **2 students** are satisfied.

Approach 1: Memoization

Algorithm

- Sort both arrays.
- Use recursion to try all possibilities.
- State: `(studentIndex, cookieIndex)`
- Choices:
    - Skip current cookie.
    - If cookie can satisfy student, assign it.

- Store results in a 2D DP table to avoid recomputation.
- Stop when all students or cookies are processed.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findContentChildren(vector<int>& student, vector<int>& cookie)
{
        sort(student.begin(), student.end());
        sort(cookie.begin(), cookie.end());

        int n = student.size(), m = cookie.size();
        vector<vector<int>> dp(n, vector<int>(m, -1));

        return solve(0, 0, student, cookie, dp);
    }

private:
    int solve(int i, int j, vector<int>& student, vector<int>& cookie,
              vector<vector<int>>& dp) {
        if (i == student.size() || j == cookie.size())
            return 0;

        if (dp[i][j] != -1)
            return dp[i][j];

        int skip = solve(i, j + 1, student, cookie, dp);
        int take = 0;

        if (cookie[j] >= student[i])
            take = 1 + solve(i + 1, j + 1, student, cookie, dp);

        return dp[i][j] = max(skip, take);
    }
```

```
};

int main() {
    vector<int> student = {1, 2, 3};
    vector<int> cookie = {1, 1};

    Solution obj;
    cout << obj.findContentChildren(student, cookie);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n × m)
  Every student–cookie pair is evaluated once.

- **Space Complexity:** O(n × m) + O(n + m)
  DP table + recursion stack.

Approach 2: Tabulation

Algorithm

- Sort both arrays.
- Create a DP table where `dp[i][j]` represents the maximum students satisfied using students from `i` and cookies from `j`.
- Fill the table bottom-up.
- At each cell:
    - Skip cookie.
    - Assign cookie if possible.
- Final answer is at `dp[0][0]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findContentChildren(vector<int>& student, vector<int>& cookie)
{
        sort(student.begin(), student.end());
        sort(cookie.begin(), cookie.end());

        int n = student.size(), m = cookie.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

        for (int i = n - 1; i >= 0; i--) {
            for (int j = m - 1; j >= 0; j--) {
                int skip = dp[i][j + 1];
                int take = 0;
                if (cookie[j] >= student[i])
                    take = 1 + dp[i + 1][j + 1];
                dp[i][j] = max(skip, take);
            }
        }
        return dp[0][0];
    }
};

int main() {
    vector<int> student = {1, 2};
    vector<int> cookie = {1, 2, 3};

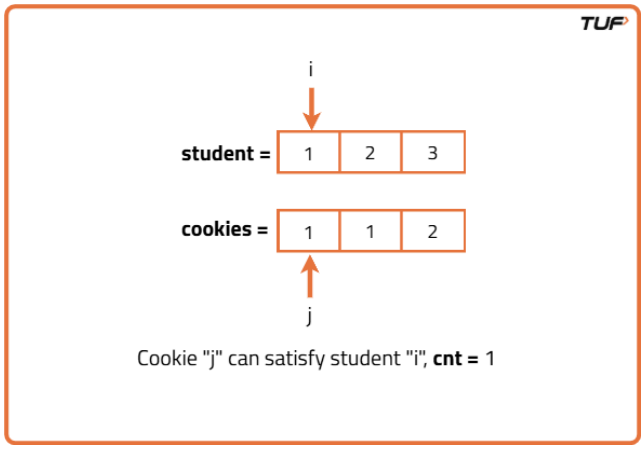
    Solution obj;
    cout << obj.findContentChildren(student, cookie);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n × m)
- **Space Complexity:** O(n × m)

Approach 3: Optimal (Greedy)

Algorithm

- Sort both arrays.
- Use two pointers:
    - One for students
    - One for cookies
- If the current cookie satisfies the current student:
    - Assign it and move both pointers.
- Otherwise:
    - Skip the cookie.
- Count how many students are satisfied.
- 

.....

This works because we always give the **smallest possible cookie** to the **least greedy student**, leaving bigger cookies for others.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findContentChildren(vector<int>& student, vector<int>& cookie)
{
        sort(student.begin(), student.end());
        sort(cookie.begin(), cookie.end());

        int i = 0, j = 0;

        while (i < student.size() && j < cookie.size()) {
            if (cookie[j] >= student[i]) {
                i++;
            }
            j++;
        }
        return i;
    }
};

int main() {
    vector<int> student = {1, 2, 3};
    vector<int> cookie = {1, 1};

    Solution obj;
    cout << obj.findContentChildren(student, cookie);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** O(n log n + m log m)
  Sorting both arrays.

- **Space Complexity:** O(1)
  No extra space used.

Minimum Coins (DP-20)

Problem Statement

Given an array `coins[ ]` of different denominations and an integer `amount`, return the **minimum number of coins** required to make that amount.
You have **infinite supply** of each coin.
If it's not possible, return **-1**.

Example 1

Input: `coins = [1, 2, 5]`, `amount = 11`
Output: 3

Explanation (step-by-step):

- Try largest useful coins first conceptually:
  - 11 = 5 + 5 + 1 → **3 coins**
- No combination uses fewer than 3 coins.
- Hence answer = **3**

Example 2

Input: `coins = [2, 5]`, `amount = 3`
Output: `-1`

Explanation:

- Possible sums using 2 and 5: 2, 4, 5, 6, 7, …
- We can **never form 3**
- Hence answer = **-1**

Why Greedy Fails ❌

Greedy = always pick the largest coin first.

Counter Example

`coins = [1, 5, 6, 9], amount = 11`

Greedy picks:

- 9 + 1 + 1 → **3 coins**

Optimal:

- 5 + 6 → **2 coins**

So greedy does **not** guarantee minimum coins.

Key DP Idea

At every step:

- Either **pick** a coin (stay on same index, reduce amount)
- Or **don't pick** it (try other coins)

We want the **minimum count**, not the number of ways.

Memoization (Top-Down DP)

State

`dp[x]` = minimum coins needed to make amount x

Transitions

`dp[x] = min(1 + dp[x - coin]) for all coin ≤ x`

Base Cases

- `dp[0] = 0`
- If amount < 0 → invalid
- If no solution → return -1

Memoization Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, -2); // -2 = not computed
        return solve(coins, amount, dp);
    }

private:
    int solve(vector<int>& coins, int rem, vector<int>& dp) {
        if (rem == 0) return 0;
        if (rem < 0) return -1;
        if (dp[rem] != -2) return dp[rem];

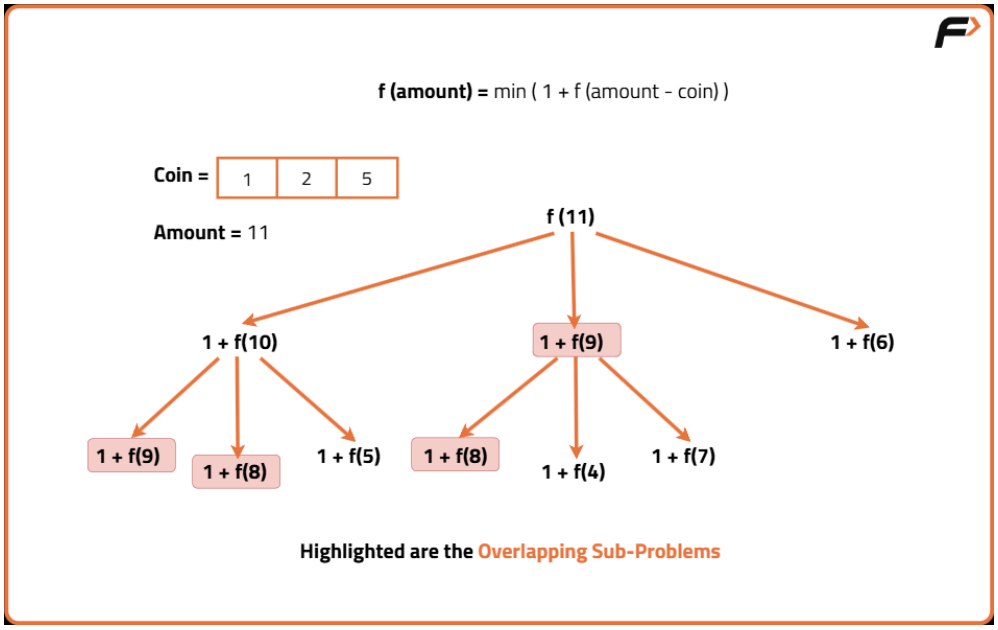
        int mini = INT_MAX;
        for (int coin : coins) {
```

```
            int res = solve(coins, rem - coin, dp);
            if (res >= 0)
                mini = min(mini, 1 + res);
        }

        dp[rem] = (mini == INT_MAX) ? -1 : mini;
        return dp[rem];
    }
};
```

Complexity

- **Time:** `O(N × amount)`
- **Space:** `O(amount)` + recursion stack



Tabulation (Bottom-Up DP)

`dp[x]` = minimum coins needed to make amount x

Initialization

- `dp[0] = 0`
- All others = ∞

Iteration

For each amount from 1 → `amount`, try all coins.

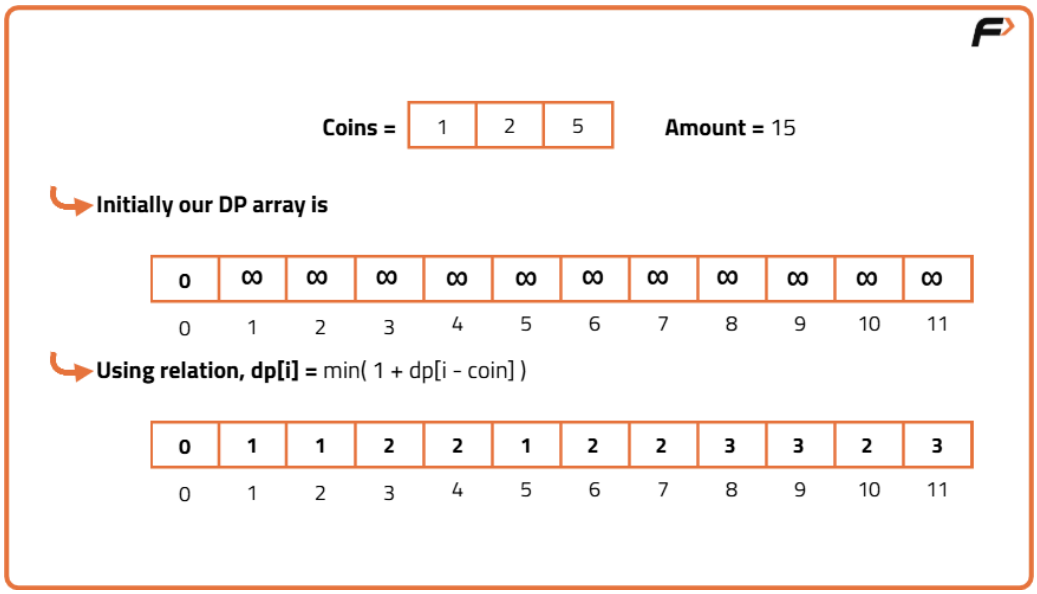
Tabulation Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        vector<int> dp(amount + 1, INT_MAX);
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (i - coin >= 0 && dp[i - coin] != INT_MAX) {
                    dp[i] = min(dp[i], 1 + dp[i - coin]);
                }
            }
        }

        return dp[amount] == INT_MAX ? -1 : dp[amount];
    }
};
```



Example Dry Run (coins = [1,2,5], amount = 5)

Amount	dp
0	0
1	1
2	1
3	2
4	2
5	1

Complexity

- **Time:** `O(N × amount)`
- **Space:** `O(amount)`

Space Optimized DP (Unbounded Knapsack Style)

We can reuse the **same row** because coins can be used unlimited times.

Space Optimized Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minimumElements(vector<int>& coins, int T) {
        int n = coins.size();
        vector<int> dp(T + 1, 1e9);
        dp[0] = 0;

        for (int coin : coins) {
            for (int t = coin; t <= T; t++) {
                dp[t] = min(dp[t], 1 + dp[t - coin]);
            }
        }

        return dp[T] >= 1e9 ? -1 : dp[T];
    }
};
```

Complexity

- **Time:** $O(N \times T)$
- **Space:** $O(T)$

Target Sum (DP - 21)

We are given an array `ARR` of size `N` and an integer `Target`. We must place either a `+` or `-` sign in front of every element of the array. After placing signs in front of all elements, we add them together. Our task is to count the total number of different ways in which the final sum becomes exactly equal to `Target`.

Example 1:
nums = [1,1,1,1,1], target = 3

Possible valid expressions:
-1 + 1 + 1 + 1 + 1 = 3
+1 - 1 + 1 + 1 + 1 = 3
+1 + 1 - 1 + 1 + 1 = 3
+1 + 1 + 1 - 1 + 1 = 3
+1 + 1 + 1 + 1 - 1 = 3

So total ways = 5.

Example 2:
nums = [1], target = 1
Only one way: +1 = 1, so answer = 1.

Approach 1: Memoization

Algorithm

Trying all `+` and `-` combinations directly is exponential. Instead, we reduce the problem to a known DP problem.

Let:

- `totSum` = sum of all array elements
- Elements with `+` sign form subset S1
- Elements with `-` sign form subset S2

We know:
S1 - S2 = target
S1 + S2 = totSum

Solving these equations:
S2 = (totSum - target) / 2

So the problem becomes:
Count number of subsets whose sum is `(totSum - target) / 2`

Edge cases:

- If `totSum < target`, answer is 0
- If `(totSum - target)` is odd, answer is 0

Now we count subsets with given sum using recursion + memoization.

Recursive definition `f(ind, target)`:

- Number of ways to form `target` using elements from index `0` to `ind`.

Base cases:

- If `ind == 0`:
    - If `target == 0` and `arr[0] == 0`, return 2 (pick or not pick)
    - If `target == 0` or `target == arr[0]`, return 1
    - Else return 0

Choices:

- Not pick current element → `f(ind-1, target)`
- Pick current element (if possible) → `f(ind-1, target - arr[ind])`

Store results in `dp[ind][target]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        int totalSum = accumulate(nums.begin(), nums.end(), 0);
        if ((totalSum - target) < 0 || (totalSum - target) % 2 != 0)
            return 0;

        int subsetSum = (totalSum - target) / 2;
        vector<vector<int>> dp(nums.size(), vector<int>(subsetSum + 1,
-1));
        return countSubsets(nums, nums.size() - 1, subsetSum, dp);
    }

    int countSubsets(vector<int>& nums, int ind, int target,
vector<vector<int>>& dp) {
        if (ind == 0) {
            if (target == 0 && nums[0] == 0) return 2;
            if (target == 0 || target == nums[0]) return 1;
            return 0;
        }
        if (dp[ind][target] != -1) return dp[ind][target];

        int notPick = countSubsets(nums, ind - 1, target, dp);
        int pick = 0;
        if (nums[ind] <= target)
            pick = countSubsets(nums, ind - 1, target - nums[ind],
dp);

        return dp[ind][target] = pick + notPick;
    }
};

int main() {
    vector<int> nums = {1,1,1,1,1};
```

```
    int target = 3;
    Solution sol;
    cout << sol.findTargetSumWays(nums, target);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*K)
There are `N*K` states and each is computed once.

Space Complexity: O(N*K) + O(N)
`N*K` for DP table and `O(N)` recursion stack.

Approach 2: Tabulation

Algorithm

We convert the memoization solution into bottom-up DP.

Let `newTarget = (totalSum + target) / 2`.

Create a DP table:
`dp[i][j]` = number of ways to form sum `j` using first `i` elements.

Initialization:

- `dp[i][0] = 1` for all `i` (empty subset gives sum 0)

Transition:

- Not pick: `dp[i-1][j]`
- Pick (if possible): `dp[i-1][j - nums[i-1]]`

Final answer: `dp[n][newTarget]`.

Eg: [4, 2, 3, 7, ..., 23]
Target = 11

target →

Ind ↓

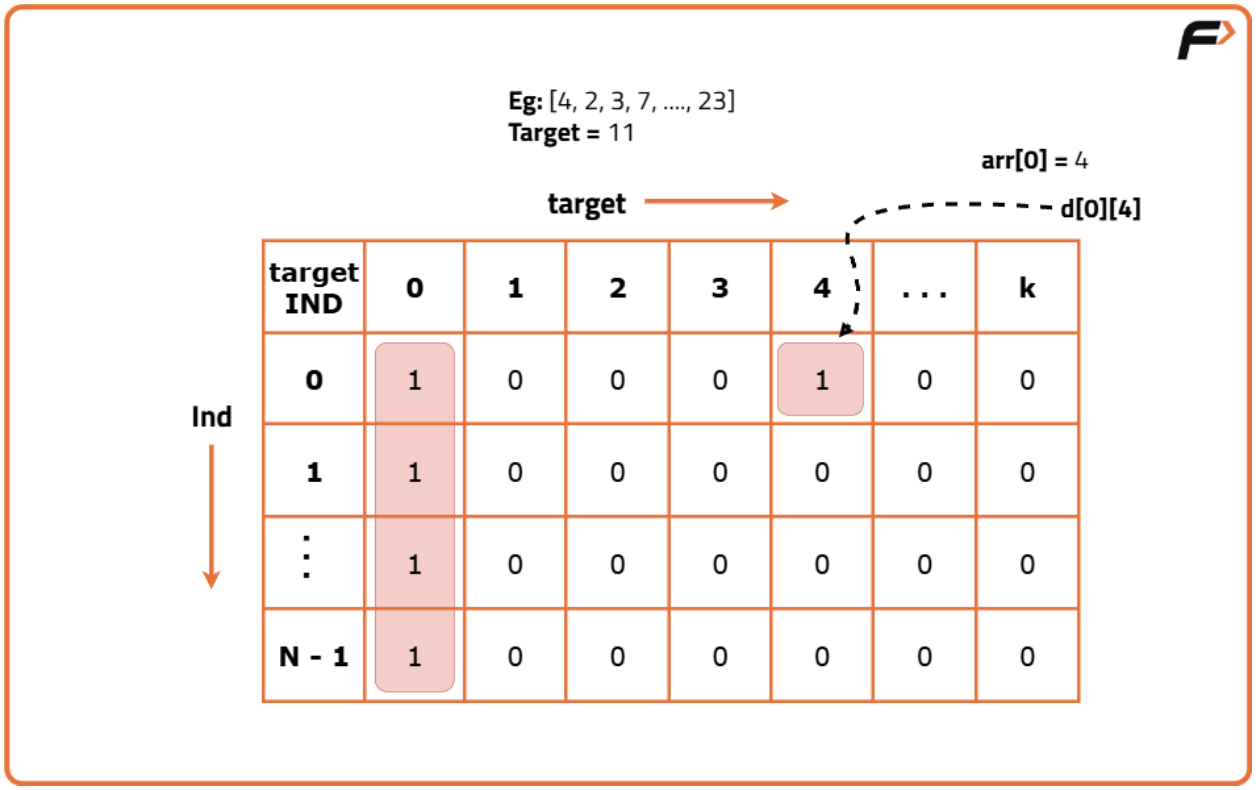
target /ind	0	1	2	3	4	. . .	k
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
⋮	0	0	0	0	0	0	0
N - 1	0	0	0	0	0	0	0

Eg: [4, 2, 3, 7, ..., 23]
Target = 11

target →

Ind ↓

target /ind	0	1	2	3	4	. . .	k
0	1	false	false	false	false	false	false
1	1	false	false	false	false	false	false
⋮	1	false	false	false	false	false	false
N - 1	1	false	false	false	false	false	false



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        int n = nums.size();
        int totalSum = accumulate(nums.begin(), nums.end(), 0);
        if ((totalSum + target) % 2 != 0 || abs(target) > totalSum)
            return 0;

        int newTarget = (totalSum + target) / 2;
        vector<vector<int>> dp(n + 1, vector<int>(newTarget + 1, 0));

        for (int i = 0; i <= n; i++) dp[i][0] = 1;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j <= newTarget; j++) {
                dp[i][j] = dp[i - 1][j];
```

```
                if (nums[i - 1] <= j)
                    dp[i][j] += dp[i - 1][j - nums[i - 1]];
            }
        }
        return dp[n][newTarget];
    }
};

int main() {
    vector<int> nums = {1,1,1,1,1};
    int target = 3;
    Solution sol;
    cout << sol.findTargetSumWays(nums, target);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*K)
Two nested loops over elements and target.

Space Complexity: O(N*K)
Full DP table is used, recursion stack removed.

Approach 3: Space Optimization

Algorithm

From the relation:
dp[ind][target] = dp[ind-1][target] + dp[ind-1][target - arr[ind]]

We only need the previous row. So we use a 1D DP array.

Steps:

- Compute `newTarget = (totalSum + target) / 2`
- Initialize `dp[0] = 1`

- For each number, update dp from right to left

Final answer is `dp[newTarget]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findTargetSumWays(vector<int>& nums, int target) {
        int total = accumulate(nums.begin(), nums.end(), 0);
        if ((total + target) % 2 != 0 || abs(target) > total)
            return 0;

        int newTarget = (total + target) / 2;
        vector<int> dp(newTarget + 1, 0);
        dp[0] = 1;

        for (int num : nums) {
            for (int j = newTarget; j >= num; j--) {
                dp[j] += dp[j - num];
            }
        }
        return dp[newTarget];
    }
};

int main() {
    vector<int> nums = {1,1,1,1,1};
    int target = 3;
    Solution sol;
    cout << sol.findTargetSumWays(nums, target);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*K)
Each element updates all possible target sums once.

Space Complexity: O(K)
Only one DP array of size K+1 is used.

Coin Change 2 (DP - 22)

We are given an array `Arr` of `N` **distinct coin denominations** and an integer `Target`. We have an **infinite supply** of each coin. Our task is to **count the total number of different ways** to form the target sum using these coins.
Order of coins does **not** matter.

Example 1:
coins = [2,4,10], amount = 10

Possible combinations:
10 = 10
10 = 4 + 4 + 2
10 = 4 + 2 + 2 + 2
10 = 2 + 2 + 2 + 2 + 2

So output = 4

Example 2:
coins = [5], amount = 5

Only one way:
5 = 5

So output = 1

Approach 1: Memoization

Algorithm

This is a classic **unbounded knapsack / coin change counting** problem.

We define a recursive function
`countWaysToMakeChangeUtil(ind, T)`
which returns the number of ways to form sum `T` using coins from index `0` to `ind`.

Base case:

- If `ind == 0`, we can only use the first coin.
- If `T` is divisible by `arr[0]`, there is exactly **1 way**.
- Otherwise, there are **0 ways**.

At each index, we have two choices:

- **Not Taken**: Do not use current coin → move to `ind-1`
- **Taken**: Use current coin → stay at `ind` and reduce `T` by `arr[ind]`

We store results in a DP table `dp[ind][T]` to avoid recomputation.

Final answer is `countWaysToMakeChange(n-1, target)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

long countWaysToMakeChangeUtil(vector<int>& arr, int ind, int T,
vector<vector<long>>& dp) {
    if (ind == 0) {
        return (T % arr[0] == 0);
    }
    if (dp[ind][T] != -1)
        return dp[ind][T];
```

```
    long notTaken = countWaysToMakeChangeUtil(arr, ind - 1, T, dp);
    long taken = 0;
    if (arr[ind] <= T)
        taken = countWaysToMakeChangeUtil(arr, ind, T - arr[ind], dp);

    return dp[ind][T] = notTaken + taken;
}

long countWaysToMakeChange(vector<int>& arr, int n, int T) {
    vector<vector<long>> dp(n, vector<long>(T + 1, -1));
    return countWaysToMakeChangeUtil(arr, n - 1, T, dp);
}

int main() {
    vector<int> arr = {1, 2, 3};
    int target = 4;
    int n = arr.size();
    cout << countWaysToMakeChange(arr, n, target);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × T)**
Each state `(ind, T)` is computed once.

Space Complexity: **O(N × T) + O(T)**
DP table takes N*T space and recursion stack can go up to T in worst case.

Approach 2: Tabulation

Algorithm

We convert the memoization approach into an iterative DP solution.

Create a DP table:
`dp[ind][target]` = number of ways to form `target` using coins from `0` to `ind`.

Initialization:

- For `ind = 0`, if `target` is divisible by `arr[0]`, set `dp[0][target] = 1`.

Transition:

- **Not Taken**: `dp[ind-1][target]`
- **Taken**: `dp[ind][target - arr[ind]]` (same row because coins are unlimited)

Final answer is `dp[n-1][T]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

long countWaysToMakeChange(vector<int>& arr, int n, int T) {
    vector<vector<long>> dp(n, vector<long>(T + 1, 0));

    for (int i = 0; i <= T; i++) {
        if (i % arr[0] == 0)
            dp[0][i] = 1;
    }

    for (int ind = 1; ind < n; ind++) {
        for (int target = 0; target <= T; target++) {
            long notTaken = dp[ind - 1][target];
            long taken = 0;
            if (arr[ind] <= target)
                taken = dp[ind][target - arr[ind]];
            dp[ind][target] = notTaken + taken;
        }
    }
    return dp[n - 1][T];
}
```

```
int main() {
    vector<int> arr = {1, 2, 3};
    int target = 4;
    int n = arr.size();
    cout << countWaysToMakeChange(arr, n, target);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × T)**
Two nested loops over coins and target.

Space Complexity: **O(N × T)**
DP table of size N*T. No recursion stack.

Approach 3: Space Optimization

Algorithm

From the relation:
`dp[ind][target] = dp[ind-1][target] + dp[ind][target - arr[ind]]`

We observe:

- We only need the **previous row** and the **current row**.

Steps:

- Initialize a 1D array `prev` for the first coin.
- For each next coin, build a `cur` array using `prev` and itself.
- After processing a coin, assign `cur` to `prev`.

Final answer is `prev[T]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

long countWaysToMakeChange(vector<int>& arr, int n, int T) {
    vector<long> prev(T + 1, 0);

    for (int i = 0; i <= T; i++) {
        if (i % arr[0] == 0)
            prev[i] = 1;
    }

    for (int ind = 1; ind < n; ind++) {
        vector<long> cur(T + 1, 0);
        for (int target = 0; target <= T; target++) {
            long notTaken = prev[target];
            long taken = 0;
            if (arr[ind] <= target)
                taken = cur[target - arr[ind]];
            cur[target] = notTaken + taken;
        }
        prev = cur;
    }
    return prev[T];
}

int main() {
    vector<int> arr = {1, 2, 3};
    int target = 4;
    int n = arr.size();
    cout << countWaysToMakeChange(arr, n, target);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × T)**
Each coin processes all target values once.

Space Complexity: **O(T)**
Only two 1D arrays of size T+1 are used.

Unbounded Knapsack (DP - 23)

A thief wants to rob a store and has a bag with capacity `W`. The store has n items and each item is available in **infinite quantity**.
The weight of items is given in array `wt[ ]` and their corresponding values are given in array `val[ ]`.
The thief can either take an item completely or not take it at all (no fractions allowed).
An item can be taken **any number of times**.
We need to find the **maximum total value** the thief can steal without exceeding the bag capacity.

Example 1:
n = 3, W = 8
wt = [2,4,6], val = [5,11,13]

Best choice is taking item of weight 4 twice:
4 + 4 = 8, value = 11 + 11 = 22

Output = 22

Example 2:
n = 2, W = 3
wt = [2,1], val = [4,2]

We can take item of weight 1 three times:
1 + 1 + 1 = 3, value = 2 + 2 + 2 = 6

Output = 6

Greedy does not work because choosing the item with highest value or best value/weight ratio locally may block better combinations later.

Approach 1: Memoization

Algorithm

We solve this using recursion with memoization.

Define a function `f(ind, W)` which returns the **maximum value** we can get using items from index `0` to `ind` with remaining capacity `W`.

Base case:

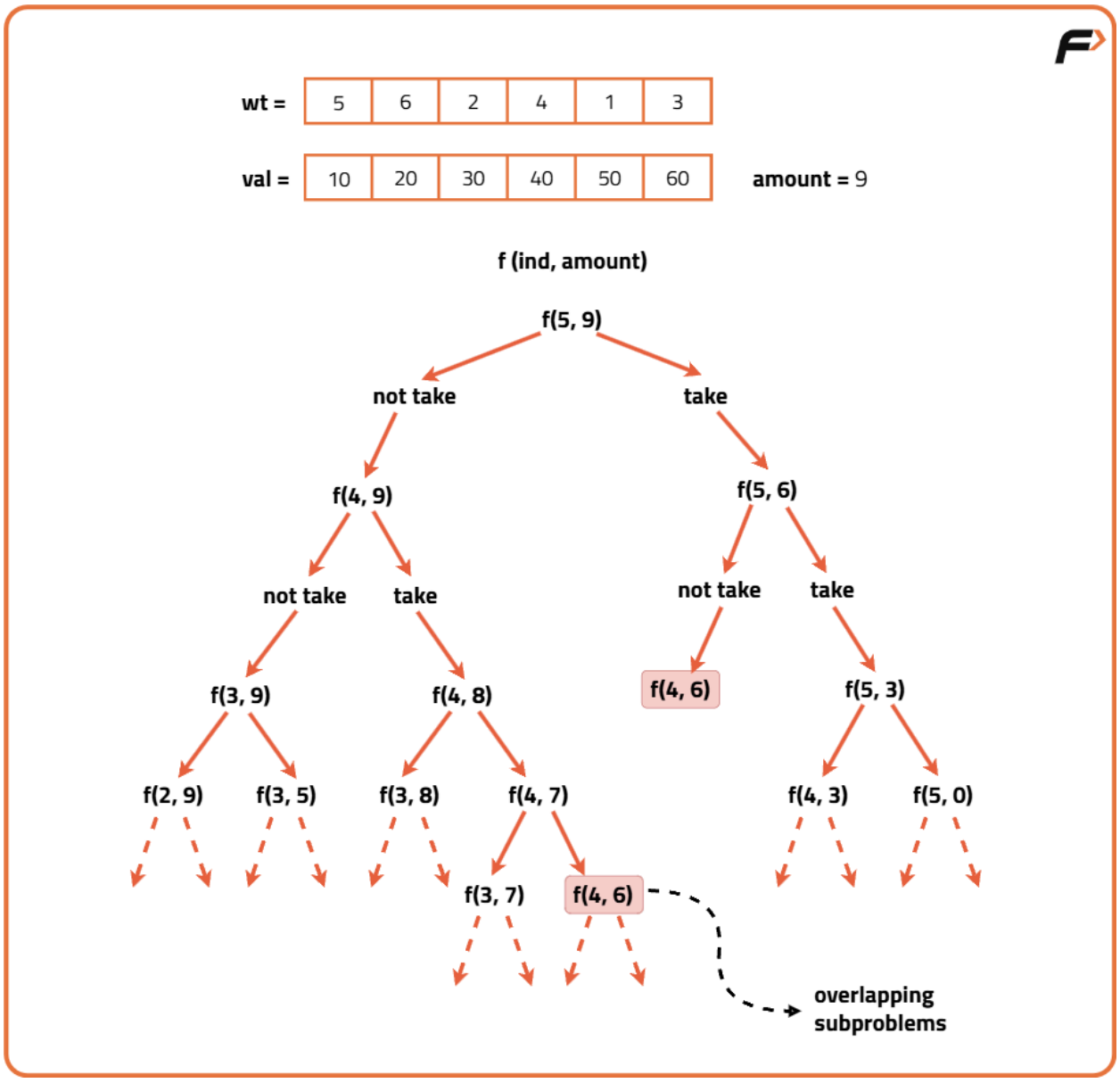
- If `ind == 0`, we only have the first item.
- Since items are unlimited, we take it `(W / wt[0])` times.
- Value = `(W / wt[0]) * val[0]`.

Choices at index `ind`:

- **Not Take**: skip current item → `f(ind-1, W)`
- **Take**: take current item and stay at same index (unbounded)
  → `val[ind] + f(ind, W - wt[ind])` (only if `wt[ind] <= W`)

We take the maximum of both choices.

To avoid recomputation, store results in `dp[ind][W]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int knapsackUtil(vector<int>& wt, vector<int>& val, int ind, int
W, vector<vector<int>>& dp) {
        if (ind == 0) {
            return (W / wt[0]) * val[0];
```

```
        }
        if (dp[ind][W] != -1)
            return dp[ind][W];

        int notTaken = knapsackUtil(wt, val, ind - 1, W, dp);
        int taken = INT_MIN;
        if (wt[ind] <= W)
            taken = val[ind] + knapsackUtil(wt, val, ind, W - wt[ind],
dp);

        return dp[ind][W] = max(notTaken, taken);
    }

    int unboundedKnapsack(int n, int W, vector<int>& val, vector<int>&
wt) {
        vector<vector<int>> dp(n, vector<int>(W + 1, -1));
        return knapsackUtil(wt, val, n - 1, W, dp);
    }
};

int main() {
    vector<int> wt = {2, 4, 6};
    vector<int> val = {5, 11, 13};
    int W = 10;
    int n = wt.size();

    Solution obj;
    cout << obj.unboundedKnapsack(n, W, val, wt);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N * W)**
Each state `(ind, W)` is solved once.

Space Complexity: **O(N * W) + O(N)**
DP table uses `N*W` space and recursion stack uses `O(N)`.

Approach 2: Tabulation

Algorithm

We convert the recursive solution into bottom-up DP.

Create a DP table:
`dp[ind][cap]` = maximum value using items `0..ind` with capacity `cap`.

Initialization:

- For `ind = 0`, we can take item 0 multiple times:
  `dp[0][cap] = (cap / wt[0]) * val[0]`

Transition:

- **Not Take**: `dp[ind-1][cap]`
- **Take**: `val[ind] + dp[ind][cap - wt[ind]]` (same row because unlimited)

Final answer is `dp[n-1][W]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int unboundedKnapsack(int n, int W, vector<int>& val, vector<int>&
wt) {
        vector<vector<int>> dp(n, vector<int>(W + 1, 0));

        for (int cap = wt[0]; cap <= W; cap++) {
            dp[0][cap] = (cap / wt[0]) * val[0];
        }

        for (int ind = 1; ind < n; ind++) {
```

```
            for (int cap = 0; cap <= W; cap++) {
                int notTaken = dp[ind - 1][cap];
                int taken = INT_MIN;
                if (wt[ind] <= cap)
                    taken = val[ind] + dp[ind][cap - wt[ind]];
                dp[ind][cap] = max(notTaken, taken);
            }
        }
        return dp[n - 1][W];
    }
};

int main() {
    vector<int> wt = {2, 4, 6};
    vector<int> val = {5, 11, 13};
    int W = 10;
    int n = wt.size();

    Solution obj;
    cout << obj.unboundedKnapsack(n, W, val, wt);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N * W)**
Two nested loops over items and capacity.

Space Complexity: **O(N * W)**
DP table of size `N * W`, recursion stack eliminated.

Approach 3: Space Optimization

Algorithm

From the relation:
`dp[ind][cap] = max(dp[ind-1][cap], val[ind] + dp[ind][cap - wt[ind]])`

We notice:

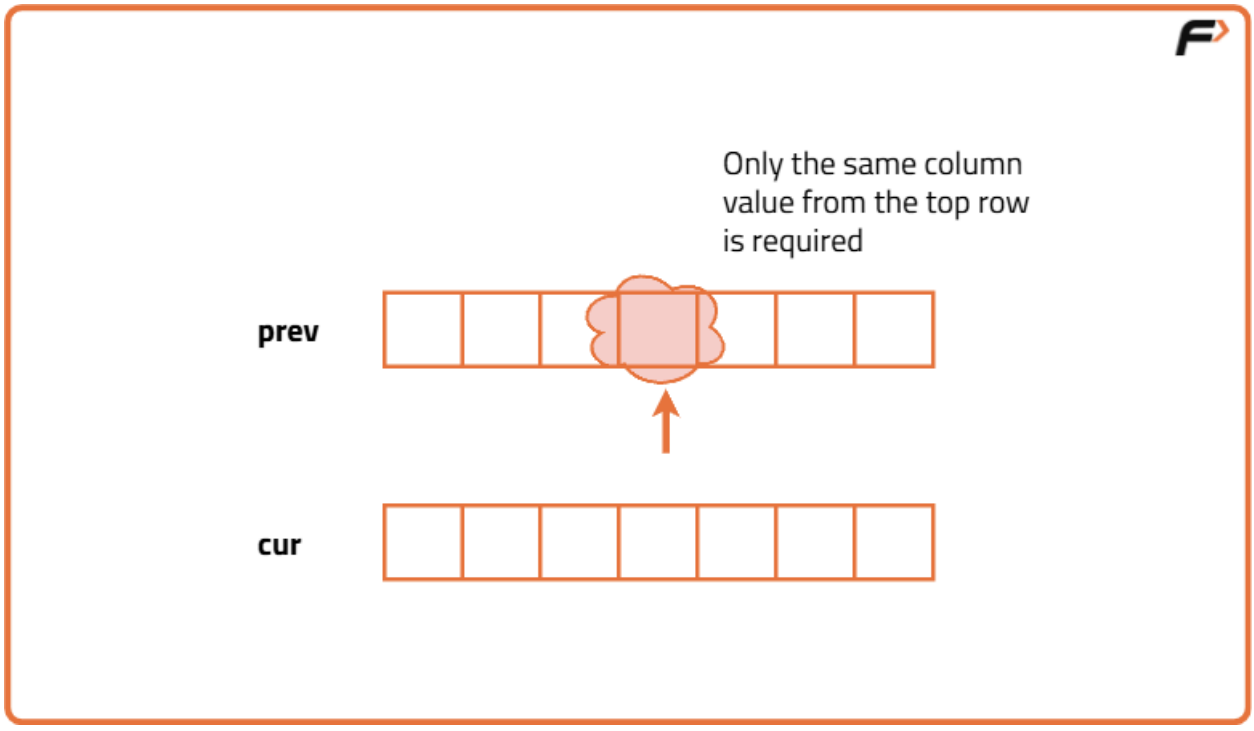
- `dp[ind-1][cap]` → previous value
- `dp[ind][cap - wt[ind]]` → current row value

So we can use **only one 1D array**.

Steps:

- Initialize the array for the first item.
- For each next item, update the same array from left to right.
- This works because unlimited items allow reuse of current row values.

Final answer is stored in `cur[W]`.



Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Solution {
public:
    int unboundedKnapsack(int n, int W, vector<int>& val, vector<int>&
wt) {
        vector<int> cur(W + 1, 0);

        for (int cap = wt[0]; cap <= W; cap++) {
            cur[cap] = (cap / wt[0]) * val[0];
        }

        for (int ind = 1; ind < n; ind++) {
            for (int cap = 0; cap <= W; cap++) {
                int notTaken = cur[cap];
                int taken = INT_MIN;
                if (wt[ind] <= cap)
                    taken = val[ind] + cur[cap - wt[ind]];
                cur[cap] = max(notTaken, taken);
            }
        }
        return cur[W];
    }
};

int main() {
    vector<int> wt = {2, 4, 6};
    vector<int> val = {5, 11, 13};
    int W = 10;
    int n = wt.size();

    Solution obj;
    cout << obj.unboundedKnapsack(n, W, val, wt);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N * W)**
Each item updates all capacities once.

Space Complexity: **O(W)**
Only one array of size `W+1` is used.

Rod Cutting Problem (DP - 24)

We are given a rod of length `N` inches and an array `price[]`, where `price[i]` represents the value of a rod piece of length `(i + 1)` inches.
We can cut the rod into any number of pieces (or not cut it at all) and sell the pieces.
Our goal is to **maximize the total value** obtained by cutting and selling the rod.

This problem is an **unbounded knapsack** problem because:

- Each rod length can be used multiple times.
- We want to maximize value under a length constraint.

Example 1:
price = [1,6,8,9,10,19,7,20], N = 8
Best cut: lengths 2 and 6 → value = 6 + 19 = 25

Example 2:
price = [1,5,8,9], N = 4
Best cut: 2 + 2 → value = 5 + 5 = 10

Greedy does not work because choosing the locally best price may block better future combinations.

Approach 1: Memoization

Algorithm

We use recursion with memoization.

Define a function `f(ind, len)` that returns the **maximum value** we can obtain using rod lengths from index `0` to `ind` to form total length `len`.

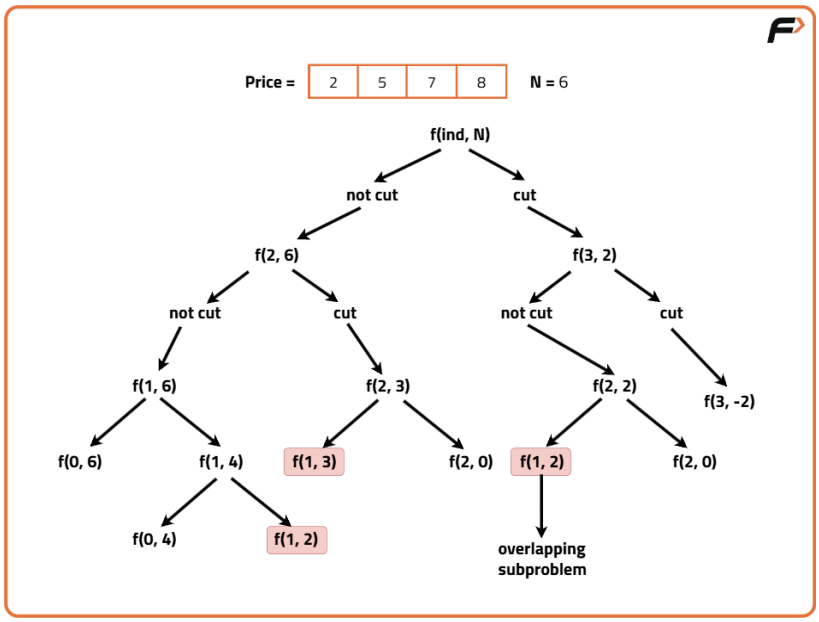
Base case:

- If `ind == 0`, only rod of length 1 is available.
- We can cut it `len` times.
- Value = `len * price[0]`.

Choices at index `ind`:

- **Not Take**: do not use current rod length
  → `f(ind-1, len)`
- **Take**: use current rod length `(ind+1)` and stay at same index
  → `price[ind] + f(ind, len - (ind+1))`
  (only if `(ind+1) <= len`)

We store results in `dp[ind][len]` to avoid recomputation.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int rodCutting(vector<int> price, int n) {
        vector<vector<int>> dp(n, vector<int>(n + 1, 0));

        for(int length = 0; length <= n; length++) {
            dp[0][length] = length * price[0];
        }

        for(int i = 1; i < n; i++) {
            for(int length = 0; length <= n; length++) {
                int notTake = dp[i - 1][length];
                int take = INT_MIN;
                int rodLength = i + 1;

                if(rodLength <= length) {
                    take = price[i] + dp[i][length - rodLength];
                }
                dp[i][length] = max(take, notTake);
            }
        }
        return dp[n - 1][n];
    }
};

int main() {
    int n = 8;
    vector<int> price = {1,5,8,9,10,17,17,20};
    Solution obj;
    cout << obj.rodCutting(price, n);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
Each state `(ind, length)` is computed once.

Space Complexity: **O(N × N)**
We use a 2D DP table.

Approach 2: Tabulation

Algorithm

This is the bottom-up version of the memoization approach.

We create a DP table:
`dp[ind][len]` = maximum value using rod lengths up to `ind` for total length `len`.

Initialization:

- For `ind = 0`,
  `dp[0][len] = len * price[0]` because rod length 1 can be used repeatedly.

Transition:

- **Not Take**: `dp[ind-1][len]`
- **Take**: `price[ind] + dp[ind][len - (ind+1)]`

Final answer is `dp[n-1][n]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int rodCutting(vector<int>& price, int n) {
        vector<vector<int>> dp(n, vector<int>(n + 1, 0));
```

```
        for(int length = 0; length <= n; length++){
            dp[0][length] = price[0] * length;
        }

        for(int ind = 1; ind < n; ind++) {
            for(int length = 1; length <= n; length++) {
                int notTaken = dp[ind - 1][length];
                int taken = INT_MIN;
                int rodLength = ind + 1;

                if(rodLength <= length) {
                    taken = price[ind] + dp[ind][length - rodLength];
                }
                dp[ind][length] = max(notTaken, taken);
            }
        }
        return dp[n - 1][n];
    }
};

int main() {
    vector<int> price = {2,4,6,8};
    int n = price.size();
    Solution sol;
    cout << sol.rodCutting(price, n);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
Two nested loops over rod lengths.

Space Complexity: **O(N × N)**
We store the full DP table.

Approach 3: Space Optimization

Algorithm

From the relation:

```
dp[ind][len] = max(dp[ind-1][len],
                   price[ind] + dp[ind][len - (ind+1)])
```

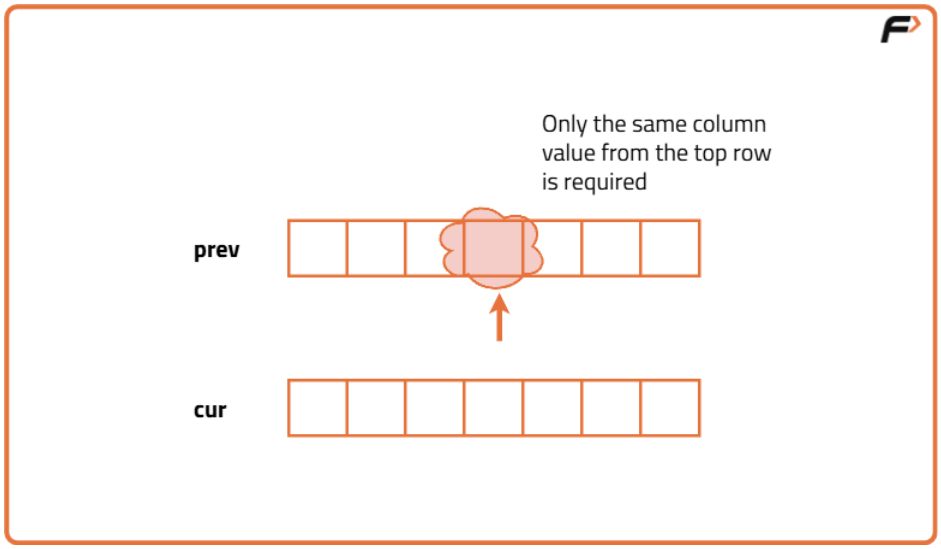
We observe:

- We only need the previous row and current row.
- So we can reduce space to **1D DP**.

Steps:

- Initialize a 1D array for the first rod length.
- Update the same array for each rod length.
- Since it is unbounded, we can reuse current row values.

Final answer will be stored in `prev[n]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int rodCutting(vector<int>& price, int n) {
        vector<int> prev(n + 1, 0), cur(n + 1, 0);

        for(int length = 0; length <= n; length++){
            prev[length] = price[0] * length;
        }

        for(int ind = 1; ind < n; ind++) {
            for(int length = 1; length <= n; length++) {
                int notTaken = prev[length];
                int taken = INT_MIN;
                int rodLength = ind + 1;

                if(rodLength <= length) {
                    taken = price[ind] + cur[length - rodLength];
                }
                cur[length] = max(notTaken, taken);
            }
            prev = cur;
        }
        return prev[n];
    }
};

int main() {
    vector<int> price = {2,4,6,8};
    int n = price.size();
    Solution sol;
    cout << sol.rodCutting(price, n);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
Each rod length processes all possible total lengths.

Space Complexity: **O(N)**
Only two 1D arrays of size N+1 are used.

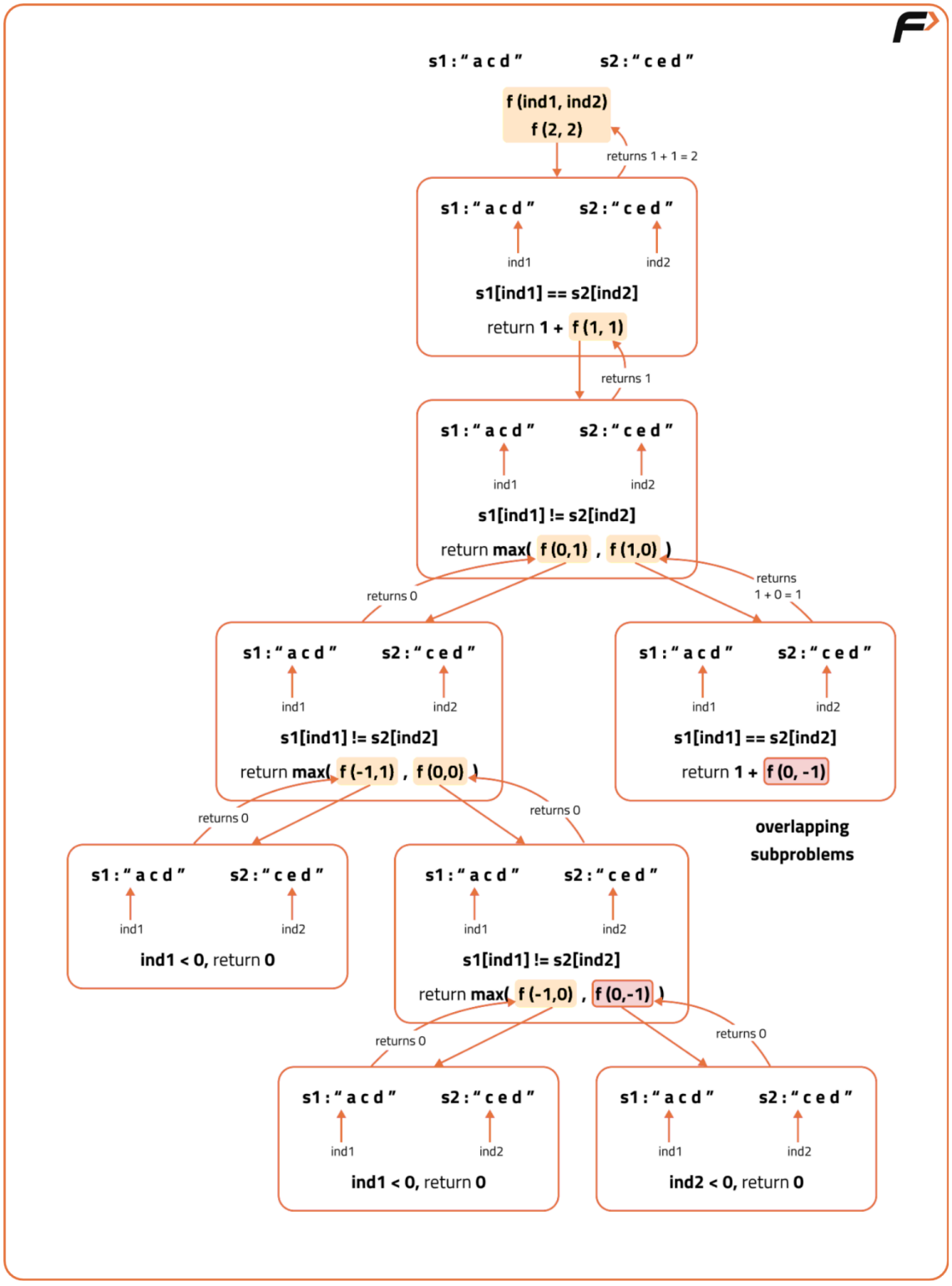
Longest Common Subsequence (DP - 25)

Given two strings `str1` and `str2`, we need to find the **length of the longest common subsequence** present in both strings.
A subsequence keeps the relative order of characters but does not need to be contiguous.

Example 1:
str1 = "bdefg", str2 = "bfg"
The longest common subsequence is `"bfg"`, so the answer is 3.

Example 2:
str1 = "mnop", str2 = "mnq"
The longest common subsequence is `"mn"`, so the answer is 2.

Approach 1: Memoization



Algorithm

We solve the problem using recursion with memoization.

We define a recursive function
`func(ind1, ind2)` → length of LCS considering `str1[0..ind1]` and `str2[0..ind2]`.

Base cases:

- If `ind1 < 0` or `ind2 < 0`, no characters are left, so return 0.

Recursive cases:

- If `str1[ind1] == str2[ind2]`, this character is part of LCS, so
  `1 + func(ind1-1, ind2-1)`

- Otherwise, we try both possibilities and take maximum:
    - `func(ind1, ind2-1)`
    - `func(ind1-1, ind2)`

Since the same `(ind1, ind2)` states repeat, we store results in a DP table `dp[ind1][ind2]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
private:
    int func(string& s1, string& s2, int ind1, int ind2,
vector<vector<int>>& dp) {
        if (ind1 < 0 || ind2 < 0)
            return 0;

        if (dp[ind1][ind2] != -1)
            return dp[ind1][ind2];

        if (s1[ind1] == s2[ind2])
            return dp[ind1][ind2] = 1 + func(s1, s2, ind1 - 1, ind2 -
1, dp);
```

```
        return dp[ind1][ind2] = max(
            func(s1, s2, ind1, ind2 - 1, dp),
            func(s1, s2, ind1 - 1, ind2, dp)
        );
    }

public:
    int lcs(string str1, string str2) {
        int n = str1.size();
        int m = str2.size();
        vector<vector<int>> dp(n, vector<int>(m, -1));
        return func(str1, str2, n - 1, m - 1, dp);
    }
};

int main() {
    string s1 = "acd";
    string s2 = "ced";
    Solution sol;
    cout << sol.lcs(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(n × m)**
There are `n*m` unique `(ind1, ind2)` states, each solved once.

Space Complexity: **O(n × m) + O(n + m)**
`O(n*m)` for DP table and `O(n+m)` for recursion stack.

Approach 2: Tabulation

Algorithm

We convert the recursive solution into a bottom-up DP approach.

We create a DP table `dp[n+1][m+1]` where
`dp[i][j]` = LCS length of `str1[0..i-1]` and `str2[0..j-1]`.

Index shifting is done to handle base cases cleanly.

Base initialization:

- `dp[i][0] = 0` for all `i`
- `dp[0][j] = 0` for all `j`

Transition:

- If characters match:
  `dp[i][j] = 1 + dp[i-1][j-1]`
- Else:
  `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

Final answer is `dp[n][m]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
    int lcs(string str1, string str2) {
        int n = str1.size();
        int m = str2.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (str1[i - 1] == str2[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
```

```
        }
        return dp[n][m];
    }
};

int main() {
    string s1 = "acd";
    string s2 = "ced";
    Solution sol;
    cout << sol.lcs(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(n × m)**
We fill each cell of the DP table once.

Space Complexity: **O(n × m)**
A 2D DP table of size `(n+1) × (m+1)` is used.

Approach 3: Space Optimization

Algorithm

From the tabulation relation, each row depends only on:

- The previous row
- The current row (left value)

So we only keep two 1D arrays:

- `prev` → previous row
- `cur` → current row

Steps:

- Initialize both arrays with 0.
- For each character of `str1`, compute `cur` using `prev`.
- After each row, assign `prev = cur`.

Final answer is stored in `prev[m]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
    int lcs(string str1, string str2) {
        int n = str1.size();
        int m = str2.size();
        vector<int> prev(m + 1, 0), cur(m + 1, 0);

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (str1[i - 1] == str2[j - 1])
                    cur[j] = 1 + prev[j - 1];
                else
                    cur[j] = max(prev[j], cur[j - 1]);
            }
            prev = cur;
        }
        return prev[m];
    }
};

int main() {
    string s1 = "acd";
    string s2 = "ced";
    Solution sol;
    cout << sol.lcs(s1, s2);
```

```
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(n × m)**
We still process all character pairs once.

Space Complexity: **O(m)**
Only two arrays of size `m+1` are used instead of a full 2D table.

Print Longest Common Subsequence (DP - 26)

Given two strings `str1` and `str2`, we need to **print the longest common subsequence (LCS)** between them.
A subsequence maintains the **relative order** of characters but does not need to be contiguous.

This problem is an extension of **Longest Common Subsequence (DP - 25)** where instead of only finding the length, we also **reconstruct and print the actual subsequence**.

Example 1:
str1 = "abcd", str2 = "bdef"
The LCS is `"bd"`.

Example 2:
str1 = "apple", str2 = "waffle"
The LCS is `"ale"`.

Approach

Algorithm

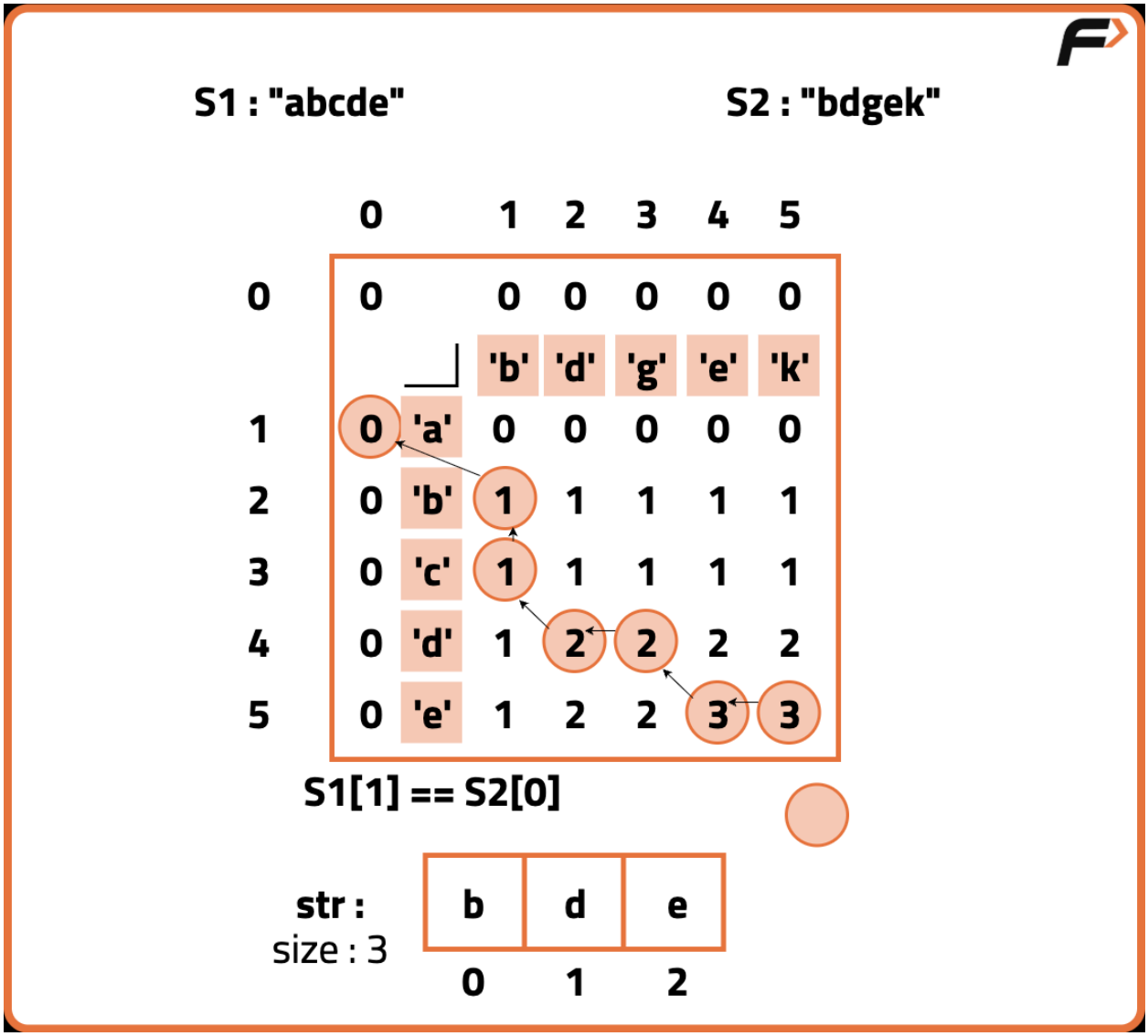
To print the LCS, we first build the **DP table for LCS length**, then **trace back** from the table to construct the subsequence.

Steps:

1. Let n = length of `str1`, `m` = length of `str2`.
2. Create a DP table `dp[n+1][m+1]` where
   `dp[i][j]` stores the length of LCS of `str1[0..i-1]` and `str2[0..j-1]`.
3. Fill the DP table:
   - If `str1[i-1] == str2[j-1]`,
     `dp[i][j] = 1 + dp[i-1][j-1]`
   - Else,
     `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`
4. After filling the table, start from `dp[n][m]` and reconstruct the LCS:
   - If characters match, add the character to answer and move diagonally `(i-1, j-1)`
   - Else, move in the direction of the larger DP value:
     - If `dp[i-1][j] > dp[i][j-1]`, move up
     - Otherwise, move left
5. Continue until `i == 0` or `j == 0`.
6. The LCS is built in reverse order, so reverse it before returning.

Example dry run (str1 = "abcd", str2 = "bdef"):

- DP table gives LCS length = 2
- Tracing back picks `'d'` then `'b'`
- Reverse → `"bd"`



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string longestCommonSubsequence(string &str1, string &str2) {
        int n = str1.size();
        int m = str2.size();

        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
```

```
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (str1[i - 1] == str2[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        int i = n, j = m;
        string lcs = "";

        while (i > 0 && j > 0) {
            if (str1[i - 1] == str2[j - 1]) {
                lcs += str1[i - 1];
                i--;
                j--;
            } else if (dp[i - 1][j] > dp[i][j - 1]) {
                i--;
            } else {
                j--;
            }
        }

        reverse(lcs.begin(), lcs.end());
        return lcs;
    }
};

int main() {
    string str1 = "abcd";
    string str2 = "bdef";

    Solution sol;
    cout << sol.longestCommonSubsequence(str1, str2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M) + O(N + M)**

- `O(N × M)` to build the DP table
- `O(N + M)` to reconstruct the LCS by tracing back

Space Complexity: **O(N × M)**
We use a 2D DP table of size `(n+1) × (m+1)` to store LCS lengths.

Longest Common Substring (DP - 27)

Given two strings `str1` and `str2`, we need to find the **length of the longest common substring** between them.
A substring is a **contiguous** sequence of characters, meaning characters must appear continuously without gaps.

Example 1:
str1 = "abcde", str2 = "abfce"
The longest common substring is `"ab"`, so the answer is 2.

Example 2:
str1 = "abcdxyz", str2 = "xyzabcd"
The longest common substring is `"abcd"`, so the answer is 4.

The key difference from LCS is that here **continuity is mandatory**. If characters do not match, the substring breaks immediately.

Approach 1: Tabulation

Algorithm

We use dynamic programming to build a table where each cell stores the length of the longest common substring **ending at those indices**.

Steps:

1. Let n be the length of `str1` and `m` be the length of `str2`.

2. Create a DP table `dp[n+1][m+1]` initialized with 0.

3. Traverse both strings:

    - If `str1[i-1] == str2[j-1]`, it means the substring can be extended, so
      `dp[i][j] = 1 + dp[i-1][j-1]`

    - If they do not match, continuity breaks, so
      `dp[i][j] = 0`

4. Keep track of the **maximum value** found in the DP table, because the longest common substring may end anywhere, not necessarily at the last index.

5. Return this maximum value as the answer.

Example dry run (str1 = "abcde", str2 = "abfce"):

- Matching `"a"` → length 1

- Matching `"b"` → length 2

- Mismatch at next characters resets length to 0
  Maximum found = 2

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestCommonSubstr(string str1, string str2) {
        int n = str1.size();
        int m = str2.size();
```

```
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
        int ans = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (str1[i - 1] == str2[j - 1]) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                    ans = max(ans, dp[i][j]);
                } else {
                    dp[i][j] = 0;
                }
            }
        }
        return ans;
    }
};

int main() {
    string s1 = "abcde";
    string s2 = "abfce";

    Solution sol;
    cout << sol.longestCommonSubstr(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(n × m)**
Every pair of characters from both strings is compared once.

Space Complexity: **O(n × m)**
A 2D DP table of size `(n+1) × (m+1)` is used.

Approach 2: Space Optimized

Algorithm

From the tabulation relation:
`dp[i][j] = 1 + dp[i-1][j-1]`

We observe that:

- To compute the current row, we only need the **previous row**.
- So instead of a full 2D table, we can use two 1D arrays:
    - `prev` → previous row
    - `cur` → current row

Steps:

1. Initialize two arrays `prev` and `cur` of size `m`+1 with 0.
2. Traverse the strings:
    - If characters match,
      `cur[j] = 1 + prev[j-1]`
    - Else,
      `cur[j] = 0`
3. Update the maximum length found.
4. After each row, assign `prev = cur`.
5. Return the maximum length.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestCommonSubstr(string str1, string str2) {
```

```
        int n = str1.size();
        int m = str2.size();

        vector<int> prev(m + 1, 0), cur(m + 1, 0);
        int ans = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (str1[i - 1] == str2[j - 1]) {
                    cur[j] = 1 + prev[j - 1];
                    ans = max(ans, cur[j]);
                } else {
                    cur[j] = 0;
                }
            }
            prev = cur;
        }
        return ans;
    }
};

int main() {
    string s1 = "abcdxyz";
    string s2 = "xyzabcd";

    Solution sol;
    cout << sol.longestCommonSubstr(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(n × m)**
All character pairs are still processed once.

Space Complexity: **O(m)**
Only two 1D arrays of size `m+1` are used instead of a full 2D DP table.

Longest Palindromic Subsequence (DP - 28)

Given a string `s`, we need to find the **length of the longest palindromic subsequence** present in it.
A palindrome reads the same from left to right and right to left.
A subsequence is formed by deleting some or no characters without changing the order of the remaining characters.

Example 1:
s = "eeeme"
The longest palindromic subsequence is `"eeee"`, so the answer is 4.

Example 2:
s = "annb"
The longest palindromic subsequence is `"nn"`, so the answer is 2.

Approach 1: Tabulation

Algorithm

A palindrome remains the same when reversed.
So, the **Longest Palindromic Subsequence (LPS)** of a string is the same as the **Longest Common Subsequence (LCS)** between the string and its reverse.

Steps:

1. Take the original string `s`.
2. Create another string `t` which is the reverse of `s`.
3. Now find the **LCS length** between `s` and `t`.
4. The obtained LCS length is the length of the longest palindromic subsequence.

DP construction:

- Create a DP table `dp[n+1][n+1]` where
  `dp[i][j]` = LCS length of `s[0..i-1]` and `t[0..j-1]`

- If characters match:
  `dp[i][j] = 1 + dp[i-1][j-1]`

- Else:
  `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

Finally, return `dp[n][n]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
    int longestPalinSubseq(string s){
        string t = s;
        reverse(t.begin(), t.end());

        int n = s.size();
        vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));

        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= n; j++){
                if(s[i - 1] == t[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[n][n];
    }
};
```

```
int main(){
    string s = "bbabcbcab";
    Solution sol;
    cout << sol.longestPalinSubseq(s);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
We fill a DP table of size `N × N`.

Space Complexity: **O(N × N)**
A 2D DP table is used to store LCS values.

Approach 2: Space Optimized

Algorithm

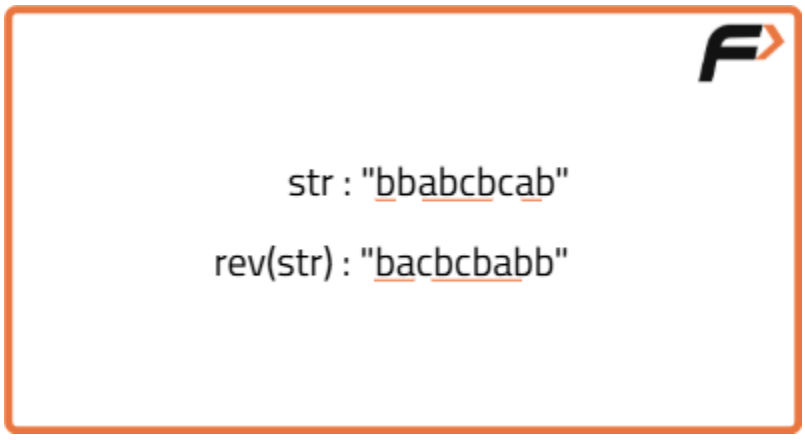
In the tabulation approach, we observe that:

- To compute the current row, we only need the **previous row**.
- So instead of a full 2D DP table, we use **two 1D arrays**.

Steps:

1. Reverse the string `s` to get `t`.
2. Use two arrays:
    - `prev` → represents dp[i-1][*]
    - `cur` → represents dp[i][*]
3. Apply the same LCS logic as before.
4. After each row, assign `prev = cur`.

5. The final answer will be in `prev[n]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
private:
    int lcs(string s1, string s2){
        int n = s1.size();
        int m = s2.size();

        vector<int> prev(m + 1, 0), cur(m + 1, 0);

        for(int i = 1; i <= n; i++){
            for(int j = 1; j <= m; j++){
                if(s1[i - 1] == s2[j - 1])
                    cur[j] = 1 + prev[j - 1];
                else
                    cur[j] = max(prev[j], cur[j - 1]);
            }
            prev = cur;
        }
        return prev[m];
    }

public:
```

```
    int longestPalinSubseq(string s){
        string t = s;
        reverse(t.begin(), t.end());
        return lcs(s, t);
    }
};

int main(){
    string s = "bbabcbcab";
    Solution sol;
    cout << sol.longestPalinSubseq(s);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
All character pairs are processed once.

Space Complexity: **O(N)**
Only two 1D arrays of size N+1 are used instead of a 2D table.

Minimum Insertions to Make String Palindrome (DP - 29)

Given a string s, we need to find the **minimum number of insertions** required to make the string a palindrome.
A palindrome reads the same forward and backward.
We are allowed to insert characters at **any position** in the string.

Example 1:
s = "abcaa"
We can insert "c" and "b" to form "abcacba", which is a palindrome.
So the answer is 2.

Example 2:
s = "ba"
Insert "a" at the beginning to form "aba".
So the answer is 1.

Approach 1: Tabulation

Algorithm

To make a string a palindrome with minimum insertions, we use the concept of **Longest Palindromic Subsequence (LPS)**.

Key idea:

- If we keep the longest palindromic subsequence intact, the remaining characters must be inserted to make the string a palindrome.

- Therefore:
  Minimum insertions = length of string − length of LPS

How to find LPS:

- The Longest Palindromic Subsequence of a string is equal to the **Longest Common Subsequence (LCS)** between the string and its reverse.

Steps:

1. Take the string `s`.

2. Reverse it to get `rev`.

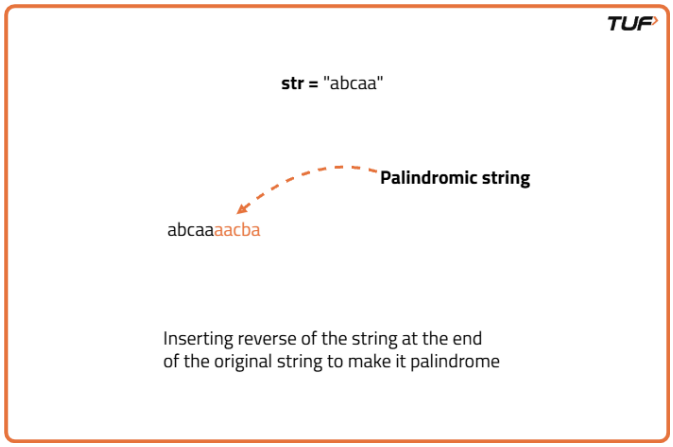
3. Find LCS of `s` and `rev` using DP.

4. Let `k` be the length of LPS.

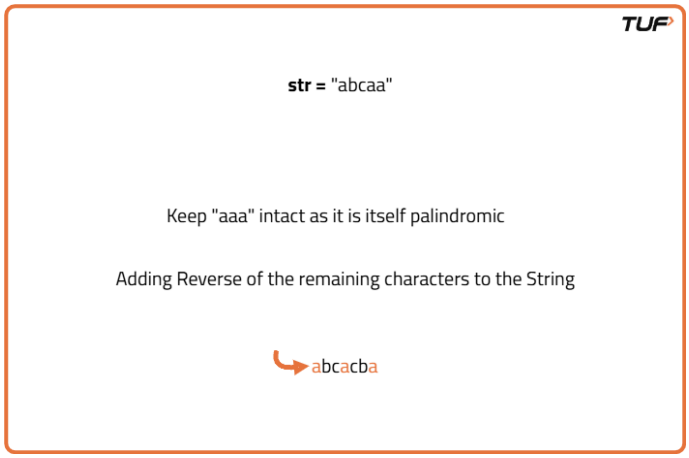
5. Answer = n − k, where n is the length of s.

Example dry run for "abcaa":

- Reverse = "aacba"
- LPS length = 3 ("aaa")
- Minimum insertions = 5 − 3 = 2







Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int lcs(string s1, string s2) {
        int n = s1.size();
        int m = s2.size();
        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (s1[i - 1] == s2[j - 1])
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[n][m];
    }

    int longestPalindromeSubsequence(string s) {
        string t = s;
        reverse(t.begin(), t.end());
        return lcs(s, t);
```

```
    }

    int minInsertion(string s) {
        int n = s.size();
        int k = longestPalindromeSubsequence(s);
        return n - k;
    }
};

int main() {
    Solution obj;
    string s = "abcaa";
    cout << obj.minInsertion(s);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
We compute LCS using a DP table of size `N × N`.

Space Complexity: **O(N × N)**
A 2D DP array is used to store LCS values.

Approach 2: Space Optimized

Algorithm

In the tabulation approach, we only use:

- the previous DP row
- the current DP row

So instead of a full 2D DP array, we use **two 1D arrays**.

Steps:

1. Reverse the string to get `rev`.
2. Use two arrays `prev` and `cur` of size `N+1`.
3. Compute LCS row by row.
4. After each row, assign `prev = cur`.
5. LPS length = final LCS value.
6. Minimum insertions = `n - LPS`.

This approach reduces space while keeping the same logic.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int lcs(string s1, string s2) {
        int n = s1.size();
        int m = s2.size();
        vector<int> prev(m + 1, 0), cur(m + 1, 0);

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (s1[i - 1] == s2[j - 1])
                    cur[j] = 1 + prev[j - 1];
                else
                    cur[j] = max(prev[j], cur[j - 1]);
            }
            prev = cur;
        }
        return prev[m];
    }

    int longestPalindromeSubsequence(string s) {
        string t = s;
```

```
        reverse(t.begin(), t.end());
        return lcs(s, t);
    }

    int minInsertion(string s) {
        int n = s.size();
        int k = longestPalindromeSubsequence(s);
        return n - k;
    }
};

int main() {
    Solution sol;
    string s = "abcaa";
    cout << sol.minInsertion(s);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × N)**
All character pairs are processed once while computing LCS.

Space Complexity: **O(N)**
Only two 1D arrays of size N+1 are used instead of a 2D DP table.

Minimum Insertions/Deletions to Convert String (DP - 30)

We are given two strings `str1` and `str2`.
We are allowed only **two operations on str1**:

1. Delete any character
2. Insert any character

Our task is to find the **minimum number of operations** required to convert `str1` into `str2`.

Pre-requisite: Longest Common Subsequence

Example 1:
str1 = "kitten", str2 = "sitting"
To convert:

- delete `'k'`, insert `'s'`
- delete `'e'`, insert `'i'`
- insert `'g'`
  Total operations = 5

Example 2:
str1 = "flaw", str2 = "lawn"
Delete `'f'`, insert `'n'`
Total operations = 2

Approach 1: Tabulation

Algorithm

Instead of directly minimizing operations, we first think how to **maximize what we can keep unchanged**.

The characters that can remain unchanged in both strings (in the same order) form the **Longest Common Subsequence (LCS)**.

Let:

- `n` = length of `str1`
- `m` = length of `str2`
- `k` = length of LCS of `str1` and `str2`

Steps:

1. Keep the k LCS characters intact.
2. Delete the remaining characters from `str1`: `(n - k)` deletions.
3. Insert the remaining characters of `str2`: `(m - k)` insertions.

So,
Minimum operations = `(n - k) + (m - k)`

To compute k, we use DP to find LCS.

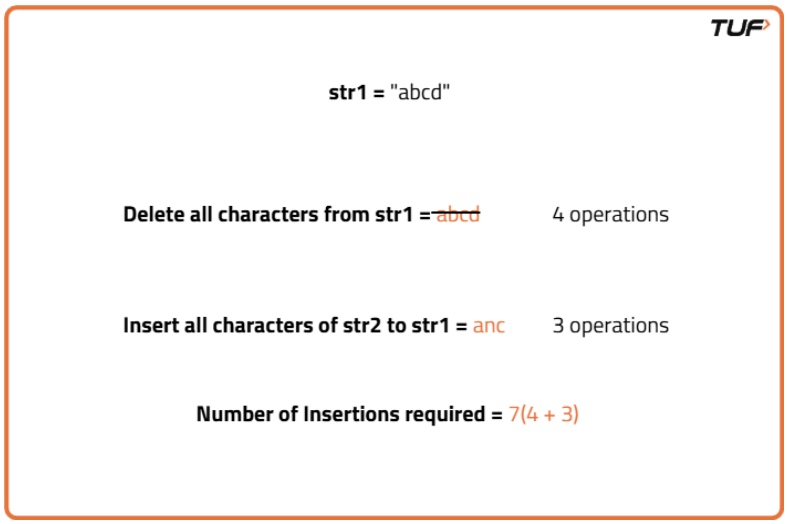
DP definition:

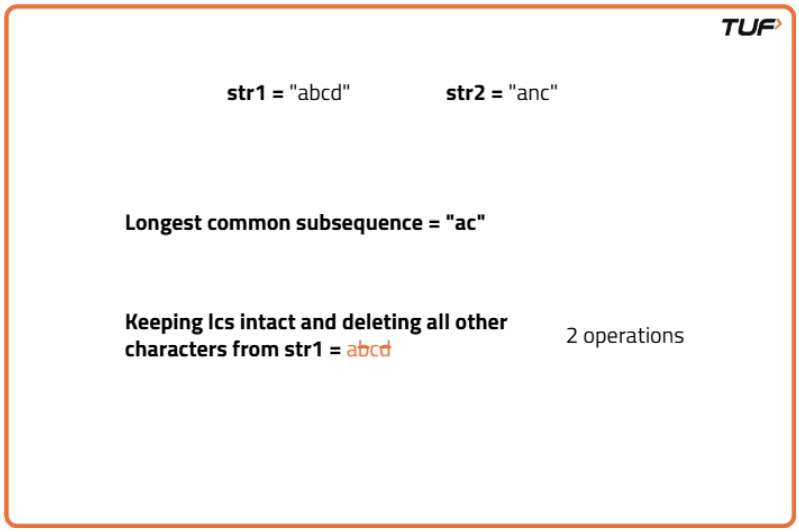
- `dp[i][j]` = length of LCS of `str1[0..i-1]` and `str2[0..j-1]`

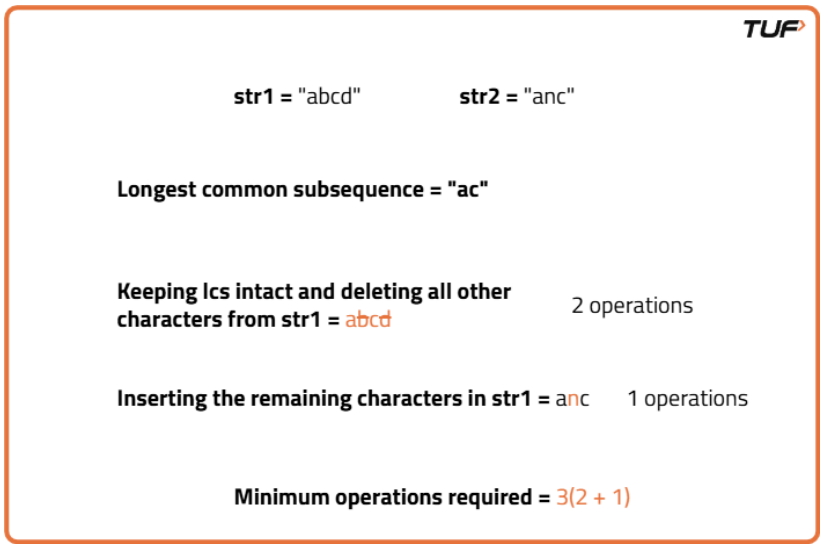
Transition:

- If characters match:
  `dp[i][j] = 1 + dp[i-1][j-1]`
- Else:
  `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

Final LCS length = `dp[n][m]`.








```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minOperations(string s1, string s2) {
        int n = s1.length();
        int m = s2.length();

        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (s1[i - 1] == s2[j - 1])
```

```
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }

        int lcs = dp[n][m];
        return (n - lcs) + (m - lcs);
    }
};

int main() {
    Solution sol;
    string s1 = "heap", s2 = "pea";
    cout << sol.minOperations(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
We fill the entire DP table once.

Space Complexity: **O(N × M)**
A 2D DP table of size $(n+1) \times (m+1)$ is used.

Approach 2: Space Optimized

Algorithm

In the tabulation approach, to compute the current DP row, we only need:

- the previous row
- the current row itself

So we can optimize space by using **two 1D arrays**:

- `prev` → represents `dp[i-1][*]`
- `cur` → represents `dp[i][*]`

Steps:

1. Initialize `prev` and `cur` arrays of size `m+1`.
2. Compute LCS row by row using the same transition.
3. After each row, assign `prev = cur`.
4. LCS length will be in `prev[m]`.
5. Answer = `(n - lcs) + (m - lcs)`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minOperations(string s1, string s2) {
        int n = s1.size(), m = s2.size();

        vector<int> prev(m + 1, 0), cur(m + 1, 0);

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (s1[i - 1] == s2[j - 1])
                    cur[j] = 1 + prev[j - 1];
                else
                    cur[j] = max(prev[j], cur[j - 1]);
            }
            prev = cur;
        }

        int lcs = prev[m];
```

```
        return (n - lcs) + (m - lcs);
    }
};

int main() {
    Solution obj;
    string s1 = "heap", s2 = "pea";
    cout << obj.minOperations(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Each character pair is processed once.

Space Complexity: **O(M)**
Only two 1D arrays of size `m`+1 are used.

Shortest Common Supersequence (DP - 31)

We are given two strings S1 and S2.
We need to return their **shortest common supersequence**.

A supersequence is a string that contains **both S1 and S2 as subsequences**, meaning both strings should appear in the same relative order, but not necessarily contiguously.

Example 1:
str1 = "mno", str2 = "nop"
The shortest common supersequence is `"mnop"`.

Example 2:
str1 = "dynamic", str2 = "program"
The shortest common supersequence is `"dynprogramic"`.

Approach

Algorithm

Pre-requisite: Longest Common Subsequence, Print Longest Common Subsequence

If we simply concatenate both strings, we will always get a supersequence, but it may not be the shortest.
To minimize the length, we should **avoid repeating common characters that appear in the same order** in both strings.

These common characters are exactly the characters of the **Longest Common Subsequence (LCS)**.

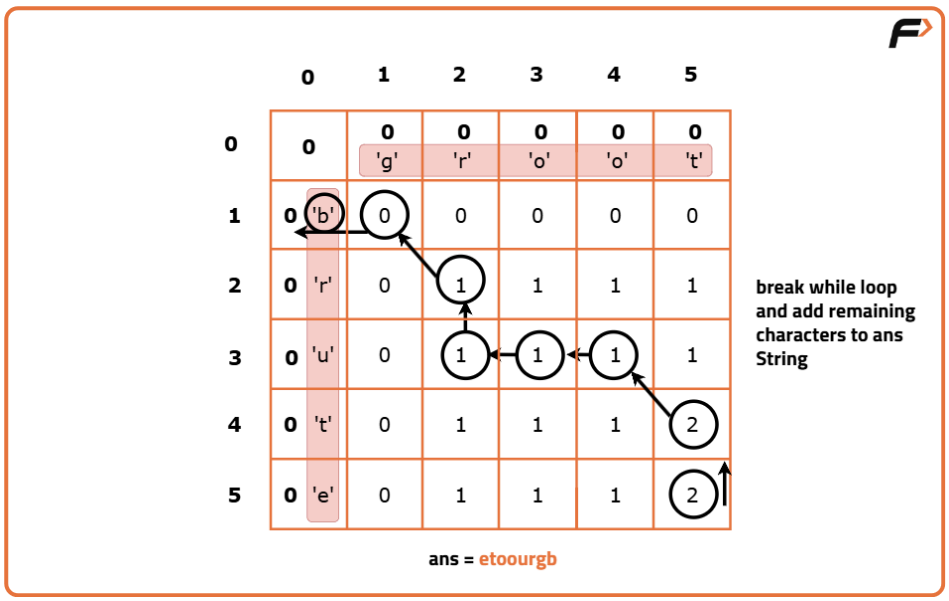
Key ideas:

- Characters of the LCS should appear **only once** in the final string.
- All non-LCS characters from both strings should be placed around the LCS while maintaining order.
- Length of Shortest Common Supersequence = `n + m - k`
  where `n = len(S1)`, `m = len(S2)`, `k = length of LCS`.

Steps to build the string:

1. Build the DP table for LCS of S1 and S2.
2. Start from `dp[n][m]` and move backwards.
3. If `S1[i-1] == S2[j-1]`, this character is part of LCS:
    - Add it once to the answer.
    - Move diagonally (`i--, j--`).

4. If characters do not match:
    - Move towards the larger of `dp[i-1][j]` or `dp[i][j-1]`.
    - Add the corresponding character to the answer.
5. After one string is exhausted, add remaining characters of the other string.
6. Reverse the built string to get the final answer.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    string shortestSupersequence(string s1, string s2) {
        int n = s1.size();
        int m = s2.size();

        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

        // Build LCS table
        for (int i = 1; i <= n; i++) {
```

```
    for (int j = 1; j <= m; j++) {
        if (s1[i - 1] == s2[j - 1])
            dp[i][j] = 1 + dp[i - 1][j - 1];
        else
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    }
}

// Backtrack to build SCS
int i = n, j = m;
string ans = "";

while (i > 0 && j > 0) {
    if (s1[i - 1] == s2[j - 1]) {
        ans += s1[i - 1];
        i--;
        j--;
    } else if (dp[i - 1][j] > dp[i][j - 1]) {
        ans += s1[i - 1];
        i--;
    } else {
        ans += s2[j - 1];
        j--;
    }
}

// Add remaining characters
while (i > 0) {
    ans += s1[i - 1];
    i--;
}
while (j > 0) {
    ans += s2[j - 1];
    j--;
}

reverse(ans.begin(), ans.end());
return ans;
```

```
    }
};

int main() {
    Solution obj;
    string s1 = "mno";
    string s2 = "nop";
    cout << obj.shortestSupersequence(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
We build the LCS DP table in `O(N × M)` time and backtracking takes `O(N + M)`.

Space Complexity: **O(N × M)**
A 2D DP table of size `(n+1) × (m+1)` is used.

Distinct Subsequences (DP - 32)

Given two strings `s` and `t`, we need to count how many **distinct subsequences** of `s` are equal to `t`.
A subsequence is formed by deleting zero or more characters from `s` without changing the order of remaining characters.

Example 1:
s = "axbxax", t = "axa"
There are 2 distinct ways to form `"axa"` from `s`.

Example 2:
s = "babgbag", t = "bag"
There are 5 distinct subsequences of s that equal `"bag"`.

The task is to count all possible valid ways.

Approach 1: Memoization

Algorithm

We use recursion with memoization.

Define a recursive function `f(i, j)`:

- `i` → current index in `s`
- `j` → current index in `t`
- It returns the number of ways to form `t[j:]` from `s[i:]`

Base cases:

- If `j == t.length()`, we have successfully formed `t`, so return 1.
- If `i == s.length()` and `j < t.length()`, `s` is exhausted, so return 0.

Recursive cases:

- If `s[i] == t[j]`, we have two choices:
    1. Take this character → `f(i+1, j+1)`
    2. Skip this character → `f(i+1, j)`
       Add both results.
- If `s[i] != t[j]`, we can only skip `s[i]` → `f(i+1, j)`.

To avoid recomputation, store results in a 2D DP array `dp[i][j]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int helper(int i, int j, string &s, string &t, vector<vector<int>>
&dp) {
        if (j == t.size()) return 1;
        if (i == s.size()) return 0;
        if (dp[i][j] != -1) return dp[i][j];

        if (s[i] == t[j]) {
            int take = helper(i + 1, j + 1, s, t, dp);
            int notTake = helper(i + 1, j, s, t, dp);
            return dp[i][j] = take + notTake;
        } else {
            return dp[i][j] = helper(i + 1, j, s, t, dp);
        }
    }

    int numDistinct(string s, string t) {
        vector<vector<int>> dp(s.size(), vector<int>(t.size(), -1));
        return helper(0, 0, s, t, dp);
    }
};

int main() {
    Solution sol;
    string s = "babgbag";
    string t = "bag";
    cout << sol.numDistinct(s, t);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Each `(i, j)` state is computed once.

Space Complexity: **O(N × M) + O(N)**
DP table plus recursion stack space.

Approach 2: Tabulation

Algorithm

We convert the memoized solution into bottom-up DP.

Define:

- `dp[i][j]` = number of ways to form `t[j:]` from `s[i:]`

Initialization:

- `dp[i][m] = 1` for all `i` because empty `t` can always be formed.
- `dp[n][j] = 0` for `j < m` because non-empty `t` cannot be formed from empty `s`.

Transition:

- If `s[i] == t[j]`:
  `dp[i][j] = dp[i+1][j+1] + dp[i+1][j]`
- Else:
  `dp[i][j] = dp[i+1][j]`

Answer is `dp[0][0]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    int numDistinct(string s, string t) {
        int n = s.size();
        int m = t.size();

        vector<vector<long long>> dp(n + 1, vector<long long>(m + 1,
0));

        for (int i = 0; i <= n; i++) {
            dp[i][m] = 1;
        }

        for (int i = n - 1; i >= 0; i--) {
            for (int j = m - 1; j >= 0; j--) {
                if (s[i] == t[j])
                    dp[i][j] = dp[i + 1][j + 1] + dp[i + 1][j];
                else
                    dp[i][j] = dp[i + 1][j];
            }
        }
        return dp[0][0];
    }
};

int main() {
    Solution sol;
    string s = "babgbag";
    string t = "bag";
    cout << sol.numDistinct(s, t);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Each DP state is filled once.

Space Complexity: **O(N × M)**
A full 2D DP table is used.

Approach 3: Space Optimization

Algorithm

In tabulation, each row depends only on the next row.
So we use a single 1D DP array of size `m+1`.

Steps:

1. Initialize `dp[m] = 1` (empty `t` case).
2. Traverse `s` from end to start.
3. For each character of `s`, update `dp` from right to left:
    - If `s[i] == t[j]`:
      `dp[j] = dp[j] + dp[j+1]`
    - Else:
      `dp[j]` remains unchanged.
4. Final answer is `dp[0]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numDistinct(string s, string t) {
        int n = s.size();
        int m = t.size();

        vector<long long> dp(m + 1, 0);
        dp[m] = 1;

        for (int i = n - 1; i >= 0; i--) {
```

```
            for (int j = m - 1; j >= 0; j--) {
                if (s[i] == t[j]) {
                    dp[j] = dp[j] + dp[j + 1];
                }
            }
        }
        return (int)dp[0];
    }
};

int main() {
    Solution sol;
    string s = "rabbbit";
    string t = "rabbit";
    cout << sol.numDistinct(s, t);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
All character pairs are processed once.

Space Complexity: **O(M)**
Only one 1D DP array of size `m+1` is used.

Edit Distance (DP - 33)

We are given two strings S1 and S2.
We need to convert S1 into S2 using the **minimum number of operations**.

Allowed operations:

1. Delete a character
2. Insert a character
3. Replace a character

We must return the minimum operations required.

Example 1:
start = "planet", target = "plan"
We delete `'e'` and `'t'`.
Answer = 2

Example 2:
start = "abcdefg", target = "azced"
Operations:

- replace b `→` z
- delete d
- delete f
- replace g `→` d
  Answer = 4

Approach 1: Brute Force + Memoization

Algorithm

We solve the problem using recursion with memoization.

Define a function `f(i, j)`:

- `i` → index in S1
- `j` → index in S2
- It returns minimum operations to convert S1`[0..i]` to S2`[0..j]`

Base cases:

- If `i < 0`, all remaining characters of S2 must be inserted → return `j + 1`
- If `j < 0`, all remaining characters of S1 must be deleted → return `i + 1`

Recursive cases:

- If S1`[i] == S2[j]`, no operation needed
  `f(i, j) = f(i-1, j-1)`
- Else, we try all three operations:
  - Replace → `1 + f(i-1, j-1)`
  - Delete → `1 + f(i-1, j)`

- Insert → `1 + f(i, j-1)`
  Take the minimum of these three.

Since there are overlapping subproblems, we store results in a DP table.

Code

```
#include <bits/stdc++.h>
using namespace std;

int editDistanceUtil(string& s1, string& s2, int i, int j, vector<vector<int>>& dp) {
    if (i < 0) return j + 1;
    if (j < 0) return i + 1;

    if (dp[i][j] != -1) return dp[i][j];

    if (s1[i] == s2[j]) {
        return dp[i][j] = editDistanceUtil(s1, s2, i - 1, j - 1, dp);
    }

    int replaceOp = 1 + editDistanceUtil(s1, s2, i - 1, j - 1, dp);
    int deleteOp  = 1 + editDistanceUtil(s1, s2, i - 1, j, dp);
    int insertOp  = 1 + editDistanceUtil(s1, s2, i, j - 1, dp);

    return dp[i][j] = min(replaceOp, min(deleteOp, insertOp));
}

int editDistance(string& s1, string& s2) {
    int n = s1.size();
    int m = s2.size();
    vector<vector<int>> dp(n, vector<int>(m, -1));
    return editDistanceUtil(s1, s2, n - 1, m - 1, dp);
}

int main() {
    string s1 = "horse";
    string s2 = "ros";
    cout << editDistance(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Each state `(i, j)` is computed once.

Space Complexity: **O(N × M) + O(N + M)**
DP table plus recursion stack space.

Approach 2: Tabulation (Bottom-Up)

Algorithm

In memoization, we used negative indices, which are not allowed in arrays.
So we shift indices by 1.

Define:

- `dp[i][j]` = minimum operations to convert `S1[0..i-1]` to `S2[0..j-1]`

Base cases:

- `dp[i][0] = i` → delete all characters from S1
- `dp[0][j] = j` → insert all characters into S1

Transitions:

- If `S1[i-1] == S2[j-1]`
  `dp[i][j] = dp[i-1][j-1]`
- Else
  `dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])`

Answer is `dp[n][m]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

int editDistance(string& s1, string& s2) {
    int n = s1.size();
    int m = s2.size();

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
```

```
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + min(dp[i - 1][j - 1],
                                   min(dp[i - 1][j], dp[i][j - 1]));
            }
        }
    }
    return dp[n][m];
}

int main() {
    string s1 = "horse";
    string s2 = "ros";
    cout << editDistance(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Two nested loops over both strings.

Space Complexity: **O(N × M)**
2D DP table is used.

Approach 3: Space Optimized

Algorithm

From the DP relation:
`dp[i][j]` depends only on:

- `dp[i-1][j-1]`
- `dp[i-1][j]`

- `dp[i][j-1]`

So we only need the **previous row**.

Steps:

1. Use two arrays: `prev` and `cur`
2. Initialize `prev[j] = j`
3. For each row `i`:
    - set `cur[0] = i`
    - compute remaining columns
4. After each row, assign `prev = cur`
5. Final answer is `prev[m]`

Code

```
#include <bits/stdc++.h>
using namespace std;

int editDistance(string& s1, string& s2) {
    int n = s1.size();
    int m = s2.size();

    vector<int> prev(m + 1, 0), cur(m + 1, 0);

    for (int j = 0; j <= m; j++) prev[j] = j;

    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                cur[j] = prev[j - 1];
            } else {
                cur[j] = 1 + min(prev[j - 1],
                                 min(prev[j], cur[j - 1]));
            }
        }
        prev = cur;
    }
    return prev[m];
}

int main() {
    string s1 = "horse";
```

```
    string s2 = "ros";
    cout << editDistance(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Same transitions as tabulation.

Space Complexity: **O(M)**
Only two rows of size `M+1` are stored.

Wildcard Matching (DP - 34)

We are given two strings S1 and S2.
S1 is a **pattern string** that can contain special characters:

- `'?'` matches **exactly one** character of S2
- `'*'` matches **any sequence** of characters of S2 (including empty)

We need to check whether the pattern S1 **matches completely** with string S2.

Example 1:
S1 = `"ab*cd"`, S2 = `"abdefcd"`
`*` matches `"def"`, rest characters match → **true**

Example 2:
S1 = `"*a*b"`, S2 = `"aaab"`
First `*` matches `"aa"`, second `*` matches empty → **true**

Approach 1: Memoization (Top-Down DP)

Algorithm

We solve this using recursion with memoization.

Define a function `f(i, j)`:

- `i` → index in pattern S1
- `j` → index in text S2
- It returns whether `S1[0..i]` matches `S2[0..j]`

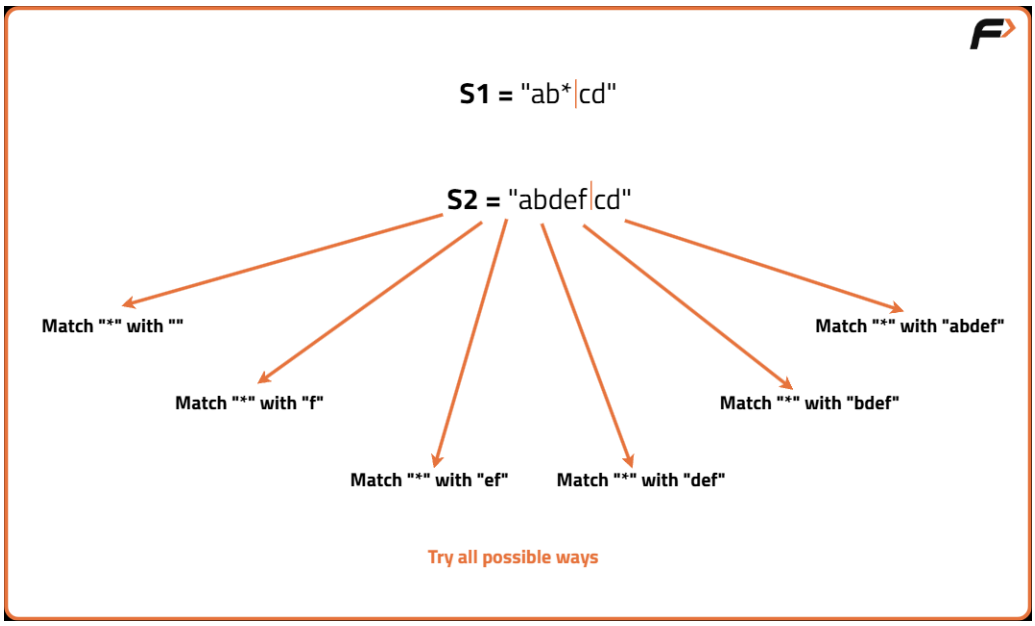
Base cases:

- If `i < 0` and `j < 0` → both strings finished → true
- If `i < 0` and `j >= 0` → pattern finished but text left → false
- If `j < 0` and `i >= 0` → text finished, pattern must be all `'*'`

Recursive cases:

- If `S1[i] == S2[j]` or `S1[i] == '?'`
  → move both pointers: `f(i-1, j-1)`
- If `S1[i] == '*'`
  - match empty sequence → `f(i-1, j)`
  - match one or more characters → `f(i, j-1)`
- Else → characters do not match → false

We store results in `dp[i][j]` to avoid recomputation.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isAllStars(string &s, int i) {
        for (int k = 0; k <= i; k++) {
            if (s[k] != '*') return false;
        }
        return true;
    }

    bool solve(int i, int j, string &s1, string &s2, vector<vector<int>> &dp) {
        if (i < 0 && j < 0) return true;
        if (i < 0 && j >= 0) return false;
        if (j < 0 && i >= 0) return isAllStars(s1, i);

        if (dp[i][j] != -1) return dp[i][j];

        if (s1[i] == s2[j] || s1[i] == '?')
            return dp[i][j] = solve(i - 1, j - 1, s1, s2, dp);

        if (s1[i] == '*')
            return dp[i][j] = solve(i - 1, j, s1, s2, dp) ||
                              solve(i, j - 1, s1, s2, dp);

        return dp[i][j] = false;
    }

    bool wildcardMatching(string &s1, string &s2) {
        int n = s1.size(), m = s2.size();
        vector<vector<int>> dp(n, vector<int>(m, -1));
        return solve(n - 1, m - 1, s1, s2, dp);
    }
};

int main() {
    string s1 = "ab*cd";
    string s2 = "abdefcd";
    Solution sol;
    cout << sol.wildcardMatching(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Each `(i, j)` state is computed once.

Space Complexity: **O(N × M) + O(N + M)**
DP table plus recursion stack.

Approach 2: Tabulation (Bottom-Up DP)

Algorithm

To remove recursion, we shift indices by 1.

Define:
`dp[i][j]` = true if `S1[0..i-1]` matches `S2[0..j-1]`

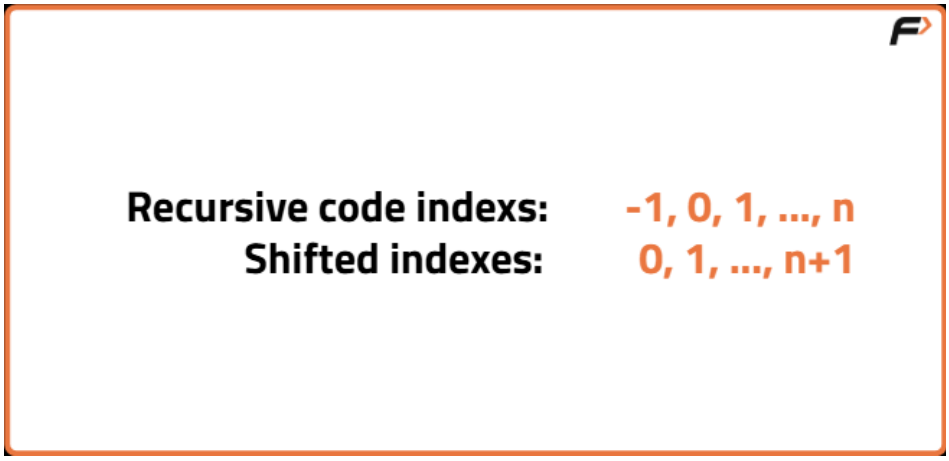
Base cases:

- `dp[0][0] = true`
- `dp[0][j] = false` (empty pattern cannot match non-empty text)
- `dp[i][0] = true` only if first `i` characters of pattern are all `'*'`

Transitions:

- If `S1[i-1] == S2[j-1]` or `S1[i-1] == '?'`
  `dp[i][j] = dp[i-1][j-1]`
- If `S1[i-1] == '*'`
  `dp[i][j] = dp[i-1][j] || dp[i][j-1]`
- Else
  `dp[i][j] = false`

Final answer is `dp[n][m]`.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isAllStars(string &s, int i) {
        for (int k = 1; k <= i; k++) {
            if (s[k - 1] != '*') return false;
        }
        return true;
    }

    bool wildcardMatching(string &s1, string &s2) {
        int n = s1.size(), m = s2.size();
        vector<vector<bool>> dp(n + 1, vector<bool>(m + 1, false));

        dp[0][0] = true;

        for (int i = 1; i <= n; i++)
            dp[i][0] = isAllStars(s1, i);

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (s1[i - 1] == s2[j - 1] || s1[i - 1] == '?')
                    dp[i][j] = dp[i - 1][j - 1];
                else if (s1[i - 1] == '*')
                    dp[i][j] = dp[i - 1][j] || dp[i][j - 1];
                else
                    dp[i][j] = false;
            }
```

```
        }
        return dp[n][m];
    }
};

int main() {
    string s1 = "ab*cd";
    string s2 = "abdefcd";
    Solution sol;
    cout << sol.wildcardMatching(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Two nested loops over pattern and text.

Space Complexity: **O(N × M)**
2D DP table.

Approach 3: Space Optimization

Algorithm

From the DP relation:
`dp[i][j]` depends on:

- `dp[i-1][j-1]`
- `dp[i-1][j]`
- `dp[i][j-1]`

So we only need:

- previous row (`prev`)
- current row (`cur`)

Steps:

1. Initialize `prev[0] = true`
2. For each pattern index `i`:

        - `cur[0] = isAllStars(S1, i)`
        - fill remaining columns
    3. After each row, do `prev = cur`
    4. Answer is `prev[m]`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isAllStars(string &s, int i) {
        for (int k = 1; k <= i; k++) {
            if (s[k - 1] != '*') return false;
        }
        return true;
    }

    bool wildcardMatching(string &s1, string &s2) {
        int n = s1.size(), m = s2.size();
        vector<bool> prev(m + 1, false), cur(m + 1, false);

        prev[0] = true;

        for (int i = 1; i <= n; i++) {
            cur[0] = isAllStars(s1, i);
            for (int j = 1; j <= m; j++) {
                if (s1[i - 1] == s2[j - 1] || s1[i - 1] == '?')
                    cur[j] = prev[j - 1];
                else if (s1[i - 1] == '*')
                    cur[j] = prev[j] || cur[j - 1];
                else
                    cur[j] = false;
            }
            prev = cur;
        }
        return prev[m];
    }
};

int main() {
    string s1 = "ab*cd";
    string s2 = "abdefcd";
```

```
    Solution sol;
    cout << sol.wildcardMatching(s1, s2);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N × M)**
Same transitions as tabulation.

Space Complexity: **O(M)**
Only two rows of DP are stored.

Stock Buy and Sell (DP-35)

We are given an array `Arr[]` of length n where `Arr[i]` represents the stock price on day `i`.
We are allowed to **buy and sell the stock only once**.
The selling day must be **the same day or after the buying day**.
Our task is to find the **maximum profit** possible.

If no profit is possible, return `0`.

Example 1:
`prices = [7,1,5,3,6,4]`
Buy at price 1 (day 2) and sell at price 6 (day 5).
Profit = `6 - 1 = 5`.

Example 2:
`prices = [7,6,4,3,1]`
Prices keep decreasing, so no transaction is done.
Profit = `0`.

Approach 1: Brute Force

Algorithm

We try **all possible buy and sell day pairs** and calculate profit for each valid pair.

Steps:

- Initialize `maxProfit = 0`
- Loop day `i` from `0` to `n-2` as buying day
- Loop day `j` from `i+1` to `n-1` as selling day
- Calculate `profit = prices[j] - prices[i]`
- Update `maxProfit` if profit is greater
- Return `maxProfit`

This checks every possible transaction.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int maxProfit = 0;

        for (int i = 0; i < prices.size(); i++) {
            for (int j = i + 1; j < prices.size(); j++) {
                int profit = prices[j] - prices[i];
                maxProfit = max(maxProfit, profit);
            }
        }
        return maxProfit;
    }
```

```
};

int main() {
    Solution obj;
    vector<int> prices = {7, 1, 5, 3, 6, 4};
    cout << obj.maxProfit(prices) << endl;
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$
  Two nested loops check all buy-sell pairs.

- **Space Complexity:** $O(1)$
  Only variables are used, no extra data structures.

Approach 2: Optimal Approach

Algorithm

Instead of checking all pairs, we track the **minimum price seen so far**.

Idea:

- If we sell on day `i`, the best buy day is the day with the **minimum price before day i**
- Keep updating the minimum price
- At each day, calculate profit if sold today
- Track the maximum profit

Steps:

- Initialize `minPrice = INT_MAX`

- Initialize `maxProfit = 0`
- Traverse prices from left to right
- Update `minPrice` if current price is smaller
- Calculate `profit = prices[i] - minPrice`
- Update `maxProfit`
- Return `maxProfit`

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minPrice = INT_MAX;
        int maxProfit = 0;

        for (int i = 0; i < prices.size(); i++) {
            if (prices[i] < minPrice)
                minPrice = prices[i];

            int profit = prices[i] - minPrice;
            if (profit > maxProfit)
                maxProfit = profit;
        }
        return maxProfit;
    }
};

int main() {
    Solution obj;
    vector<int> prices = {7, 1, 5, 3, 6, 4};
    cout << "Maximum Profit: " << obj.maxProfit(prices) << endl;
```

```
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(n)`
  Single pass through the array.

- **Space Complexity:** `O(1)`
  Only constant extra variables are used.

Buy and Sell Stock – III (DP-37)

We are given an array `Arr[]` of length `n` where `Arr[i]` represents the stock price on day `i`.

Rules:

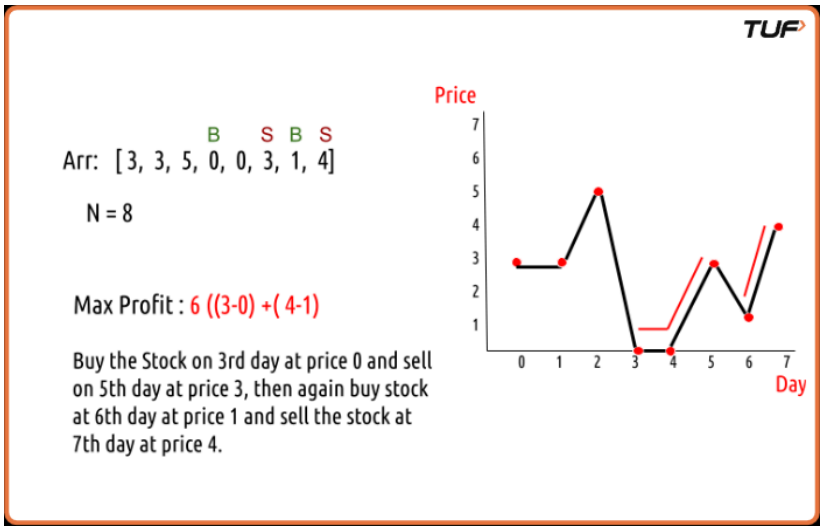
- We can buy and sell the stock multiple times.
- To sell, we must have bought the stock before.
- We cannot hold more than one stock at a time.
- We can do **at most 2 transactions** (one transaction = one buy + one sell).

We need to find the **maximum profit**.



Example:
`Arr = {3,3,5,0,0,3,1,4}`
Maximum profit = `(3 - 0) + (4 - 1) = 6`

Explanation:
Buy at price `0`, sell at `3`, then buy at `1`, sell at `4`.



Approach 1: Memoization Approach

We solve this using recursion with memoization.

On each day, we decide:

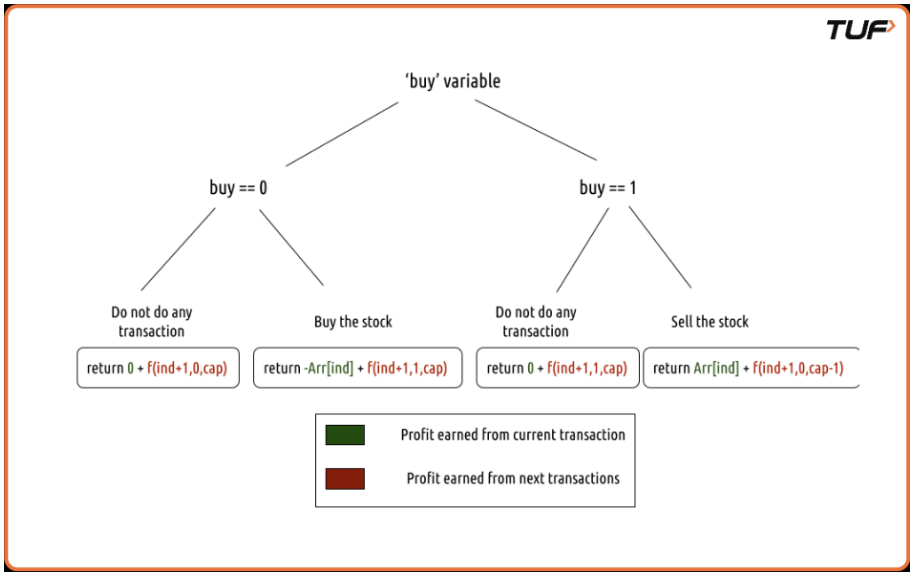
- Whether we can **buy** or must **sell**
- How many transactions are still left

State is defined by:

- `ind` → current day
- `buy` → 0 means we can buy, 1 means we can sell
- `cap` → remaining transactions

Algorithm

- If we reach the end of days or `cap == 0`, profit is `0`
- If `buy == 0`:
    - Skip the day
    - Buy the stock
- If `buy == 1`:
    - Skip the day
    - Sell the stock and reduce transaction count
- Take the maximum of choices
- Store results in a 3D DP array to avoid recomputation



Code

```
#include <bits/stdc++.h>
using namespace std;

class StockProfit {
public:
    int getAns(vector<int>& Arr, int n, int ind, int buy, int cap,
               vector<vector<vector<int>>>& dp) {

        if (ind == n || cap == 0)
            return 0;

        if (dp[ind][buy][cap] != -1)
            return dp[ind][buy][cap];

        int profit;
        if (buy == 0) {
            profit = max(
                getAns(Arr, n, ind + 1, 0, cap, dp),
                -Arr[ind] + getAns(Arr, n, ind + 1, 1, cap, dp)
            );
        } else {
            profit = max(
                getAns(Arr, n, ind + 1, 1, cap, dp),
```

```
                Arr[ind] + getAns(Arr, n, ind + 1, 0, cap - 1, dp)
            );
        }
        return dp[ind][buy][cap] = profit;
    }

    int maxProfit(vector<int>& prices, int n) {
        vector<vector<vector<int>>> dp(
            n, vector<vector<int>>(2, vector<int>(3, -1))
        );
        return getAns(prices, n, 0, 0, 2, dp);
    }
};

int main() {
    vector<int> prices = {3,3,5,0,0,3,1,4};
    StockProfit obj;
    cout << "The maximum profit that can be generated is "
         << obj.maxProfit(prices, prices.size());
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N * 2 * 3)`
  Each state `(day, buy, cap)` is computed once.

- **Space Complexity:** `O(N * 2 * 3) + O(N)`
  DP table plus recursion stack.

Approach 2: Tabulation Approach

We convert the memoization into a bottom-up DP.

- Create a DP table `dp[ind][buy][cap]`

- Base cases are already `0` when:
    - `ind == n`
    - `cap == 0`
- Fill table from last day to first day
- For each state, apply the same buy/sell logic
- Final answer is `dp[0][0][2]`

Code

```
#include <bits/stdc++.h>
using namespace std;

class StockProfit {
public:
    int maxProfit(vector<int>& Arr, int n) {
        vector<vector<vector<int>>> dp(
            n + 1, vector<vector<int>>(2, vector<int>(3, 0))
        );

        for (int ind = n - 1; ind >= 0; ind--) {
            for (int buy = 0; buy <= 1; buy++) {
                for (int cap = 1; cap <= 2; cap++) {
                    if (buy == 0) {
                        dp[ind][buy][cap] = max(
                            dp[ind + 1][0][cap],
                            -Arr[ind] + dp[ind + 1][1][cap]
                        );
                    } else {
                        dp[ind][buy][cap] = max(
                            dp[ind + 1][1][cap],
                            Arr[ind] + dp[ind + 1][0][cap - 1]
                        );
                    }
                }
```

```
            }
        }
        return dp[0][0][2];
    }
};

int main() {
    vector<int> prices = {3,3,5,0,0,3,1,4};
    StockProfit obj;
    cout << "The maximum profit that can be generated is "
         << obj.maxProfit(prices, prices.size());
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N * 2 * 3)`
- **Space Complexity:** `O(N * 2 * 3)`

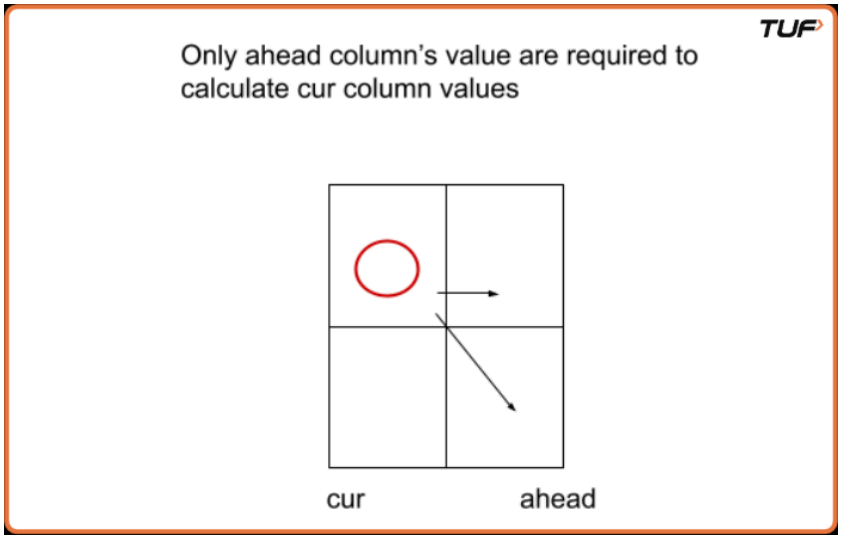
Approach 3: Space Optimization Approach

We observe that only the **next day** values are needed.

Algorithm

- Use two 2D arrays:
    - `ahead` → represents `dp[ind+1]`
    - `cur` → represents `dp[ind]`
- Update `cur` using `ahead`
- After each day, copy `cur` to `ahead`

- Final answer is `ahead[0][2]`



Code

```
#include <bits/stdc++.h>
using namespace std;

class StockProfit {
public:
    int maxProfit(vector<int>& Arr, int n) {
        vector<vector<int>> ahead(2, vector<int>(3, 0));
        vector<vector<int>> cur(2, vector<int>(3, 0));

        for (int ind = n - 1; ind >= 0; ind--) {
            for (int buy = 0; buy <= 1; buy++) {
                for (int cap = 1; cap <= 2; cap++) {
                    if (buy == 0) {
                        cur[buy][cap] = max(
                            ahead[0][cap],
                            -Arr[ind] + ahead[1][cap]
                        );
                    } else {
                        cur[buy][cap] = max(
                            ahead[1][cap],
                            Arr[ind] + ahead[0][cap - 1]
                        );
```

```
                    }
                }
            }
            ahead = cur;
        }
        return ahead[0][2];
    }
};

int main() {
    vector<int> prices = {3,3,5,0,0,3,1,4};
    StockProfit obj;
    cout << "The maximum profit that can be generated is "
         << obj.maxProfit(prices, prices.size());
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(N * 2 * 3)$
- **Space Complexity:** $O(1)$
  Only two 2×3 arrays are used.

Buy and Sell Stock – IV (DP-38)

We are given stock prices for n days and allowed to perform **at most K transactions**.

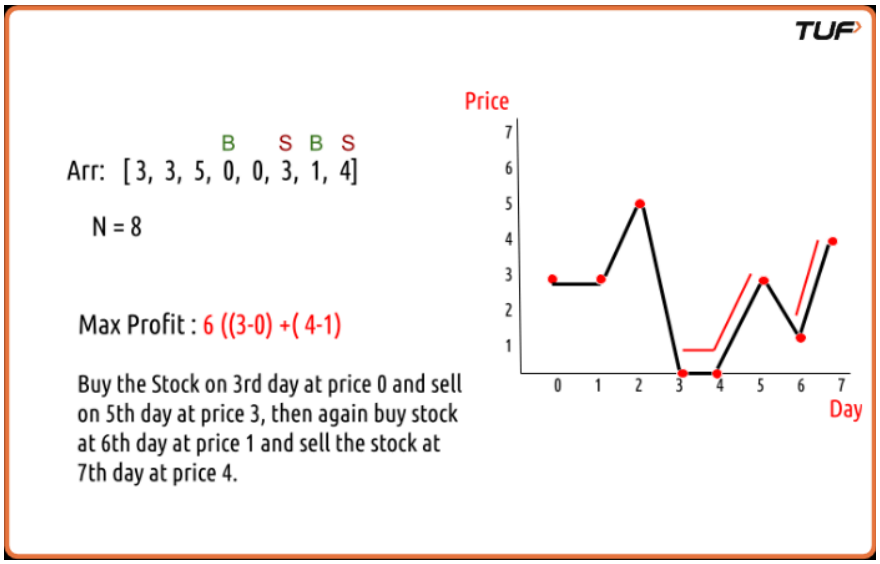
Rules recap

- One transaction = **buy + sell**

- You must sell before buying again
- You cannot sell before buying
- Goal: **maximize total profit**



This is a **generalization of Stock-III** (where K = 2).



At any day, your decision depends on **three things**:

1. `day (ind)`
2. buy → are you allowed to buy (`0`) or must sell (1)
3. `cap` → how many transactions are still left

So the DP state is:

`dp[ind][buy][cap]`

Meaning:

> Maximum profit from day `ind` onward,
> given whether we can buy or sell,
> and `cap` transactions remaining.

Transitions

If `buy == 0` (we can buy)

- Skip the day → `dp[ind+1][0][cap]`
- Buy stock → `-price[ind] + dp[ind+1][1][cap]`

```
dp[ind][0][cap] = max(
    dp[ind+1][0][cap],
    -Arr[ind] + dp[ind+1][1][cap]
)
```

If `buy == 1` (we can sell)

- Skip the day → `dp[ind+1][1][cap]`
- Sell stock → `+price[ind] + dp[ind+1][0][cap-1]`

```
dp[ind][1][cap] = max(
    dp[ind+1][1][cap],
    Arr[ind] + dp[ind+1][0][cap-1]
)
```

Base Cases

- If `ind == n` → no days left → profit `0`
- If `cap == 0` → no transactions left → profit `0`

1️⃣ Memoization (Top-Down)

Code

```
#include <bits/stdc++.h>
using namespace std;

class StockBuySell {
public:
    int solve(int ind, int buy, int cap,
              vector<int>& arr,
              vector<vector<vector<int>>>& dp) {

        if (ind == arr.size() || cap == 0)
            return 0;

        if (dp[ind][buy][cap] != -1)
            return dp[ind][buy][cap];

        int profit;
        if (buy == 0) {
            profit = max(
                solve(ind + 1, 0, cap, arr, dp),
                -arr[ind] + solve(ind + 1, 1, cap, arr, dp)
            );
        } else {
            profit = max(
                solve(ind + 1, 1, cap, arr, dp),
```

```
                arr[ind] + solve(ind + 1, 0, cap - 1, arr, dp)
            );
        }

        return dp[ind][buy][cap] = profit;
    }

    int maximumProfit(vector<int>& prices, int k) {
        int n = prices.size();
        vector<vector<vector<int>>> dp(
            n, vector<vector<int>>(2, vector<int>(k + 1, -1))
        );
        return solve(0, 0, k, prices, dp);
    }
};

int main() {
    vector<int> prices = {3,3,5,0,0,3,1,4};
    int k = 2;

    StockBuySell obj;
    cout << obj.maximumProfit(prices, k);
    return 0;
}
```

Complexity

- **Time:** $O(N \times 2 \times K)$
- **Space:** $O(N \times 2 \times K)$ + recursion stack

---

2 Tabulation (Bottom-Up)

Idea

Convert recursion into loops:

- Day from $n-1 \rightarrow 0$
- buy in $\{0, 1\}$
- cap from $1 \rightarrow K$

Code

```
#include <bits/stdc++.h>
using namespace std;

class StockBuySell {
public:
    int maximumProfit(vector<int>& arr, int k) {
        int n = arr.size();

        vector<vector<vector<int>>> dp(
            n + 1, vector<vector<int>>(2, vector<int>(k + 1, 0))
        );

        for (int ind = n - 1; ind >= 0; ind--) {
            for (int buy = 0; buy <= 1; buy++) {
                for (int cap = 1; cap <= k; cap++) {
                    if (buy == 0) {
                        dp[ind][buy][cap] = max(
                            dp[ind + 1][0][cap],
                            -arr[ind] + dp[ind + 1][1][cap]
                        );
                    } else {
                        dp[ind][buy][cap] = max(
                            dp[ind + 1][1][cap],
                            arr[ind] + dp[ind + 1][0][cap - 1]
                        );
                    }
                }
            }
        }
```

```
        return dp[0][0][k];
    }
};

int main() {
    vector<int> prices = {3,3,5,0,0,3,1,4};
    int k = 2;

    StockBuySell obj;
    cout << obj.maximumProfit(prices, k);
    return 0;
}
```

Complexity

- **Time:** `O(N × 2 × K)`
- **Space:** `O(N × 2 × K)`

3️⃣ Space Optimization (Best)

Observation

`dp[ind]` depends only on `dp[ind+1]`.

So we only need **two 2×(K+1) arrays**:

- ahead → day `ind+1`
- `cur` → day `ind`

Code

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;

class StockBuySell {
public:
    int maximumProfit(vector<int>& arr, int k) {
        int n = arr.size();

        vector<vector<int>> ahead(2, vector<int>(k + 1, 0));
        vector<vector<int>> cur(2, vector<int>(k + 1, 0));

        for (int ind = n - 1; ind >= 0; ind--) {
            for (int buy = 0; buy <= 1; buy++) {
                for (int cap = 1; cap <= k; cap++) {
                    if (buy == 0) {
                        cur[buy][cap] = max(
                            ahead[0][cap],
                            -arr[ind] + ahead[1][cap]
                        );
                    } else {
                        cur[buy][cap] = max(
                            ahead[1][cap],
                            arr[ind] + ahead[0][cap - 1]
                        );
                    }
                }
            }
            ahead = cur;
        }
        return ahead[0][k];
    }
};

int main() {
    vector<int> prices = {3,3,5,0,0,3,1,4};
    int k = 2;

    StockBuySell obj;
    cout << obj.maximumProfit(prices, k);
```

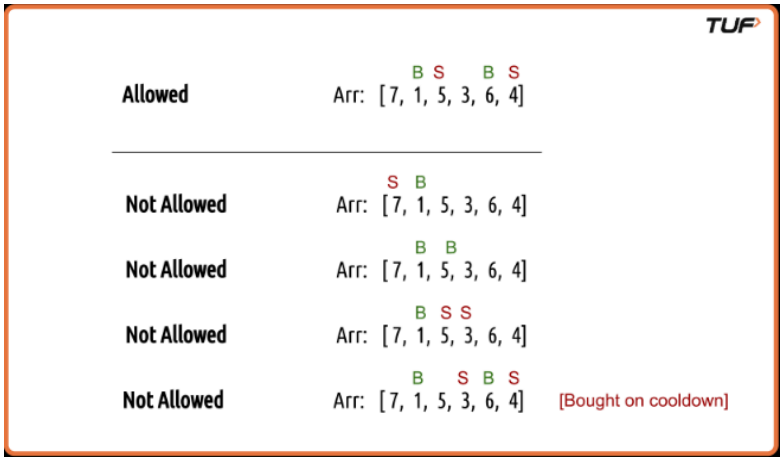
```
    return 0;
}
```

Complexity

- **Time:** $O(N \times 2 \times K)$
- **Space:** $O(2 \times K) \rightarrow$ **O(K)**

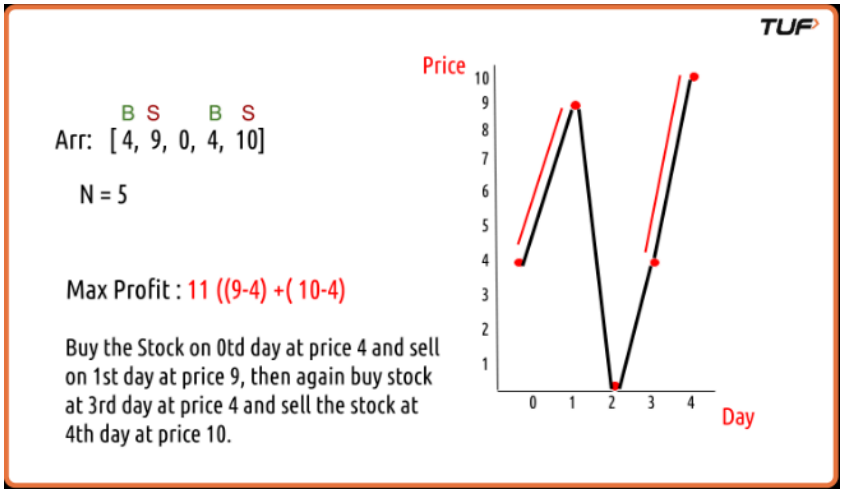
Buy and Sell Stocks With Cooldown | (DP - 39)

We are given an array Arr[] where Arr[i] represents the stock price on day i.
We can buy and sell the stock multiple times but with some rules.
We must buy before selling. After selling, we cannot buy on the very next day because of the cooldown rule.
Our task is to find the maximum profit that can be earned.



Example:
Arr = [4,9,0,4,10]
If we buy on day 0 (price 4) and sell on day 1 (price 9), profit = 5.

Due to cooldown, we cannot buy on day 2.
Then we buy on day 3 (price 4) and sell on day 4 (price 10), profit = 6.
Total profit = 11.



Approach 1: Memoization

Algorithm

We process the array day by day using recursion.
At each day, we track whether we are allowed to buy or sell.

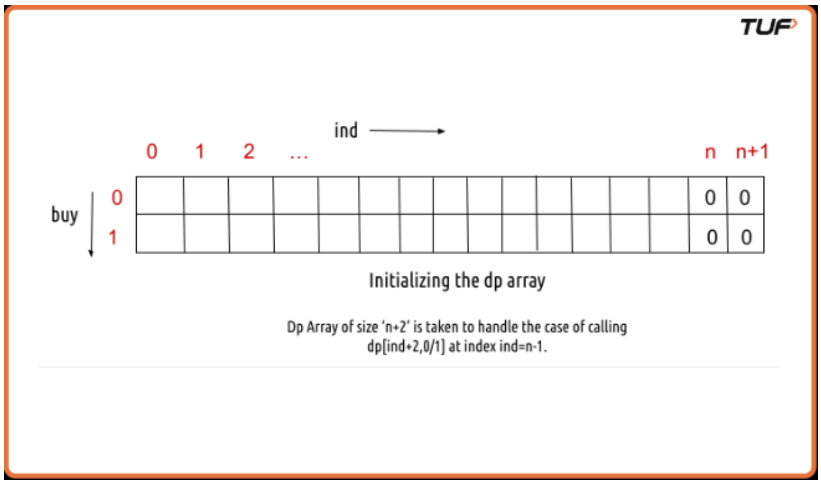
If buying is allowed:

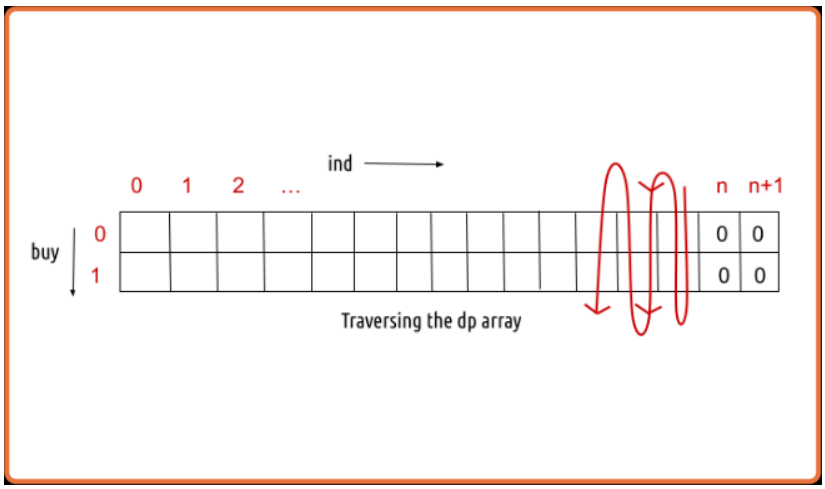
- Option 1: Skip the day.
- Option 2: Buy the stock and move to sell state.

If selling is allowed:

- Option 1: Skip the day.
- Option 2: Sell the stock and move two days ahead because of cooldown.

We use a DP table dp[ind][buy] to store results so the same state is not recalculated.





Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
int getAns(vector<int> Arr,int ind,int buy,int n,vector<vector<int>>
&dp){
    if(ind>=n) return 0;
    if(dp[ind][buy]!=-1) return dp[ind][buy];
    int profit;
    if(buy==0){
        profit=max(getAns(Arr,ind+1,0,n,dp),
                   -Arr[ind]+getAns(Arr,ind+1,1,n,dp));
    }
```

```
    if(buy==1){
        profit=max(getAns(Arr,ind+1,1,n,dp),
                   Arr[ind]+getAns(Arr,ind+2,0,n,dp));
    }
    return dp[ind][buy]=profit;
}

int stockProfit(vector<int> &Arr){
    int n=Arr.size();
    vector<vector<int>> dp(n,vector<int>(2,-1));
    return getAns(Arr,0,0,n,dp);
}
};

int main(){
    vector<int> prices={4,9,0,4,10};
    Solution obj;
    cout<<obj.stockProfit(prices);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*2)
There are N days and 2 states (buy or sell). Each state is computed once.

Space Complexity: O(N2*)* + *O(N)*
*DP table uses N*2 space and recursion stack can go up to N.

Approach 2: Tabulation

Algorithm

We convert recursion into iteration.
We create a DP table dp[ind][buy] where ind goes from last day to first.

Base case: after the last day, profit is 0.
We fill the table backwards using the same transitions as memoization.
The final answer is dp[0][0].

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
int stockProfit(vector<int> &Arr){
    int n=Arr.size();
    vector<vector<int>> dp(n+2,vector<int>(2,0));
    for(int ind=n-1;ind>=0;ind--){
        for(int buy=0;buy<=1;buy++){
            int profit;
            if(buy==0){
                profit=max(dp[ind+1][0],
                           -Arr[ind]+dp[ind+1][1]);
            }
            if(buy==1){
                profit=max(dp[ind+1][1],
                           Arr[ind]+dp[ind+2][0]);
            }
            dp[ind][buy]=profit;
        }
    }
    return dp[0][0];
}
};

int main(){
    vector<int> prices={4,9,0,4,10};
    Solution obj;
    cout<<obj.stockProfit(prices);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*2)
Two nested loops over days and buy/sell states.

Space Complexity: O(N*2)
Only the DP table is used, no recursion stack.

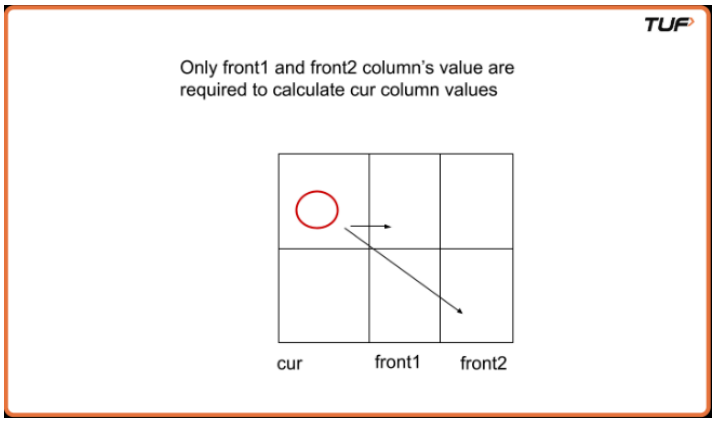
Approach 3: Space Optimization

Algorithm

For each day, we only need results from the next one or two days.
So instead of a full DP table, we keep only three arrays:

- cur for current day
- front1 for next day
- front2 for day after next

We iterate from last day to first and update these arrays.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
```

```
public:
int stockProfit(vector<int> &Arr){
    int n=Arr.size();
    vector<int> cur(2,0),front1(2,0),front2(2,0);
    for(int ind=n-1;ind>=0;ind--){
        for(int buy=0;buy<=1;buy++){
            int profit;
            if(buy==0){
                profit=max(front1[0],
                           -Arr[ind]+front1[1]);
            }
            if(buy==1){
                profit=max(front1[1],
                           Arr[ind]+front2[0]);
            }
            cur[buy]=profit;
        }
        front2=front1;
        front1=cur;
    }
    return cur[0];
}
};

int main(){
    vector<int> prices={4,9,0,4,10};
    Solution obj;
    cout<<obj.stockProfit(prices);
    return 0;
}
```

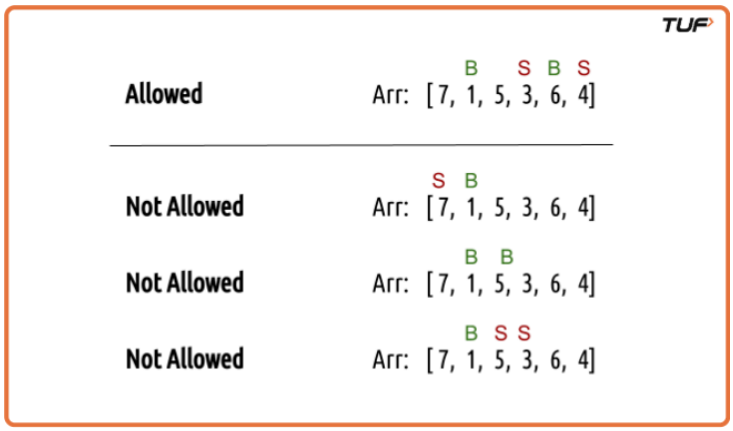
Complexity Analysis

Time Complexity: O(N*2)
We process each day with two states.

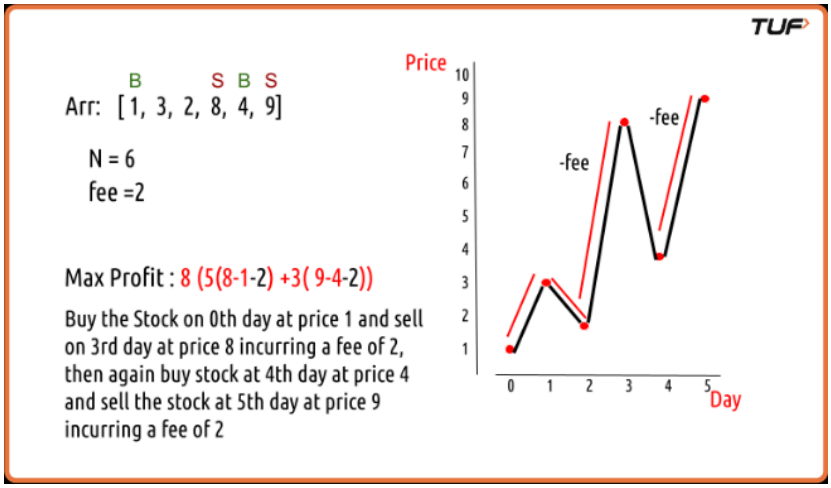
Space Complexity: O(1)
Only three arrays of size 2 are used.

Buy and Sell Stocks With Transaction Fees | (DP - 40)

We are given an array Arr[] where Arr[i] represents the stock price on day i.
We can buy and sell stocks multiple times, but we must follow these rules.
We must buy before selling. After selling, we are allowed to buy again.
Each complete transaction (buy + sell) has a fixed transaction fee that is deducted when we sell.
Our goal is to find the maximum profit possible.



Example:
Arr = [1,3,2,8,4,9], Fee = 2
Buy on day 0 at price 1 and sell on day 3 at price 8.
Profit = 8 - 1 - 2 = 5
Then buy on day 4 at price 4 and sell on day 5 at price 9.
Profit = 9 - 4 - 2 = 3
Total profit = 8



Approach 1: Memoization

Algorithm

We solve the problem using recursion and dynamic programming.

At each day, we decide based on whether we are allowed to buy or need to sell.

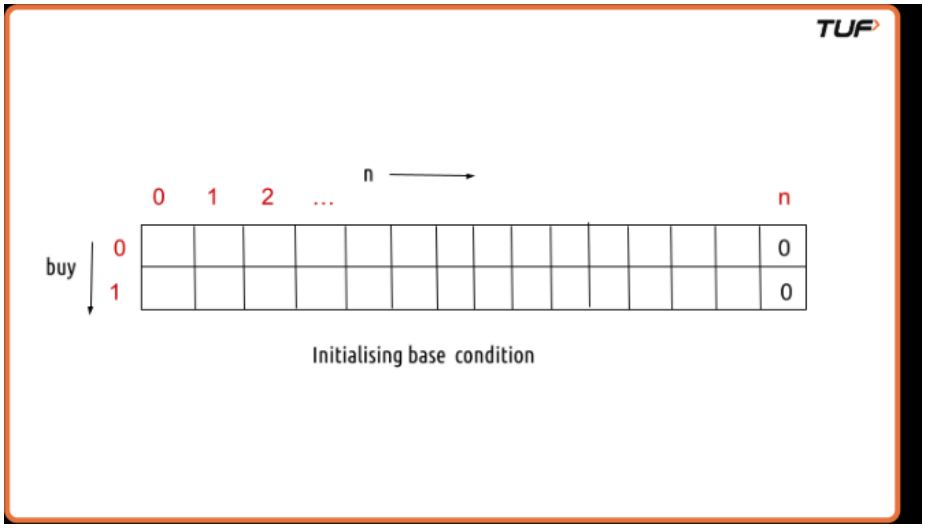
If buy = 0 (allowed to buy):

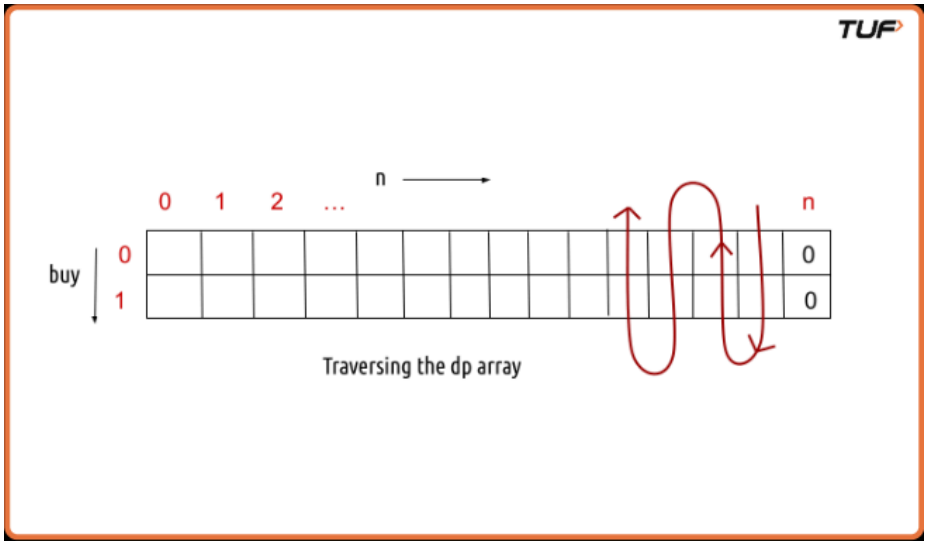
- Skip the day and move to the next day.
- Buy the stock today and move to sell state.

If buy = 1 (allowed to sell):

- Skip the day and continue holding.
- Sell the stock today, subtract the transaction fee, and move to buy state.

We store results in a DP table dp[ind][buy] to avoid recomputation.





Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
int getAns(vector<int> &Arr,int ind,int buy,int n,int
fee,vector<vector<int>> &dp){
    if(ind==n) return 0;
    if(dp[ind][buy]!=-1) return dp[ind][buy];
    int profit;
    if(buy==0){
```

```
        profit=max(getAns(Arr,ind+1,0,n,fee,dp),
                   -Arr[ind]+getAns(Arr,ind+1,1,n,fee,dp));
    }
    if(buy==1){
        profit=max(getAns(Arr,ind+1,1,n,fee,dp),
                   Arr[ind]-fee+getAns(Arr,ind+1,0,n,fee,dp));
    }
    return dp[ind][buy]=profit;
}

int maximumProfit(int n,int fee,vector<int> &Arr){
    vector<vector<int>> dp(n,vector<int>(2,-1));
    return getAns(Arr,0,0,n,fee,dp);
}
};

int main(){
    vector<int> prices={1,3,2,8,4,9};
    int fee=2;
    Solution obj;
    cout<<obj.maximumProfit(prices.size(),fee,prices);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*2)
There are N days and 2 states (buy or sell). Each state is computed once.

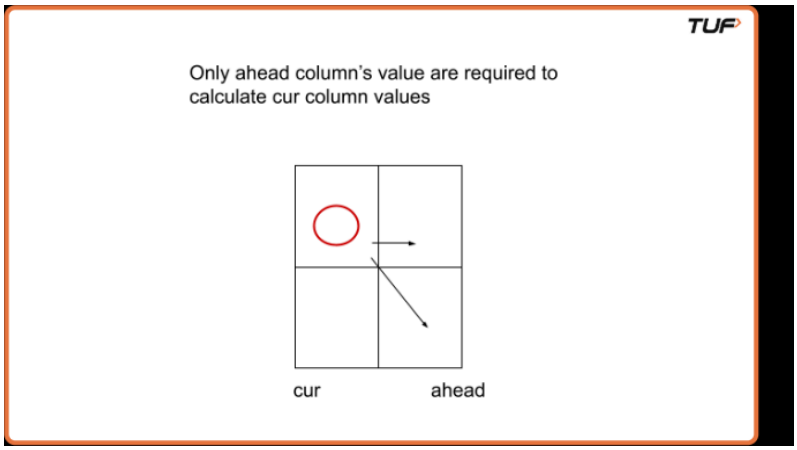
Space Complexity: O(N2*)* + *O(N)*
*DP table uses N*2 space and recursion stack takes up to N.

Approach 2: Tabulation

Algorithm

We convert recursion into an iterative DP approach.

Create a 2D DP table dp[ind][buy].
Base case: when ind = n, profit is 0.
Fill the table from the last day to the first using the same transitions as memoization.
The answer is dp[0][0], meaning starting on day 0 with permission to buy.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
public:
int maximumProfit(int n,int fee,vector<int> &Arr){
    vector<vector<int>> dp(n+1,vector<int>(2,0));
    for(int ind=n-1;ind>=0;ind--){
        for(int buy=0;buy<=1;buy++){
            int profit;
            if(buy==0){
                profit=max(dp[ind+1][0],
                           -Arr[ind]+dp[ind+1][1]);
            }
            if(buy==1){
                profit=max(dp[ind+1][1],
                           Arr[ind]-fee+dp[ind+1][0]);
            }
            dp[ind][buy]=profit;
        }
    }
    return dp[0][0];
```

```
}
};

int main(){
    vector<int> prices={1,3,2,8,4,9};
    int fee=2;
    Solution obj;
    cout<<obj.maximumProfit(prices.size(),fee,prices);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*2)
Two nested loops over days and buy/sell states.

Space Complexity: O(N*2)
Only the DP table is used, no recursion stack.

Approach 3: Space Optimization

Algorithm

For each day, we only need the results from the next day.
So instead of a full DP table, we use two arrays:

- ahead: stores next day results
- cur: stores current day results

We iterate from last day to first and update values accordingly.
The final answer is cur[0].

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
```

```
public:
int maximumProfit(int n,int fee,vector<int> &Arr){
    vector<long> ahead(2,0),cur(2,0);
    for(int ind=n-1;ind>=0;ind--){
        for(int buy=0;buy<=1;buy++){
            long profit;
            if(buy==0){
                profit=max(ahead[0],
                           -Arr[ind]+ahead[1]);
            }
            if(buy==1){
                profit=max(ahead[1],
                           Arr[ind]-fee+ahead[0]);
            }
            cur[buy]=profit;
        }
        ahead=cur;
    }
    return cur[0];
}
};

int main(){
    vector<int> prices={1,3,2,8,4,9};
    int fee=2;
    Solution obj;
    cout<<obj.maximumProfit(prices.size(),fee,prices);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*2)
Each day processes two states.

Space Complexity: O(1)
Only two arrays of size 2 are used.

Longest Increasing Subsequence | (DP - 41)

We are given an integer array nums.
We need to find the length of the longest subsequence such that the elements are in strictly increasing order.
A subsequence is formed by deleting some elements without changing the order of the remaining elements.

Example:
nums = [10, 9, 2, 5, 3, 7, 101, 18]
One longest increasing subsequence is [2, 3, 7, 101].
Its length is 4, which is the answer.

Approach

Algorithm

We use recursion with memoization to try all possible subsequences.

We define a function that works with two variables:

- ind: current index we are at
- prev_ind: index of the previously taken element in the subsequence
  If no element has been taken yet, prev_ind = -1.

At every index, we have two choices:

- Do not take the current element and move to the next index.
- Take the current element if it is strictly greater than the previously taken element.

To store overlapping subproblems, we use a 2D DP array:

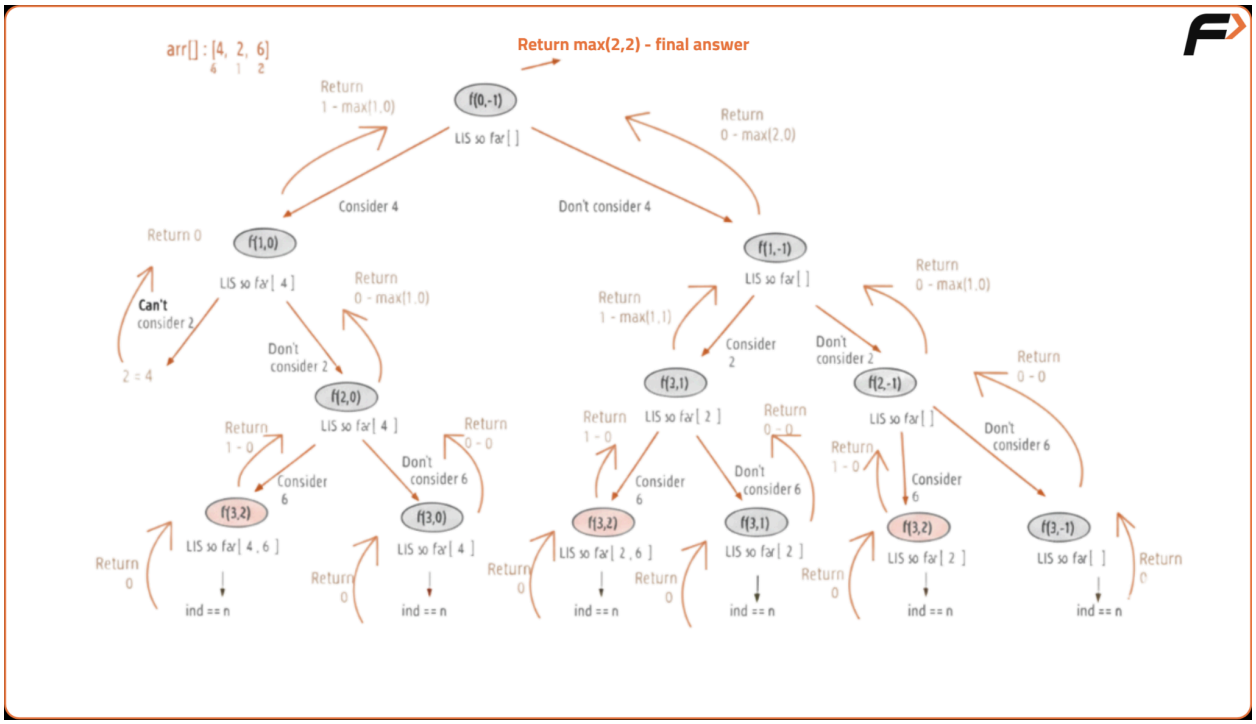
- dp[ind][prev_ind + 1]
  We add 1 to prev_ind because prev_ind can be -1, and array indices cannot be negative.

Base case:

- If we are at the last index, we check if we can take it or not and return accordingly.

The answer is the maximum length obtained by either taking or not taking the current element.

Example dry run (nums = [10, 9, 2, 5, 3, 7, 101, 18]):
We start at index 0 with prev_ind = -1.
We try skipping 10 and also try taking 10.
This process continues for every index, and the DP table ensures we do not recompute states.
The maximum length found is 4.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int func(int i,int prevInd,vector<int> &arr,vector<vector<int>> &dp){
        if(i==arr.size()-1){
            if(prevInd==-1 || arr[prevInd]<arr[i]) return 1;
            return 0;
        }
        if(dp[i][prevInd+1]!=-1) return dp[i][prevInd+1];
        int notTake=func(i+1,prevInd,arr,dp);
        int take=0;
        if(prevInd==-1 || arr[i]>arr[prevInd])
            take=1+func(i+1,i,arr,dp);
        return dp[i][prevInd+1]=max(take,notTake);
    }
```

```
public:
    int LIS(vector<int>& nums){
        int n=nums.size();
        vector<vector<int>> dp(n,vector<int>(n+1,-1));
        return func(0,-1,nums,dp);
    }
};

int main(){
    vector<int> nums={10,9,2,5,3,7,101,18};
    Solution sol;
    cout<<"The length of the LIS for the given array is: "<<sol.LIS(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(n^2)$
There are n possible values for ind and n possible values for prev_ind. Each state is computed once.

Space Complexity: $O(n^2)$
We use a 2D DP array of size n × (n+1).
The recursion stack can go up to O(n), but it is dominated by the DP array.

Printing Longest Increasing Subsequence | (DP - 42)

We are given an array arr of size n.
Our task is to return the **Longest Increasing Subsequence (LIS)** such that:

- The subsequence is strictly increasing.

- If multiple LIS are possible, we return the one that is **index-wise lexicographically smallest**, meaning the elements are chosen as early as possible from the original array.

A subsequence is formed by deleting elements without changing the order.

Example:
arr = [1,3,2,4,6,5]
Possible LIS are [1,3,4,6] and [1,2,4,6].
Since 3 appears earlier than 2 in the array, [1,3,4,6] is chosen.

Approach

Algorithm

We use Dynamic Programming to first compute the length of LIS ending at each index.

Steps:

1. Create a DP array dp[] where dp[i] stores the length of LIS ending at index i.
2. Create another array prev[] to store the previous index of the element used in LIS for reconstruction.
3. Initialize dp[i] = 1 for all i because every element alone is an LIS of length 1.
4. For every index i, check all previous indices j < i:
    - If arr[j] < arr[i] and dp[j] + 1 > dp[i], update dp[i] and set prev[i] = j.
    - This ensures we always pick the earliest valid sequence, giving lexicographically smallest LIS.
5. Find the index with the maximum value in dp[].
6. Reconstruct the LIS by following prev[] pointers backward.
7. Reverse the collected sequence to get the correct LIS order.

Example dry run (arr = [10,22,9,33,21,50,41,60,80]):
The dp array identifies LIS length as 6.

Using prev[], we trace back elements like 80 → 60 → 50 → 33 → 22 → 10.
After reversing, we get [10,22,33,50,60,80].

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> longestIncreasingSubsequence(vector<int> &nums) {
        int n = nums.size();
        vector<int> dp(n,1), prev(n,-1);

        for(int i=1;i<n;i++){
            for(int j=0;j<i;j++){
                if(nums[j]<nums[i] && dp[j]+1>dp[i]){
                    dp[i]=dp[j]+1;
                    prev[i]=j;
                }
            }
        }

        int maxLen=0,lastIndex=0;
        for(int i=0;i<n;i++){
            if(dp[i]>maxLen){
                maxLen=dp[i];
                lastIndex=i;
            }
        }

        vector<int> lis;
        while(lastIndex!=-1){
            lis.push_back(nums[lastIndex]);
            lastIndex=prev[lastIndex];
        }

        reverse(lis.begin(),lis.end());
        return lis;
    }
```

```
};

int main(){
    vector<int> nums={10,22,9,33,21,50,41,60,80};
    Solution sol;
    vector<int> lis=sol.longestIncreasingSubsequence(nums);
    for(int x:lis) cout<<x<<" ";
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N²)
Two nested loops are used to compute the dp and prev arrays.

Space Complexity: O(N)
We use dp[] and prev[] arrays of size N to store LIS length and reconstruction path.

Longest Increasing Subsequence | Binary Search | (DP - 43)

We are given an integer array nums.
Our task is to find the length of the longest subsequence such that all elements are in strictly increasing order.
A subsequence is formed by removing some elements without changing the order of the remaining elements.

Example:
nums = [10, 9, 2, 5, 3, 7, 101, 18]
One longest increasing subsequence is [2, 3, 7, 101].
The length of this subsequence is 4.

Approach

Algorithm

We solve this problem using a greedy approach combined with binary search.

We maintain a temporary array temp.
temp does not store the actual LIS, but it stores the smallest possible ending value of an increasing subsequence of a given length.

Steps:

1. Initialize an empty array temp.
2. Insert the first element of nums into temp.
3. Traverse the array from the second element onward:
    - If the current element is greater than the last element of temp, append it to temp.
    - Otherwise, find the index of the smallest element in temp that is greater than or equal to the current element using binary search (lower_bound), and replace that element.
4. At the end, the size of temp gives the length of the LIS.

Dry run example:
nums = [1, 5, 7, 2, 3]

- Start: temp = [1]
- 5 > 1 → temp = [1, 5]
- 7 > 5 → temp = [1, 5, 7]
- 2 ≤ 7 → replace 5 → temp = [1, 2, 7]
- 3 ≤ 7 → replace 7 → temp = [1, 2, 3]

Length of temp = 3, which is the LIS length.

Binary search helps keep temp minimal, allowing better chances to build longer subsequences later.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int LIS(vector<int>& nums) {
      int n = nums.size();
      vector<int> temp;
```

```
        temp.push_back(nums[0]);
        for(int i=1;i<n;i++){
            if(nums[i]>temp.back()){
                temp.push_back(nums[i]);
            } else {
                int ind = lower_bound(temp.begin(), temp.end(), nums[i]) - temp.begin();
                temp[ind] = nums[i];
            }
        }
        return temp.size();
    }
};

int main(){
    vector<int> nums={10,9,2,5,3,7,101,18};
    Solution sol;
    cout<<"The length of the LIS for the given array is: "<<sol.LIS(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N log N)
For each element, we perform a binary search on the temp array, which takes log N time.

Space Complexity: O(N)
The temp array can grow up to size N in the worst case.

Longest Divisible Subset | (DP - 44)

We are given an array nums of positive integers.
We need to find the largest subset such that for every pair (a, b) in the subset, either a % b == 0 or b % a == 0 holds true.
If multiple valid subsets exist, we can return any one of them.

Example:
nums = [3, 5, 10, 20]
One valid largest divisible subset is [5, 10, 20] because 10 is divisible by 5 and 20 is divisible by 10.

Approach

Algorithm

We solve this problem using Dynamic Programming with reconstruction.

1. First, sort the array in ascending order.
   This helps because if nums[i] is divisible by nums[j] and i > j, then nums[i] can extend the divisible subset ending at j.
2. Create two arrays:
   - dp[i]: length of the longest divisible subset ending at index i.
   - parent[i]: previous index used to form the subset ending at i.
3. Initialize:
   - dp[i] = 1 for all i, since each element alone is a valid subset.
   - parent[i] = i initially.
4. For each index i, check all previous indices prevInd < i:
   - If nums[i] % nums[prevInd] == 0 and dp[prevInd] + 1 > dp[i],
     update dp[i] and set parent[i] = prevInd.
5. While filling dp, keep track of the index where the maximum subset length ends (lastIndex).
6. Backtrack from lastIndex using the parent array to collect the elements of the subset.

Example dry run (nums = [16, 8, 2, 4, 32]):
After sorting → [2, 4, 8, 16, 32]
Each element is divisible by the previous one, so dp becomes [1,2,3,4,5].
Backtracking gives the subset [2, 4, 8, 16, 32].

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   vector<int> largestDivisibleSubset(vector<int> nums) {
      int n = nums.size();
      sort(nums.begin(), nums.end());
```

```
        vector<int> dp(n,1), parent(n);
        int maxLen = 0, lastIndex = 0;

        for(int i=0;i<n;i++){
            parent[i] = i;
            for(int prevInd=0;prevInd<i;prevInd++){
                if(nums[i] % nums[prevInd] == 0 && dp[prevInd] + 1 > dp[i]){
                    dp[i] = dp[prevInd] + 1;
                    parent[i] = prevInd;
                }
            }
            if(dp[i] > maxLen){
                maxLen = dp[i];
                lastIndex = i;
            }
        }

        vector<int> ans;
        int i = lastIndex;
        while(parent[i] != i){
            ans.push_back(nums[i]);
            i = parent[i];
        }
        ans.push_back(nums[i]);

        return ans;
    }
};

int main(){
    vector<int> nums = {3,5,10,20};
    Solution sol;
    vector<int> ans = sol.largestDivisibleSubset(nums);
    for(int x: ans) cout<<x<<" ";
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n²)
We use two nested loops to fill the dp and parent arrays.

Space Complexity: O(n)
We use dp and parent arrays of size n to store subset length and reconstruction path.

Longest String Chain | (DP - 45)

We are given an array of strings words[].
We need to find the length of the longest string chain.

A string chain follows these rules:

- Each next word is formed by inserting exactly one character into the previous word.
- The order of characters must remain the same.
- The first word can be any word from the array.

Example:
words = ["a","ab","abc","abcd","abcde"]
Each word is formed by adding one character to the previous word.
So the longest chain length is 5.

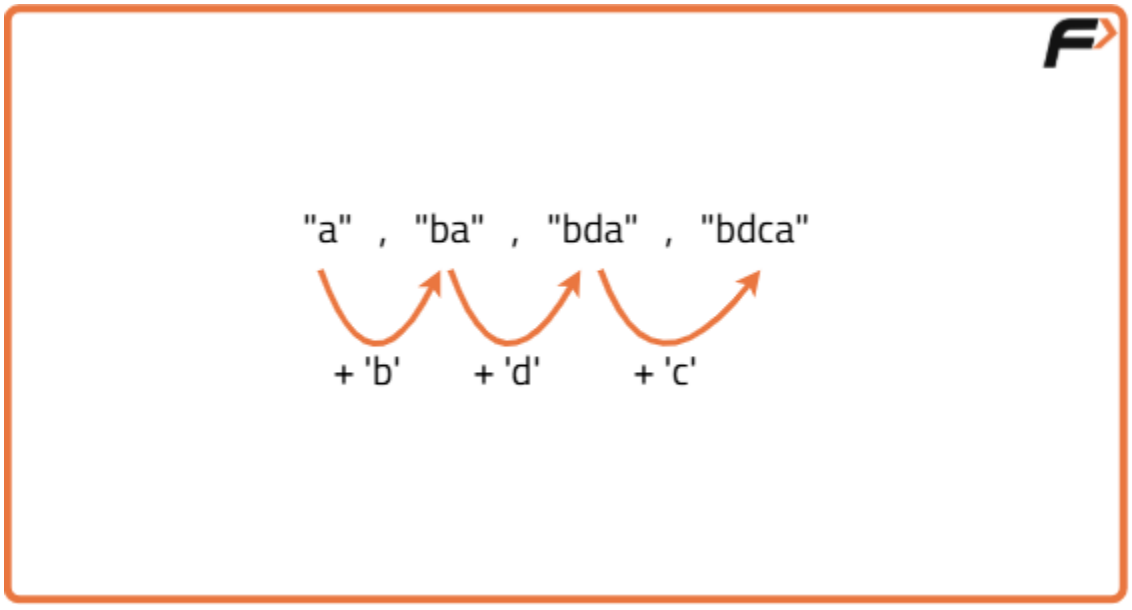
Approach

Algorithm

We use Dynamic Programming to solve this problem.

1. Sort all words based on their length in ascending order.
   This ensures that when we process a word, all possible shorter predecessor words are already processed.

2. Create a DP array dp[] where:
    - dp[i] stores the length of the longest string chain ending at words[i].
    - Initially, dp[i] = 1 because every word alone forms a chain of length 1.
3. For each word at index i, check all previous words j < i:
    - If words[j] can form words[i] by inserting exactly one character, then words[i] can extend the chain ending at j.
    - If dp[j] + 1 > dp[i], update dp[i].
4. To check if a word t is a valid predecessor of s:
    - Length of s must be exactly one more than t.
    - Traverse both strings using two pointers and allow exactly one extra character in s.
5. Keep track of the maximum value in dp[], which is the length of the longest string chain.



Example dry run (words = ["d","do","dot","dots"]):
After sorting → ["d","do","dot","dots"]
Each next word is formed by adding one character.
dp becomes [1,2,3,4].
Maximum chain length = 4.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestStringChain(vector<string>& words) {
        int n = words.size();
        sort(words.begin(), words.end(), compare);
        vector<int> dp(n,1);
        int maxLen = 0;

        for(int i=0;i<n;i++){
            for(int j=0;j<i;j++){
                if(checkPossible(words[i],words[j]) && dp[i]<dp[j]+1){
                    dp[i]=dp[j]+1;
                }
            }
            if(dp[i]>maxLen) maxLen=dp[i];
        }
        return maxLen;
    }

private:
    static bool compare(string &s,string &t){
        return s.size()<t.size();
    }

    bool checkPossible(string &s,string &t){
        if(s.size()!=t.size()+1) return false;
        int i=0,j=0;
        while(i<s.size()){
            if(j<t.size() && s[i]==t[j]){
                i++;j++;
            } else {
                i++;
            }
        }
```

```
        return j==t.size();
    }
};

int main(){
    vector<string> words={"a","ab","abc","abcd","abcde"};
    Solution sol;
    cout<<"The length of the Longest String Chain is:
"<<sol.longestStringChain(words);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n² * m)
For each word, we compare it with all previous words, and each comparison takes up to m time where m is the average word length.

Space Complexity: O(n)
We use a DP array of size n to store the longest chain length for each word.

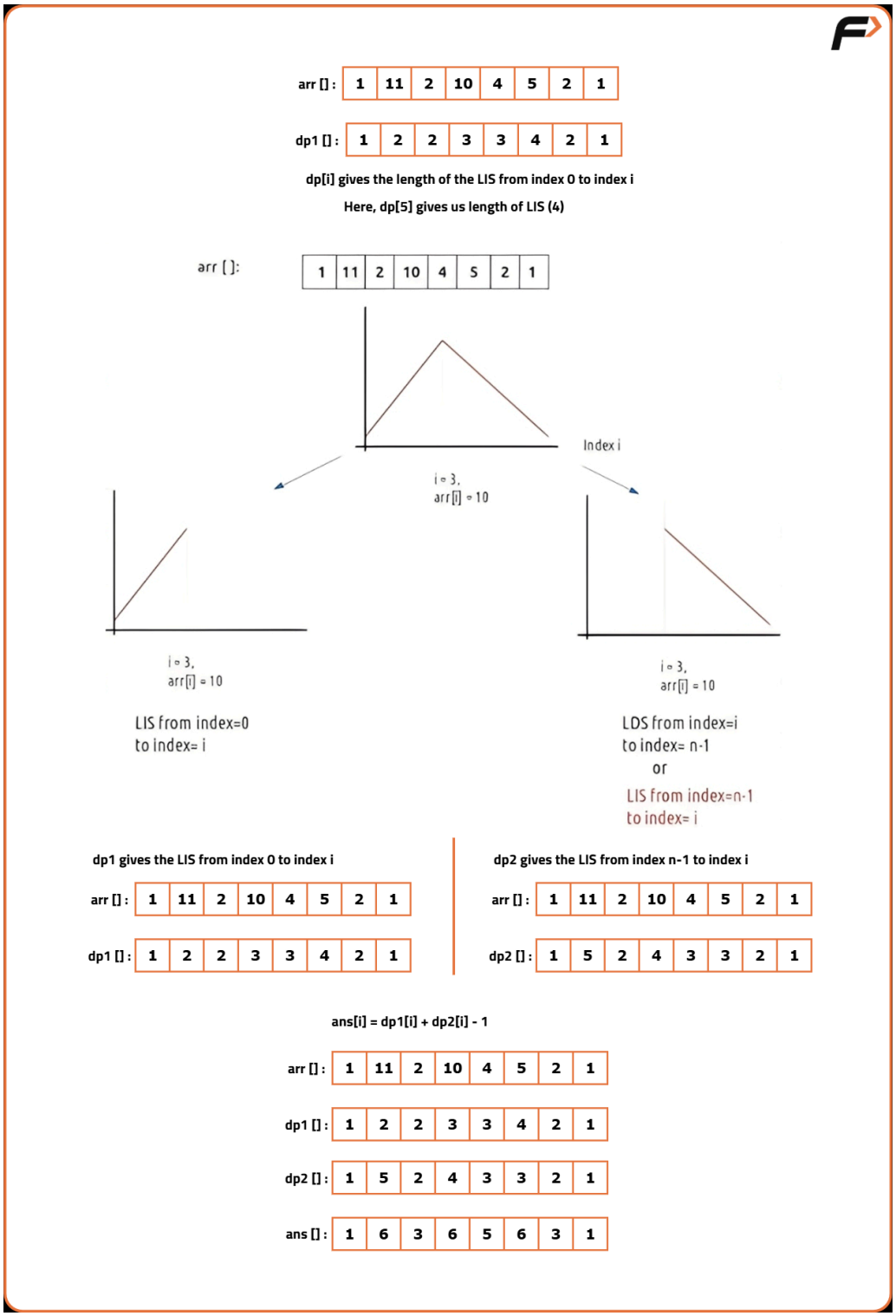
Longest Bitonic Subsequence | (DP - 46)

We are given an array arr of size n.
We need to find the length of the longest **bitonic subsequence**.

A bitonic subsequence is a sequence that:

- First strictly increases
- Then strictly decreases
  The elements do not need to be contiguous, but their order must be preserved.

Example:
arr = [5,1,4,2,3,6,8,7]
One longest bitonic subsequence is [1,2,3,6,8,7].
Its length is 6.



Approach

Algorithm

We solve this problem using Dynamic Programming by combining two sequences.

1. **Longest Increasing Subsequence (LIS)**
   For every index i, compute the length of the longest increasing subsequence ending at i.
   - Initialize LIS_dp[i] = 1 for all i.
   - For each i, check all previous indices prev < i.
   - If arr[prev] < arr[i], update LIS_dp[i] = max(LIS_dp[i], LIS_dp[prev] + 1).
2. **Longest Decreasing Subsequence (LDS)**
   For every index i, compute the length of the longest decreasing subsequence starting at i.
   - Initialize LDS_dp[i] = 1 for all i.
   - Traverse from right to left.
   - For each i, check all next indices prev > i.
   - If arr[prev] < arr[i], update LDS_dp[i] = max(LDS_dp[i], LDS_dp[prev] + 1).
3. **Combine LIS and LDS**
   For each index i, the length of the bitonic subsequence with peak at i is:
   LIS_dp[i] + LDS_dp[i] - 1
   We subtract 1 because arr[i] is counted twice.
4. Track the maximum value obtained from all indices.

Example dry run (arr = [10,20,30,40,50,40,30,20]):
LIS_dp = [1,2,3,4,5,4,3,2]
LDS_dp = [1,2,3,4,5,4,3,2]
Maximum bitonic length = 5 + 5 - 1 = 9?
But correct alignment gives full array length = 8.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int LongestBitonicSequence(vector<int>& arr) {
      int n = arr.size();
      vector<int> LIS_dp(n,1), LDS_dp(n,1);
      int maxLen = 0;
```

```
        // LIS calculation
        for(int i=0;i<n;i++){
            for(int prev=0;prev<i;prev++){
                if(arr[prev] < arr[i] && LIS_dp[i] < LIS_dp[prev] + 1){
                    LIS_dp[i] = LIS_dp[prev] + 1;
                }
            }
        }

        // LDS calculation
        for(int i=n-1;i>=0;i--){
            for(int prev=n-1;prev>i;prev--){
                if(arr[prev] < arr[i] && LDS_dp[i] < LDS_dp[prev] + 1){
                    LDS_dp[i] = LDS_dp[prev] + 1;
                }
            }
            maxLen = max(maxLen, LIS_dp[i] + LDS_dp[i] - 1);
        }

        return maxLen;
    }
};

int main(){
    vector<int> arr = {5,1,4,2,3,6,8,7};
    Solution sol;
    cout << "The length of the Longest Bitonic Sequence is: "
         << sol.LongestBitonicSequence(arr);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n²)
We use two nested loops to compute LIS and LDS.

Space Complexity: O(n)
We use two arrays of size n to store LIS and LDS values.

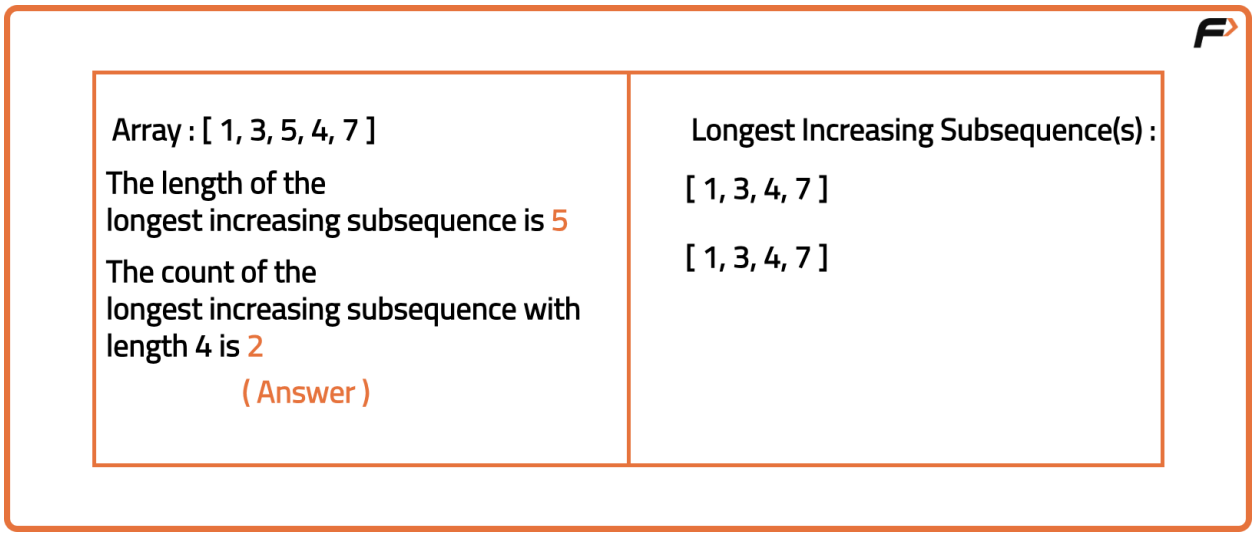
Number of Longest Increasing Subsequences | (DP - 47)

We are given an integer array nums.
Our task is to find how many **Longest Increasing Subsequences (LIS)** exist in the array.

A Longest Increasing Subsequence is a subsequence where:

- Elements are strictly increasing.
- The length of the subsequence is maximum possible.

Example:
nums = [1,3,5,4,7]
The LIS length is 4.
Two LIS exist: [1,3,4,7] and [1,3,5,7].
So the answer is 2.



Approach

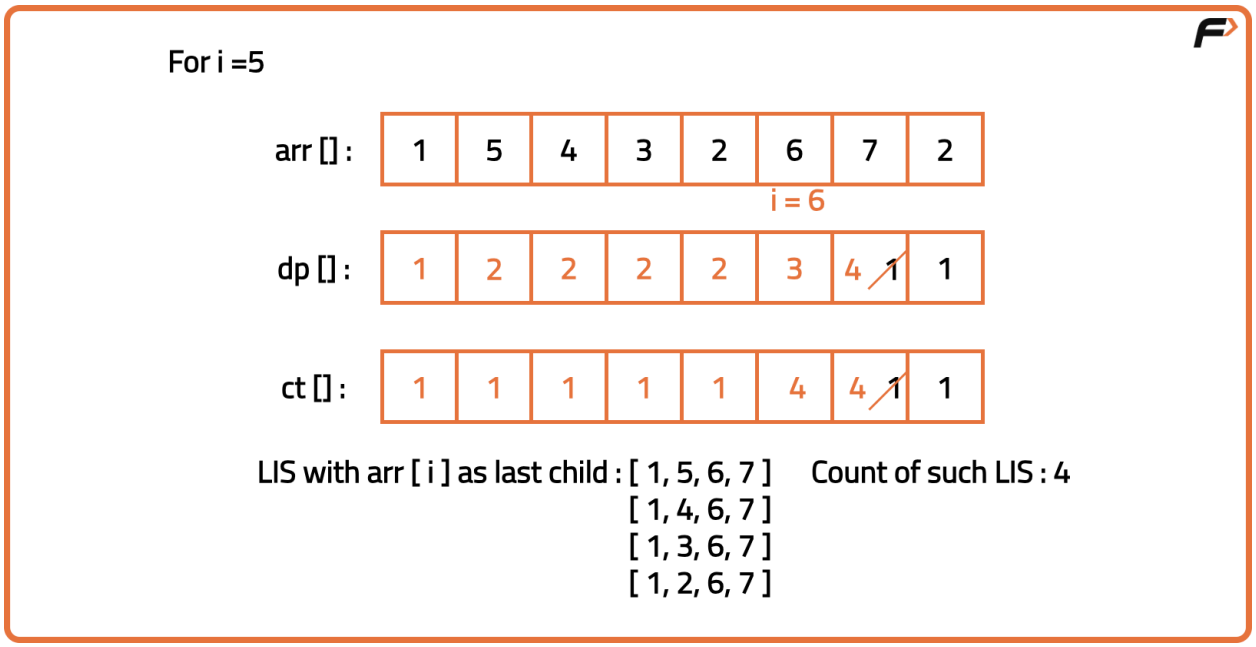
Algorithm

In the normal LIS problem, we only track the length of LIS ending at each index.
Here, we also need to count how many such LIS end at each index.

Steps:

1. Create two arrays:
    - dp[i]: length of LIS ending at index i.
    - ct[i]: number of LIS of length dp[i] ending at index i.
2. Initialize:
    - dp[i] = 1 for all i, because each element alone forms an LIS of length 1.
    - ct[i] = 1 for all i, because there is one such subsequence.
3. For each index i, check all previous indices j < i:
    - If nums[j] < nums[i] and dp[j] + 1 > dp[i]:
        - Update dp[i] = dp[j] + 1.
        - Set ct[i] = ct[j] because we found a longer LIS.
    - Else if nums[j] < nums[i] and dp[j] + 1 == dp[i]:
        - Add ct[j] to ct[i], because another LIS of same length is found.
4. Keep track of the maximum LIS length found.
5. At the end, sum ct[i] for all indices where dp[i] equals the maximum length.

Example dry run (nums = [1,3,5,4,7]):
dp = [1,2,3,3,4]
ct = [1,1,1,1,2]
Maximum LIS length = 4
Total count = 2



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
```

```
public:
    int findNumberOfLIS(vector<int>& arr) {
        int n = arr.size();
        vector<int> dp(n,1), ct(n,1);
        int maxi = 1;

        for(int i=0;i<n;i++){
            for(int j=0;j<i;j++){
                if(arr[j] < arr[i] && dp[j] + 1 > dp[i]){
                    dp[i] = dp[j] + 1;
                    ct[i] = ct[j];
                }
                else if(arr[j] < arr[i] && dp[j] + 1 == dp[i]){
                    ct[i] += ct[j];
                }
            }
            maxi = max(maxi, dp[i]);
        }

        int countLIS = 0;
        for(int i=0;i<n;i++){
            if(dp[i] == maxi){
                countLIS += ct[i];
            }
        }
        return countLIS;
    }
};

int main(){
    vector<int> arr = {1,5,4,3,2,6,7,2};
    Solution sol;
    cout << "The count of Longest Increasing Subsequences (LIS) is "
         << sol.findNumberOfLIS(arr);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N²)
For each element, we compare it with all previous elements.

Space Complexity: O(N)
We use two arrays of size N to store LIS length and count.

Matrix Chain Multiplication | (DP - 48)

We are given an array arr of size n+1.
This array represents n matrices such that:

- Matrix A1 has dimension arr[0] x arr[1]
- Matrix A2 has dimension arr[1] x arr[2]
- …
- Matrix An has dimension arr[n-1] x arr[n]

Our task is to find the **minimum number of scalar multiplications** needed to multiply all matrices together.
The order of multiplication matters because matrix multiplication cost depends on dimensions.

Example:
arr = [40, 20, 30, 10, 30]
Best order is ((A1 × (A2 × A3)) × A4)
Minimum cost = 26000

Approach 1: Recursive Approach

Algorithm

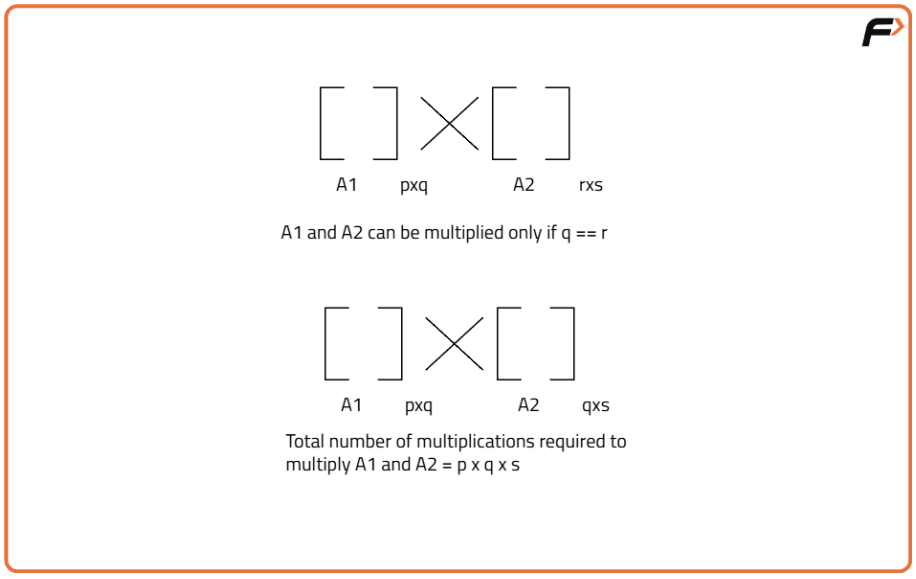
This is a classic **partition DP** problem.

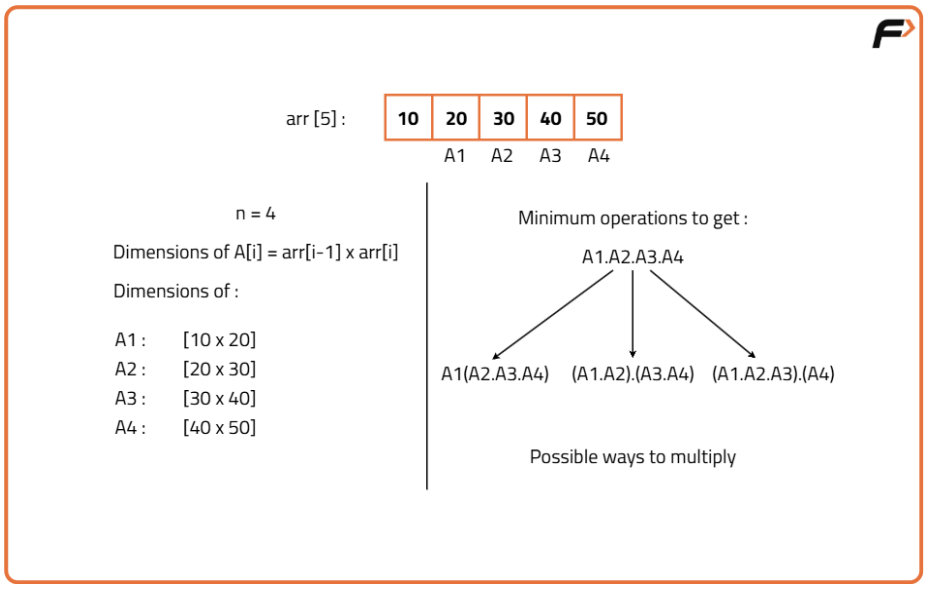
1. We define a function f(i, j) that returns the minimum cost to multiply matrices from Ai to Aj.
2. **Base case**:
    - If i == j, there is only one matrix, so no multiplication is needed. Return 0.
3. **Try all possible partitions**:
    - For k from i to j-1, split the chain into:

- f(i, k)
- f(k+1, j)

4. **Cost calculation**:
    - Cost = f(i, k) + f(k+1, j) + (arr[i-1] * arr[k] * arr[j])
5. Return the minimum cost among all partitions.

Example dry run (arr = [40,20,30,10,30]):
We try all possible k splits and compute cost for each parenthesization.
The minimum among them is returned.





Code

```
#include <bits/stdc++.h>
```

```
using namespace std;

class Solution {
public:
    int matrixChainOrder(vector<int>& arr, int i, int j) {
        if(i == j) return 0;
        int minCost = INT_MAX;
        for(int k = i; k < j; k++) {
            int cost1 = matrixChainOrder(arr, i, k);
            int cost2 = matrixChainOrder(arr, k + 1, j);
            int costMultiply = arr[i - 1] * arr[k] * arr[j];
            int total = cost1 + cost2 + costMultiply;
            minCost = min(minCost, total);
        }
        return minCost;
    }
};

int main() {
    vector<int> arr = {40, 20, 30, 10, 30};
    Solution sol;
    cout << "Minimum number of multiplications is: "
         << sol.matrixChainOrder(arr, 1, arr.size() - 1);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(2^n)$
Every possible parenthesization is explored recursively.

Space Complexity: O(n)
Maximum recursion depth is n.

Approach 2: Memoization (Top-Down DP)

Algorithm

The recursive approach has overlapping subproblems, so we store results.

1. Create a 2D dp array where dp[i][j] stores the minimum cost to multiply matrices from i to j.
2. Initialize dp with -1.
3. Before computing f(i, j), check:
    - If dp[i][j] is already computed, return it.
4. Otherwise, compute using the same recursive logic and store the result in dp[i][j].
5. Final answer is dp[1][n-1].

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(vector<int>& arr, int i, int j, vector<vector<int>>& dp) {
        if(i == j) return 0;
        if(dp[i][j] != -1) return dp[i][j];
        int minCost = INT_MAX;
        for(int k = i; k < j; k++) {
            int cost1 = solve(arr, i, k, dp);
            int cost2 = solve(arr, k + 1, j, dp);
            int costMultiply = arr[i - 1] * arr[k] * arr[j];
            int total = cost1 + cost2 + costMultiply;
            minCost = min(minCost, total);
        }
        return dp[i][j] = minCost;
    }

    int matrixChainOrder(vector<int>& arr) {
        int n = arr.size();
        vector<vector<int>> dp(n, vector<int>(n, -1));
        return solve(arr, 1, n - 1, dp);
    }
};

int main() {
    vector<int> arr = {40, 20, 30, 10, 30};
    Solution sol;
    cout << "Minimum number of multiplications is: "
         << sol.matrixChainOrder(arr);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(n^3)$
There are $O(n^2)$ states (i, j) and for each we try O(n) partitions.

Space Complexity: $O(n^2)$
We use a 2D dp table of size n × n and recursion stack of O(n).

Matrix Chain Multiplication | Tabulation Method | (DP - 49)

We are given an array nums of size n.
This array represents a chain of matrices such that:

- Matrix A1 has dimension nums[0] × nums[1]
- Matrix A2 has dimension nums[1] × nums[2]
- …
- Matrix A(n-1) has dimension nums[n-2] × nums[n-1]

Our task is to find the **minimum number of scalar multiplications** needed to multiply the entire chain of matrices by placing parentheses optimally.

Example:
nums = [10, 15, 20, 25]
Two ways:

- A1 × (A2 × A3) → 10×15×25 + 15×20×25 = 11250
- (A1 × A2) × A3 → 10×15×20 + 10×20×25 = 8000
  Minimum cost = 8000

Approach

We solve this using **tabulation (bottom-up DP)**.

1. Create a 2D DP array dp where:
    - dp[i][j] represents the minimum cost to multiply matrices from Ai to Aj.
2. Initialize:
    - dp[i][i] = 0 because a single matrix needs no multiplication.
3. Fill the table by increasing chain length:
    - Start from chains of length 2 up to n−1.
4. For each pair (i, j), try all possible split points k:
    - Cost = dp[i][k] + dp[k+1][j] + nums[i-1] × nums[k] × nums[j]
5. Store the minimum cost for dp[i][j].
6. Final answer is stored in dp[1][n−1].

Example dry run (nums = [4,2,3]):
Only one multiplication:
Cost = 4 × 2 × 3 = 24
So dp[1][2] = 24.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int matrixMultiplication(vector<int>& nums) {
      int n = nums.size();
      vector<vector<int>> dp(n, vector<int>(n, INT_MAX));

      // Single matrix cost is zero
      for(int i = 1; i < n; i++) {
         dp[i][i] = 0;
      }

      // Chain length from 2 to n-1
      for(int len = 2; len < n; len++) {
         for(int i = 1; i <= n - len; i++) {
            int j = i + len - 1;
            for(int k = i; k < j; k++) {
               int cost = dp[i][k] + dp[k+1][j]
                        + nums[i-1] * nums[k] * nums[j];
               dp[i][j] = min(dp[i][j], cost);
            }
         }
      }
```

```
        return dp[1][n-1];
    }
};

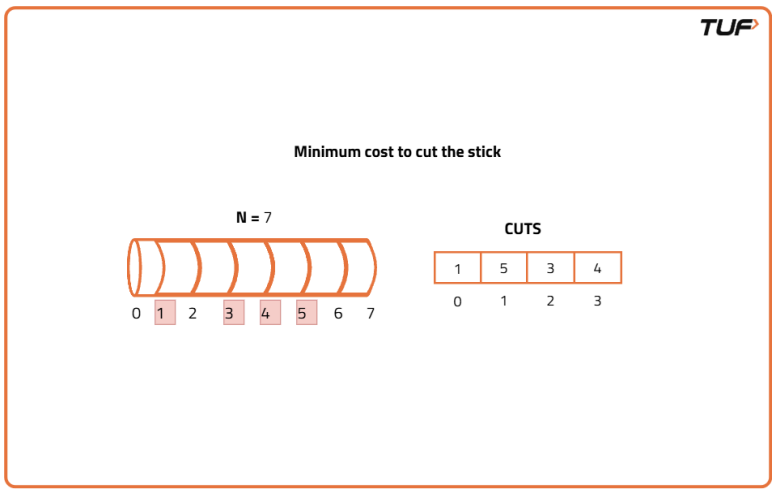
int main() {
    vector<int> nums = {10, 15, 20, 25};
    Solution sol;
    cout << sol.matrixMultiplication(nums);
    return 0;
}
```

Complexity Analysis

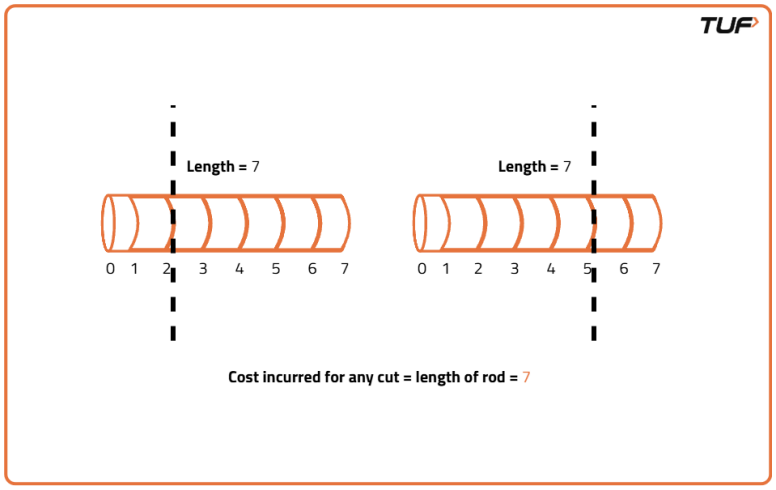
Time Complexity: $O(N^3)$
We use three nested loops: chain length, start index, and split point.

Space Complexity: $O(N^2)$
A 2D DP table is used to store minimum multiplication costs.

Minimum Cost to Cut the Stick | (DP - 50)

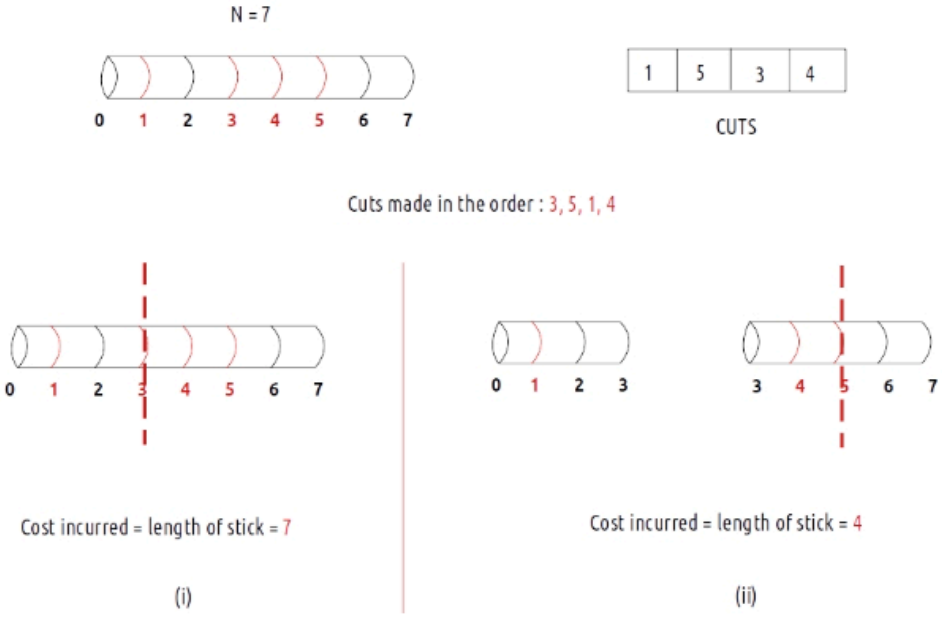


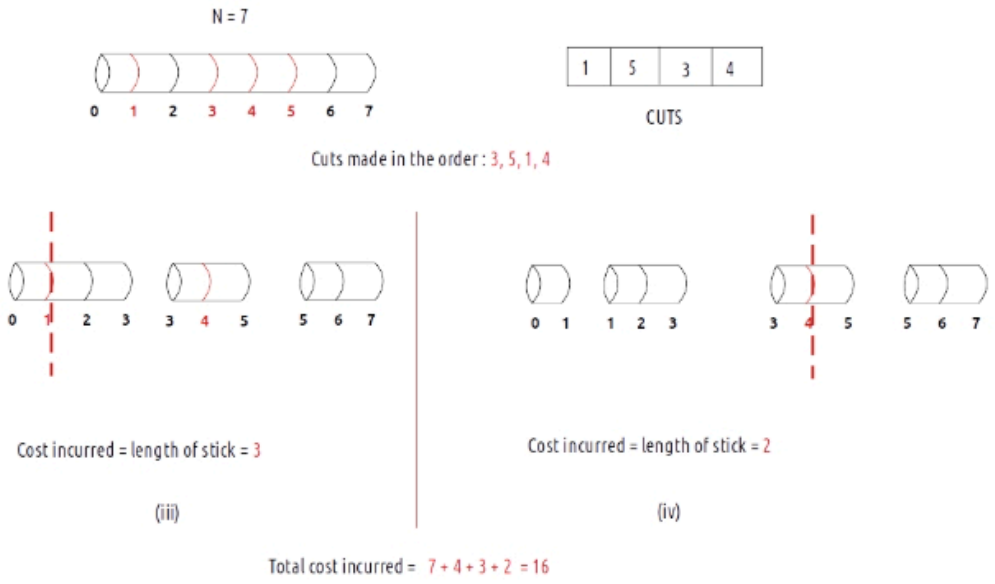
We are given a stick of length n and an array cuts of size c.
Each value in cuts represents a position where the stick must be cut.
Every cut costs the **current length of the stick segment** being cut.



We must perform **all cuts**, but we can choose the order.
The goal is to find the **minimum total cost**.

Example:
n = 7, cuts = [1,3,4,5]
One optimal order gives total cost = 16.





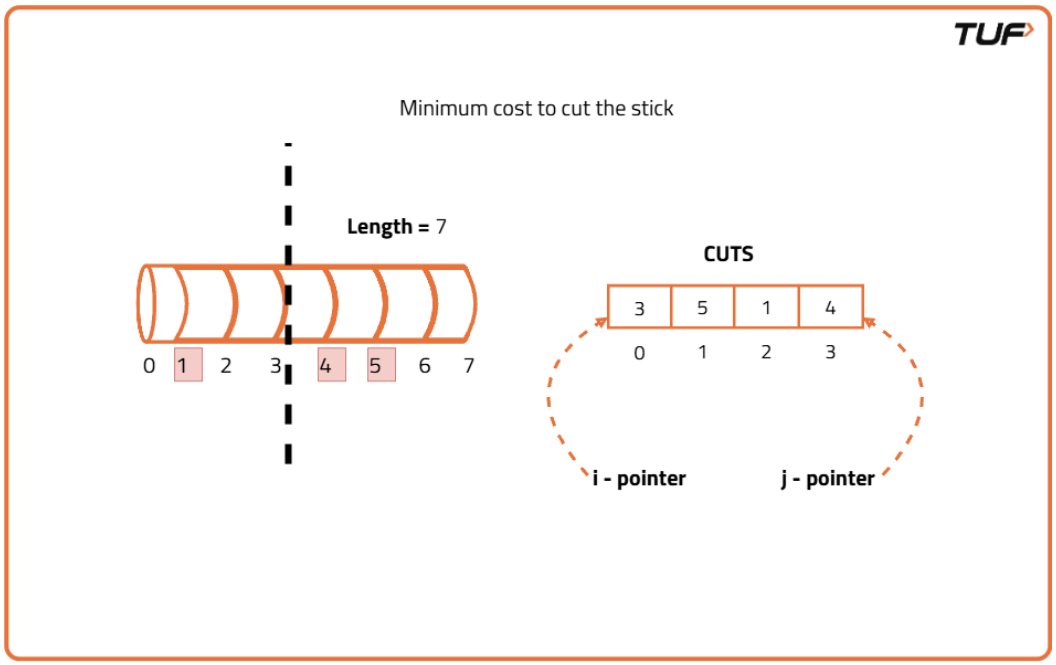
Approach 1: Recursion

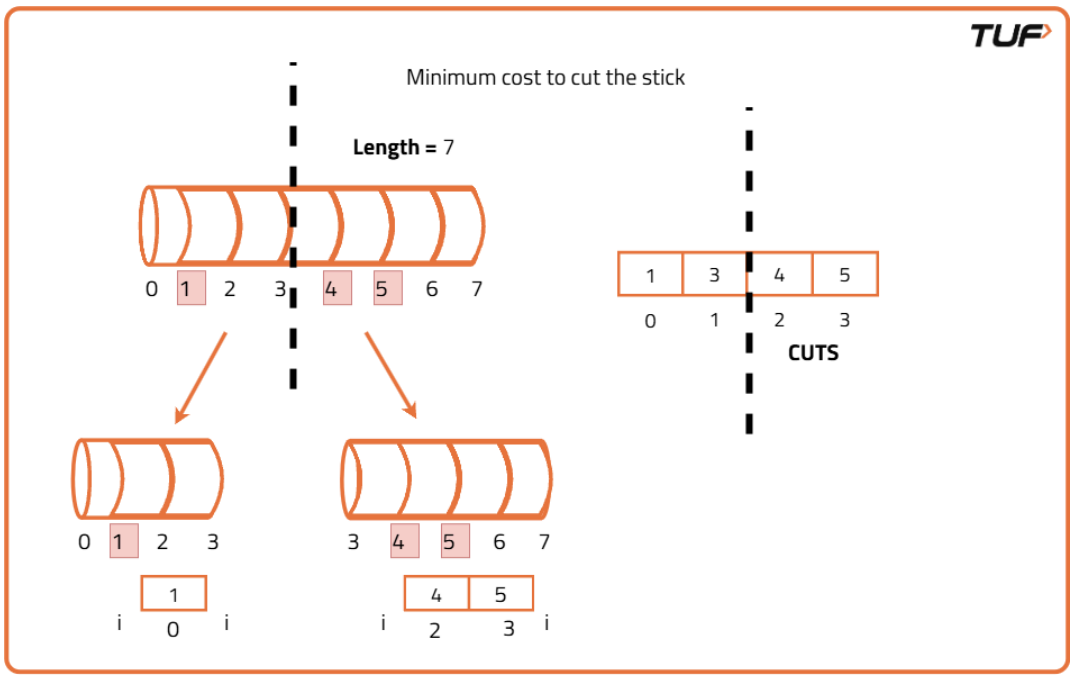
Algorithm

This problem depends on the **order of cuts**, so we use **Partition DP**.

Steps:

1. First sort the cuts array.
2. Add 0 at the start and n at the end to represent stick boundaries.
3. Define a recursive function f(i, j):
    - It returns the minimum cost to cut between cuts[i] and cuts[j].
4. Base case:
    - If i > j, there are no cuts to make, so cost = 0.
5. For every possible cut index ind from i to j:
    - Cost of current cut = cuts[j+1] - cuts[i-1]
    - Add cost of left part: f(i, ind-1)
    - Add cost of right part: f(ind+1, j)
6. Return the minimum cost among all choices.






```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i,int j,vector<int> &cuts){
```

```
        if(i>j) return 0;
        int mini=INT_MAX;
        for(int ind=i;ind<=j;ind++){
            int cost = cuts[j+1]-cuts[i-1]
                    + solve(i,ind-1,cuts)
                    + solve(ind+1,j,cuts);
            mini=min(mini,cost);
        }
        return mini;
    }

    int minimumCost(int n,int c,vector<int> &cuts){
        cuts.push_back(n);
        cuts.insert(cuts.begin(),0);
        sort(cuts.begin(),cuts.end());
        return solve(1,c,cuts);
    }
};

int main(){
    vector<int> cuts={3,5,1,4};
    int n=7;
    Solution sol;
    cout<<sol.minimumCost(n,cuts.size(),cuts);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(2^C)
All possible cut orders are tried recursively.

Space Complexity: O(C)
Recursive stack depth.

Approach 2: Memoization

Algorithm

The recursive solution has overlapping subproblems.
We store results in a DP table to avoid recomputation.

Steps:

1. Use a 2D dp array where dp[i][j] stores minimum cost for cuts i to j.
2. Initialize dp with -1.
3. Before computing f(i,j), check dp[i][j].
4. If already computed, return it.
5. Otherwise compute using recursion and store in dp.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i,int j,vector<int> &cuts,vector<vector<int>> &dp){
        if(i>j) return 0;
        if(dp[i][j]!=-1) return dp[i][j];
        int mini=INT_MAX;
        for(int ind=i;ind<=j;ind++){
            int cost = cuts[j+1]-cuts[i-1]
                    + solve(i,ind-1,cuts,dp)
                    + solve(ind+1,j,cuts,dp);
            mini=min(mini,cost);
        }
        return dp[i][j]=mini;
    }

    int minimumCost(int n,int c,vector<int> &cuts){
        cuts.push_back(n);
        cuts.insert(cuts.begin(),0);
        sort(cuts.begin(),cuts.end());
        vector<vector<int>> dp(c+1,vector<int>(c+1,-1));
        return solve(1,c,cuts,dp);
    }
};

int main(){
    vector<int> cuts={3,5,1,4};
    int n=7;
    Solution sol;
    cout<<sol.minimumCost(n,cuts.size(),cuts);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(C^3)$
There are C^2 states and for each we try C cuts.

Space Complexity: $O(C^2)$
DP table + recursion stack.

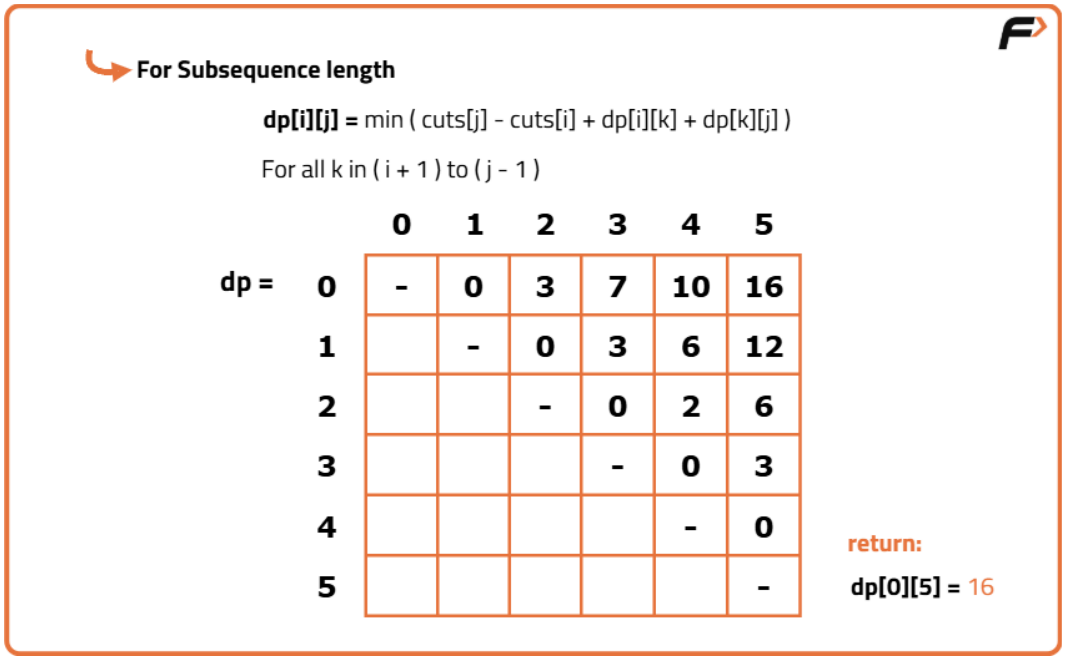
Approach 3: Tabulation

Algorithm

We convert memoization into bottom-up DP.

Steps:

1. Sort cuts and add 0 and n.
2. Create dp table initialized with 0.
3. dp[i][j] represents minimum cost for cuts i to j.
4. Fill dp by increasing segment length:
    - i moves from c to 1
    - j moves from i to c
5. For every possible cut ind between i and j:
    - dp[i][j] = min(dp[i][j],
      cuts[j+1]-cuts[i-1] + dp[i][ind-1] + dp[ind+1][j])
6. Final answer is dp[1][c].



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
   int minimumCost(int n,int c,vector<int> &cuts){
      cuts.push_back(n);
      cuts.insert(cuts.begin(),0);
      sort(cuts.begin(),cuts.end());

      vector<vector<int>> dp(c+2,vector<int>(c+2,0));

      for(int i=c;i>=1;i--){
         for(int j=i;j<=c;j++){
            int mini=INT_MAX;
            for(int ind=i;ind<=j;ind++){
               int cost = cuts[j+1]-cuts[i-1]
                        + dp[i][ind-1]
                        + dp[ind+1][j];
               mini=min(mini,cost);
            }
            dp[i][j]=mini;
         }
      }
      return dp[1][c];
```

```
    }
};

int main(){
    vector<int> cuts={3,5,1,4};
    int n=7;
    Solution sol;
    cout<<sol.minimumCost(n,cuts.size(),cuts);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(C^3)$
Three nested loops over cuts.

Space Complexity: $O(C^2)$
2D DP table used.

Burst Balloons | Partition DP | (DP - 51)

We are given an array arr[] of size n where each element represents a balloon with a number on it.
We need to burst all the balloons one by one.

If we burst the i-th balloon, the coins gained are:
arr[i-1] * arr[i] * arr[i+1]

If i-1 or i+1 goes out of bounds, we treat that value as 1.

Our task is to find the **maximum coins** we can collect by bursting the balloons in an optimal order.

Example:
arr = [3,1,5,8]
One optimal order gives total coins = 167.

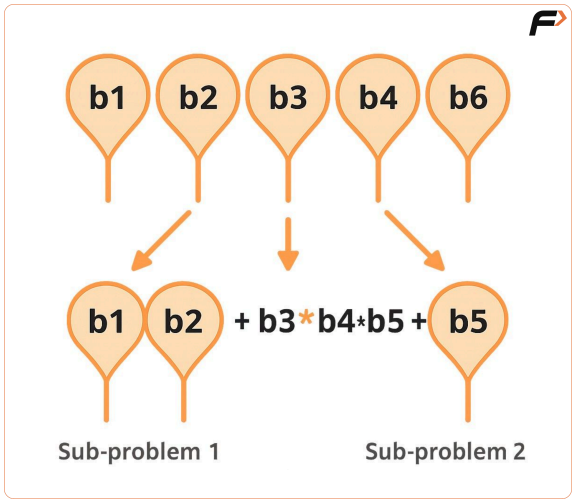
Approach 1: Recursive Approach

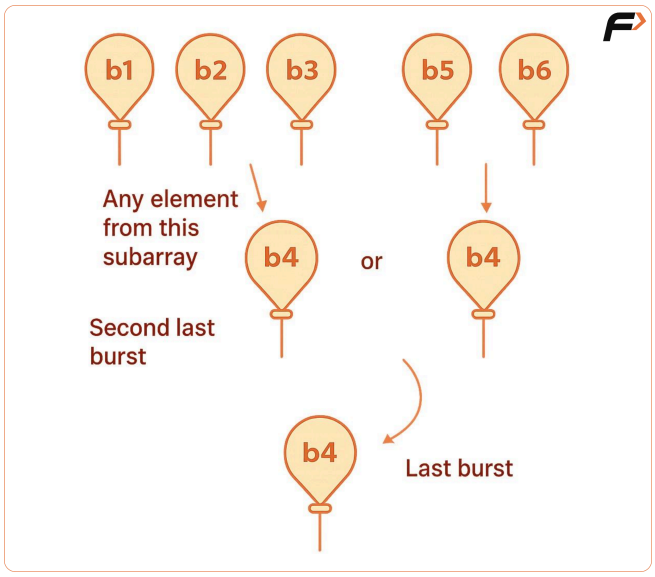
Algorithm

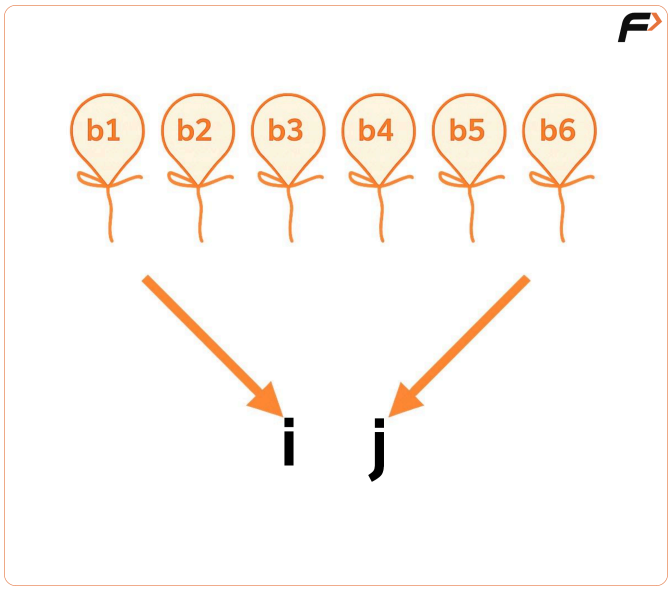
The order of bursting balloons affects the total coins.
Instead of choosing the first balloon to burst, we think in reverse and choose the **last balloon to burst**.

Steps:

1. Add 1 at the beginning and end of the array to handle boundary conditions.
2. Define a recursive function f(i, j) which returns the maximum coins obtainable by bursting balloons from index i to j.
3. Base case:
    - If i > j, no balloons left, return 0.
4. Try every balloon k between i and j as the **last balloon to burst**.
5. Coins gained:
    - nums[i-1] * nums[k] * nums[j+1]
6. Add coins from left part f(i, k-1) and right part f(k+1, j).
7. Return the maximum value among all choices.







Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i,int j,vector<int> &nums){
        if(i>j) return 0;
        int maxi=INT_MIN;
        for(int k=i;k<=j;k++){
            int coins = nums[i-1]*nums[k]*nums[j+1]
                    + solve(i,k-1,nums)
```

```
                    + solve(k+1,j,nums);
            maxi = max(maxi, coins);
        }
        return maxi;
    }

    int maxCoins(vector<int> &nums){
        int n=nums.size();
        nums.insert(nums.begin(),1);
        nums.push_back(1);
        return solve(1,n,nums);
    }
};

int main(){
    vector<int> nums={3,1,5,8};
    Solution sol;
    cout<<"Maximum coins obtained: "<<sol.maxCoins(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: Exponential
All possible bursting orders are explored recursively.

Space Complexity: O(N)
Due to recursion stack.

Approach 2: Memoization

Algorithm

The recursive solution has overlapping subproblems, so we store results in a DP table.

Steps:

1. Create a 2D dp array where dp[i][j] stores maximum coins for range i to j.
2. Initialize dp with -1.
3. Before solving f(i, j), check if dp[i][j] is already computed.
4. If yes, return it.

5. Otherwise, compute using recursion and store the result in dp.

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int solve(int i,int j,vector<int> &nums,vector<vector<int>> &dp){
        if(i>j) return 0;
        if(dp[i][j]!=-1) return dp[i][j];
        int maxi=INT_MIN;
        for(int k=i;k<=j;k++){
            int coins = nums[i-1]*nums[k]*nums[j+1]
                        + solve(i,k-1,nums,dp)
                        + solve(k+1,j,nums,dp);
            maxi = max(maxi, coins);
        }
        return dp[i][j]=maxi;
    }

    int maxCoins(vector<int> &nums){
        int n=nums.size();
        nums.insert(nums.begin(),1);
        nums.push_back(1);
        vector<vector<int>> dp(n+2,vector<int>(n+2,-1));
        return solve(1,n,nums,dp);
    }
};

int main(){
    vector<int> nums={3,1,5,8};
    Solution sol;
    cout<<"Maximum coins obtained: "<<sol.maxCoins(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^3)$
There are N^2 states and for each state we try N partitions.

Space Complexity: $O(N^2) + O(N)$
DP table plus recursion stack.

Approach 3: Tabulation

Algorithm

We convert the memoized solution into bottom-up DP.

Steps:

1. Add 1 at the beginning and end of the array.
2. Create a 2D dp array initialized with 0.
3. dp[i][j] represents maximum coins for bursting balloons from i to j.
4. Fill dp table such that smaller ranges are solved first:
    - i goes from n to 1
    - j goes from 1 to n
5. For each (i, j), try all possible last balloons ind between i and j.
6. Update dp[i][j] with the maximum value.
7. Final answer is dp[1][n].

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxCoins(vector<int> &nums){
        int n=nums.size();
        nums.insert(nums.begin(),1);
        nums.push_back(1);
        vector<vector<int>> dp(n+2,vector<int>(n+2,0));

        for(int i=n;i>=1;i--){
            for(int j=1;j<=n;j++){
                if(i>j) continue;
                int maxi=INT_MIN;
                for(int ind=i;ind<=j;ind++){
                    int coins = nums[i-1]*nums[ind]*nums[j+1]
                                + dp[i][ind-1]
                                + dp[ind+1][j];
                    maxi = max(maxi, coins);
```

```
            }
            dp[i][j]=maxi;
          }
        }
        return dp[1][n];
    }
};

int main(){
    vector<int> nums={3,1,5,8};
    Solution sol;
    cout<<"Maximum coins obtained: "<<sol.maxCoins(nums);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^3)$
Three nested loops over ranges and partitions.

Space Complexity: $O(N^2)$
2D DP table is used.

Evaluate Boolean Expression to True | Partition DP | (DP - 52)

We are given a boolean expression string consisting of:

- Operands: T (true) and F (false)
- Operators: & (AND), | (OR), ^ (XOR)

We need to count the number of ways to parenthesize the expression such that the final result evaluates to **True**.
The answer should be returned modulo **10^9 + 7**.

Example:
expression = "F|T^F"
There are 2 different ways to parenthesize the expression so that it evaluates to True.

Approach 1: Brute Force (Recursive Partition DP)

Algorithm

This is a classic **partition DP** problem where the result depends on how we split the expression.

We define a recursive function:
`f(i, j, isTrue)`
It returns the number of ways to evaluate the substring from index `i` to `j` such that:

- `isTrue = 1` → expression evaluates to True
- `isTrue = 0` → expression evaluates to False

Base cases:

- If `i > j`, return 0 (invalid expression).
- If `i == j`, check if the single character matches the required boolean value.

Recursive step:

- Try all operators between `i` and `j`.
- Split the expression into left and right parts.
- Recursively compute:
    - leftTrue, leftFalse
    - rightTrue, rightFalse
- Combine results based on the operator:
    - & → True only if both sides are True
    - | → True if any side is True
    - ^ → True if exactly one side is True
- Sum all valid combinations.

Code

```
#include <bits/stdc++.h>
using namespace std;

const int mod = 1000000007;
```

```
int solve(int i,int j,int isTrue,string &exp){
    if(i>j) return 0;
    if(i==j){
        if(isTrue) return exp[i]=='T';
        else return exp[i]=='F';
    }
    long long ways=0;
    for(int k=i+1;k<=j-1;k+=2){
        long long lT=solve(i,k-1,1,exp);
        long long lF=solve(i,k-1,0,exp);
        long long rT=solve(k+1,j,1,exp);
        long long rF=solve(k+1,j,0,exp);

        if(exp[k]=='&'){
            if(isTrue) ways=(ways+lT*rT)%mod;
            else ways=(ways+lF*rT+lT*rF+lF*rF)%mod;
        }
        else if(exp[k]=='|'){
            if(isTrue) ways=(ways+lT*rT+lT*rF+lF*rT)%mod;
            else ways=(ways+lF*rF)%mod;
        }
        else{
            if(isTrue) ways=(ways+lT*rF+lF*rT)%mod;
            else ways=(ways+lT*rT+lF*rF)%mod;
        }
    }
    return ways;
}

int evaluateExp(string &exp){
    return solve(0,exp.size()-1,1,exp);
}

int main(){
    string exp="F|T^F";
    cout<<evaluateExp(exp);
    return 0;
}
```

Complexity Analysis

Time Complexity: Exponential
All possible parenthesizations are explored.

Space Complexity: O(N)
Recursion stack depth.

Approach 2: Memoization (Top-Down DP)

Algorithm

The brute force solution has overlapping subproblems.

We use a 3D DP array:
`dp[i][j][isTrue]`
It stores the number of ways to evaluate substring `i` to `j` as True or False.

Before solving any subproblem, we check if it is already computed.

Code

```
#include <bits/stdc++.h>
using namespace std;

const int mod = 1000000007;

int solve(int i,int j,int isTrue,string &exp,vector<vector<vector<int>>> &dp){
    if(i>j) return 0;
    if(i==j){
        if(isTrue) return exp[i]=='T';
        else return exp[i]=='F';
    }
    if(dp[i][j][isTrue]!=-1) return dp[i][j][isTrue];

    long long ways=0;
    for(int k=i+1;k<=j-1;k+=2){
        long long lT=solve(i,k-1,1,exp,dp);
        long long lF=solve(i,k-1,0,exp,dp);
        long long rT=solve(k+1,j,1,exp,dp);
        long long rF=solve(k+1,j,0,exp,dp);

        if(exp[k]=='&'){
            if(isTrue) ways=(ways+lT*rT)%mod;
            else ways=(ways+lF*rT+lT*rF+lF*rF)%mod;
        }
        else if(exp[k]=='|'){
```

```
            if(isTrue) ways=(ways+lT*rT+lT*rF+lF*rT)%mod;
            else ways=(ways+lF*rF)%mod;
        }
        else{
            if(isTrue) ways=(ways+lT*rF+lF*rT)%mod;
            else ways=(ways+lT*rT+lF*rF)%mod;
        }
    }
    return dp[i][j][isTrue]=ways;
}

int evaluateExp(string &exp){
    int n=exp.size();
    vector<vector<vector<int>>> dp(n,vector<vector<int>>(n,vector<int>(2,-1)));
    return solve(0,n-1,1,exp,dp);
}

int main(){
    string exp="F|T^F";
    cout<<evaluateExp(exp);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^3)$
There are $O(N^2 \times 2)$ states, and for each we try $O(N)$ partitions.

Space Complexity: $O(N^2) + O(N)$
DP table and recursion stack.

Approach 3: Tabulation (Bottom-Up DP)

Algorithm

We convert memoization into bottom-up DP.

Steps:

- Create a 3D DP table `dp[i][j][isTrue]`.
- Base case: when `i == j`, directly evaluate T or F.

- Fill DP table for increasing substring lengths.
- Use the same operator combination logic as recursion.
- Final answer is `dp[0][n-1][1]`.

Code

```
#include <bits/stdc++.h>
using namespace std;

const int mod = 1000000007;

int evaluateExp(string &exp){
    int n=exp.size();
    vector<vector<vector<long long>>> dp(n,vector<vector<long long>>(n,vector<long
long>(2,0)));

    for(int i=0;i<n;i++){
        dp[i][i][1]=(exp[i]=='T');
        dp[i][i][0]=(exp[i]=='F');
    }

    for(int len=3;len<=n;len+=2){
        for(int i=0;i+len-1<n;i++){
            int j=i+len-1;
            for(int k=i+1;k<=j-1;k+=2){
                for(int isTrue=0;isTrue<=1;isTrue++){
                    long long lT=dp[i][k-1][1];
                    long long lF=dp[i][k-1][0];
                    long long rT=dp[k+1][j][1];
                    long long rF=dp[k+1][j][0];

                    if(exp[k]=='&'){
                        if(isTrue) dp[i][j][1]=(dp[i][j][1]+lT*rT)%mod;
                        else dp[i][j][0]=(dp[i][j][0]+lF*rT+lT*rF+lF*rF)%mod;
                    }
                    else if(exp[k]=='|'){
                        if(isTrue) dp[i][j][1]=(dp[i][j][1]+lT*rT+lT*rF+lF*rT)%mod;
                        else dp[i][j][0]=(dp[i][j][0]+lF*rF)%mod;
                    }
                    else{
                        if(isTrue) dp[i][j][1]=(dp[i][j][1]+lT*rF+lF*rT)%mod;
                        else dp[i][j][0]=(dp[i][j][0]+lT*rT+lF*rF)%mod;
                    }
                }
            }
```

```
        }
    }
    return dp[0][n-1][1];
}

int main(){
    string exp="F|T^F";
    cout<<evaluateExp(exp);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^3)$
Triple nested loops over substring length, partition, and boolean state.

Space Complexity: $O(N^2)$
3D DP table used.

Palindrome Partitioning – II | Front Partition | (DP - 53)

We are given a string s.
We need to partition the string such that **every substring is a palindrome**.
Our task is to return the **minimum number of cuts** needed to achieve this.

A cut means dividing the string into two parts.

Example:
s = "aab"
One valid partition is: aa | b
Only 1 cut is needed, so the answer is 1.

Approach 1: Memoization (Front Partition)

Algorithm

This problem is solved using the **front partition technique**.

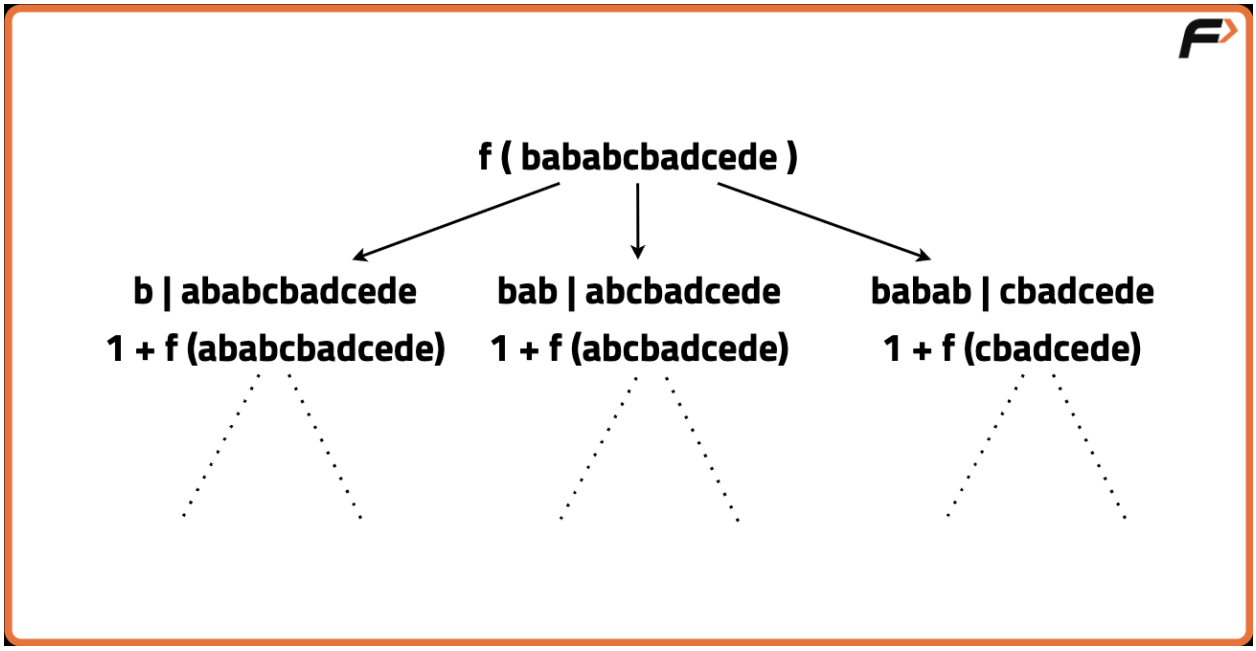
We start partitioning the string from the first index and move forward.

Steps:

1. Define a function `f(start)` which returns the minimum cuts needed for substring `s[start ... end]`.
2. Base case:
    - If `start` reaches the end of the string, return 0.
    - If the substring `s[start ... end]` itself is a palindrome, return 0.
3. From index `start`, try all possible ending indices `end`.
4. If substring `s[start ... end]` is a palindrome:
    - Make 1 cut and recursively solve for `end + 1`.
5. Take the minimum among all possible cuts.
6. Use a memo array `dp[start]` to store results and avoid recomputation.

Example dry run (s = "aab"):

- From index 0:
    - "a" is palindrome → 1 + f(1)
    - "aa" is palindrome → 1 + f(2)
- Best choice gives minimum cuts = 1.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    bool isPalindrome(const string& s,int i,int j){
        while(i<j){
            if(s[i]!=s[j]) return false;
            i++; j--;
        }
        return true;
    }

    int solve(const string& s,int start,vector<int>& dp){
        int n=s.size();
        if(start==n) return 0;
        if(isPalindrome(s,start,n-1)) return 0;
        if(dp[start]!=-1) return dp[start];

        int mini=INT_MAX;
        for(int end=start;end<n;end++){
            if(isPalindrome(s,start,end)){
                int cuts=1+solve(s,end+1,dp);
                mini=min(mini,cuts);
            }
```

```
        }
        return dp[start]=mini;
    }

public:
    int minCut(string s){
        int n=s.size();
        vector<int> dp(n,-1);
        return solve(s,0,dp);
    }
};

int main(){
    string s="aab";
    Solution sol;
    cout<<"Minimum cuts needed: "<<sol.minCut(s);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^2)$
There are N states, and for each state we check palindrome substrings.

Space Complexity: O(N) + O(N)
DP array plus recursion stack.

Approach 2: Tabulation (Bottom-Up)

Algorithm

We convert the memoization approach into a bottom-up DP.

Steps:

1. Define a DP array `dp[i]` where:
    - `dp[i]` = minimum cuts needed for substring `s[i ... end]`.
2. Base case:
    - `dp[n] = -1` (no cut needed after the last character).
3. Fill DP from right to left.
4. For each index `i`, try all substrings `s[i ... j]`.

5. If `s[i ... j]` is a palindrome:
    - `dp[i] = min(dp[i], 1 + dp[j+1])`
6. Final answer is `dp[0]`.

Example dry run (s = "aab"):

- dp[3] = -1
- dp[2] = 0 ("b")
- dp[1] = 1 ("a|b")
- dp[0] = 1 ("aa|b")

Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
   bool isPalindrome(const string& s,int i,int j){
      while(i<j){
         if(s[i]!=s[j]) return false;
         i++; j--;
      }
      return true;
   }

public:
   int minCut(string s){
      int n=s.size();
      vector<int> dp(n+1,0);
      dp[n]=-1;

      for(int i=n-1;i>=0;i--){
         int mini=INT_MAX;
         for(int j=i;j<n;j++){
            if(isPalindrome(s,i,j)){
               mini=min(mini,1+dp[j+1]);
            }
         }
         dp[i]=mini;
      }
      return dp[0];
   }
};
```

```
int main(){
    string s="aab";
    Solution sol;
    cout<<"Minimum cuts needed: "<<sol.minCut(s);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N²)
For each index, all substrings are checked.

Space Complexity: O(N)
Only a DP array is used.

Partition Array for Maximum Sum | Front Partition | (DP - 54)

We are given an integer array `arr` of length `n` and an integer `k`.
We can partition the array into one or more **contiguous sub-arrays**, where each sub-array length is between 1 and `k`.

After making a partition:

- Every element in a sub-array is replaced by the **maximum value** of that sub-array.
- The final array is only used to calculate the total sum.

Our task is to return the **maximum possible sum** after performing exactly one such partition-and-replace operation.

Example:
arr = [1,15,7,9,2,5,10], k = 3
One optimal partition is: [1,15,7 | 9 | 2,5,10]

After replacement: [15,15,15,9,10,10,10]
Sum = 84

Approach 1: Memoization (Front Partition)

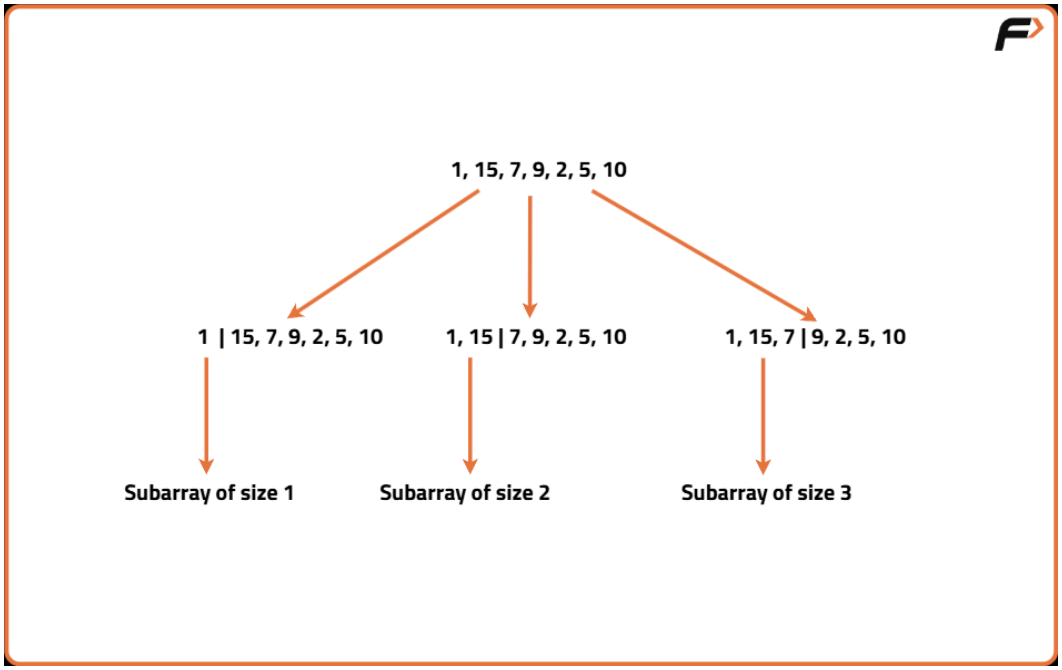
Algorithm

This is a **front partition DP** problem where we decide partitions starting from the first index.

1. Define a recursive function `f(start)` that returns the maximum sum obtainable from index `start` to the end.
2. Base case:
    - If `start == n`, return 0 (no elements left).
3. Use a DP array `dp[start]` to store the result for each starting index.
4. From index `start`, try all sub-arrays of length 1 to k (without crossing array bounds).
5. While extending the sub-array, keep track of the maximum element.
6. For each valid length:
    - Current contribution = `maxElement * length`
    - Total sum = current contribution + `f(start + length)`
7. Take the maximum among all choices and store it in `dp[start]`.

Example dry run (arr = [2,2,2,2], k = 2):

- From index 0:
    - Take [2] → 2*1 + f(1)
    - Take [2,2] → 2*2 + f(2)
- Best choice gives total sum = 8.



Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
private:
    int solve(vector<int>& arr,int k,int start,vector<int>& dp){
        int n = arr.size();
        if(start == n) return 0;
        if(dp[start] != -1) return dp[start];

        int maxSum = 0;
        int maxElem = 0;

        for(int len = 1; len <= k && start + len <= n; len++){
            maxElem = max(maxElem, arr[start + len - 1]);
            int cur = maxElem * len + solve(arr, k, start + len, dp);
            maxSum = max(maxSum, cur);
        }
        return dp[start] = maxSum;
    }

public:
    int maxSumAfterPartitioning(vector<int>& arr,int k){
        int n = arr.size();
```

```
        vector<int> dp(n, -1);
        return solve(arr, k, 0, dp);
    }
};

int main(){
    vector<int> arr = {1,15,7,9,2,5,10};
    int k = 3;
    Solution sol;
    cout << sol.maxSumAfterPartitioning(arr, k);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*K)
There are N states, and for each state we try up to K partition lengths.

Space Complexity: O(N) + O(N)
DP array plus recursion stack.

Approach 2: Tabulation (Bottom-Up)

Algorithm

We convert the memoization solution into an iterative DP.

1. Create a DP array `dp` of size `n+1` where:
    - `dp[i]` = maximum sum obtainable from index `i` to the end.
2. Base case:
    - `dp[n] = 0`
3. Fill `dp` from right to left (from `n-1` to `0`).
4. For each index `i`:
    - Try all sub-arrays of length 1 to `k`.
    - Track the maximum element in the current sub-array.
    - Update `dp[i] = max(dp[i], maxElem * length + dp[i+length])`
5. Final answer is `dp[0]`.

Code

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int maxSumAfterPartitioning(vector<int>& arr,int k){
        int n = arr.size();
        vector<int> dp(n+1, 0);

        for(int i = n-1; i >= 0; i--){
            int maxElem = 0;
            int maxSum = 0;

            for(int len = 1; len <= k && i + len <= n; len++){
                maxElem = max(maxElem, arr[i + len - 1]);
                maxSum = max(maxSum, maxElem * len + dp[i + len]);
            }
            dp[i] = maxSum;
        }
        return dp[0];
    }
};

int main(){
    vector<int> arr = {1,15,7,9,2,5,10};
    int k = 3;
    Solution sol;
    cout << sol.maxSumAfterPartitioning(arr, k);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N*K)
For each index, we check up to K partition lengths.

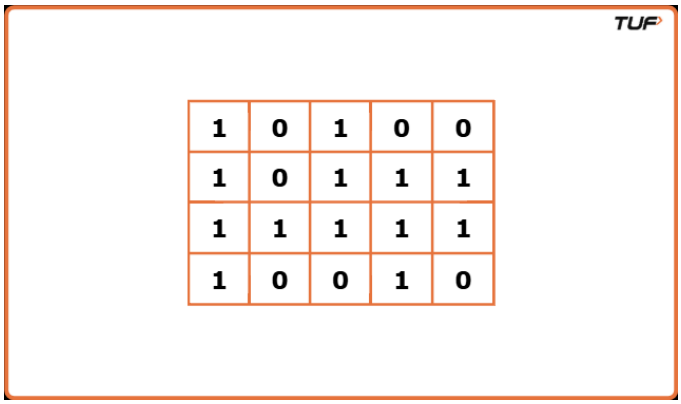
Space Complexity: O(N)
Only the DP array is used.

Maximum Rectangle Area with all 1's | DP on Rectangles | (DP - 55)

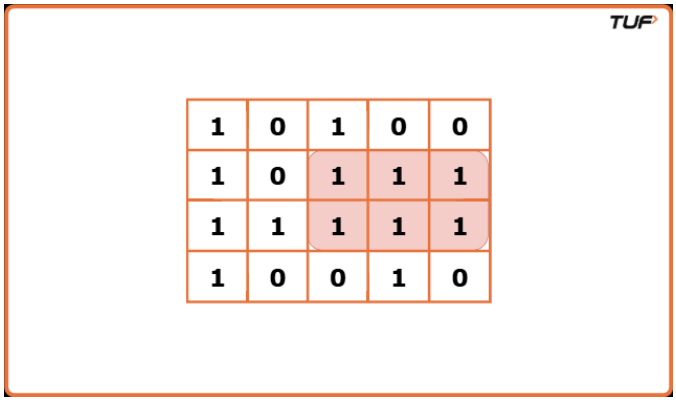
We are given a binary matrix of size m × n containing only 0's and 1's.
We need to find the **largest rectangular area** that contains only 1's and return its area.

A rectangle must be formed using **contiguous rows and contiguous columns**.

Example:
matrix =
[[1,0,1,0,0],
[1,0,1,1,1],
[1,1,1,1,1],
[1,0,0,1,0]]



The maximum rectangle of only 1's has area = 6.



Approach

Algorithm

This problem is solved by **converting each row of the matrix into a histogram** and then applying the **Largest Rectangle in Histogram** logic.

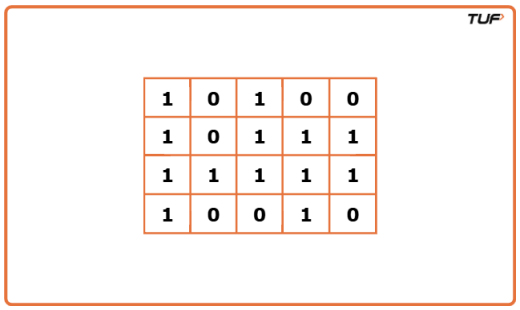
Steps:

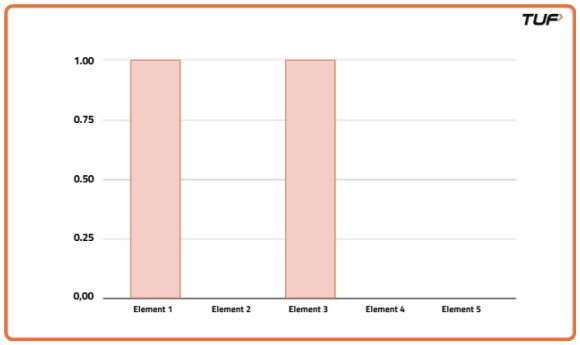
1. Create an array `height[]` of size equal to the number of columns.
2. Traverse the matrix row by row.
3. For each cell:
    - If the value is `1`, increase the height of that column by 1.
    - If the value is `0`, reset the height of that column to 0.
4. After processing a row, the `height[]` array represents a histogram.
5. Compute the **largest rectangle area in this histogram** using a stack-based approach.
6. Keep track of the maximum area across all rows.
7. The final maximum is the answer.

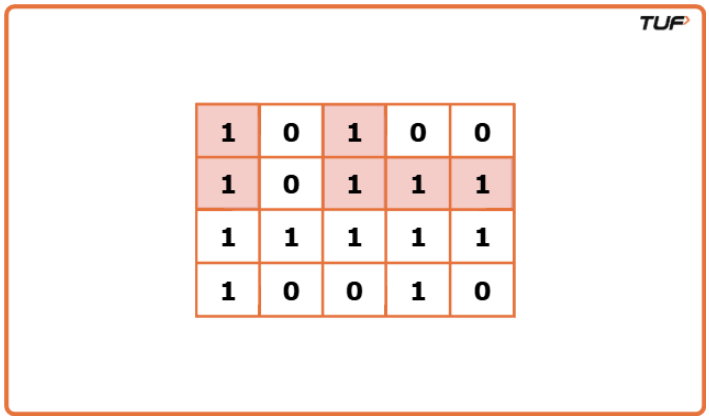
Example dry run (for first two rows):

- Row 1 → height = [1,0,1,0,0]
- Row 2 → height = [2,0,2,1,1]

Each of these histograms is processed to get the largest rectangle area.







Code

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        stack<int> st;
        int maxArea = 0;

        heights.push_back(0); // sentinel

        for(int i = 0; i < heights.size(); i++) {
            while(!st.empty() && heights[i] < heights[st.top()]) {
                int h = heights[st.top()];
                st.pop();
                int w = st.empty() ? i : i - st.top() - 1;
                maxArea = max(maxArea, h * w);
            }
            st.push(i);
        }
        return maxArea;
    }

    int maximalRectangle(vector<vector<char>>& matrix) {
        if(matrix.empty()) return 0;

        int m = matrix[0].size();
        vector<int> height(m, 0);
        int maxArea = 0;
```

```
        for(auto &row : matrix) {
            for(int i = 0; i < m; i++) {
                if(row[i] == '1') height[i]++;
                else height[i] = 0;
            }
            maxArea = max(maxArea, largestRectangleArea(height));
        }
        return maxArea;
    }
};

int main() {
    vector<vector<char>> matrix = {
        {'1','0','1','0','0'},
        {'1','0','1','1','1'},
        {'1','1','1','1','1'},
        {'1','0','0','1','0'}
    };

    Solution sol;
    cout << sol.maximalRectangle(matrix);
    return 0;
}
```

Time Complexity: O(N * M)
For each row (N), we process the histogram in O(M).

Space Complexity: O(M)
Used for the height array and stack.

Count Square Submatrices with All 1s | DP on Rectangles | (DP - 56)

We are given a binary matrix of size n × m that contains only 0s and 1s.
Our task is to count how many **square submatrices** exist such that **all elements inside the square are 1**.

A square submatrix means:

- Same number of rows and columns
- All values inside are 1

Example:
matrix =
[[0,1,1,1],
[1,1,1,1],
[0,1,1,1]]

There are:

- 10 squares of size 1
- 4 squares of size 2
- 1 square of size 3
  Total = 15

Approach

Algorithm

Instead of checking every possible square (which is slow), we use Dynamic Programming.

We create a DP matrix where:

- dp[i][j] = size of the **largest square of all 1s** that ends at cell (i, j) as the bottom-right corner.

Steps:

1. Create a DP matrix of the same size as the input matrix.
2. First row and first column:
    - dp[i][j] = matrix[i][j]
    - Because a square larger than size 1 cannot be formed there.
3. For remaining cells:
    - If matrix[i][j] == 0 → dp[i][j] = 0
    - If matrix[i][j] == 1 →
      dp[i][j] = 1 + min(
      dp[i-1][j], // top
      dp[i][j-1], // left
      dp[i-1][j-1] // top-left
)
4. Why this works:
    - A square of size k can only be formed if all three neighboring squares can form at least size k-1.
5. Every dp[i][j] contributes dp[i][j] squares:
    - If dp[i][j] = 3, it means there are squares of size 1, 2, and 3 ending at that cell.
6. Sum all values of the DP matrix to get the answer.

Code

```
#include <bits/stdc++.h>
using namespace std;

int countSquares(int n, int m, vector<vector<int>> &arr) {
```

```
    vector<vector<int>> dp(n, vector<int>(m, 0));

    // First row
    for(int j = 0; j < m; j++)
        dp[0][j] = arr[0][j];

    // First column
    for(int i = 0; i < n; i++)
        dp[i][0] = arr[i][0];

    // Fill DP table
    for(int i = 1; i < n; i++) {
        for(int j = 1; j < m; j++) {
            if(arr[i][j] == 0)
                dp[i][j] = 0;
            else
                dp[i][j] = 1 + min(dp[i-1][j],
                                   min(dp[i][j-1], dp[i-1][j-1]));
        }
    }

    // Count total squares
    int total = 0;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            total += dp[i][j];
        }
    }

    return total;
}

int main() {
    vector<vector<int>> arr = {
        {0,1,1,1},
        {1,1,1,1},
        {0,1,1,1}
    };
```

```
    int n = 3, m = 4;
    cout << "The number of squares: " << countSquares(n, m, arr);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(n × m)
Each cell is processed exactly once.

Space Complexity: O(n × m)
A DP table of the same size as the matrix is used.